



Università degli studi di Pavia

Dipartimento di Ingegneria Industriale e dell'Informazione

Applied Robotics Technologies Project 2025/2026:

“Trajectory Planning and Dynamic Modelling of a 4-DOF Robot for Automated Helmet Visor Manipulation”

Assigned by: **Prof. Hermes Giberti**

Curated by:

Gianluca Broglia [ID 557998]

Davide Faiola [ID 563969]

Sergi Goldsmits Ybarra [ID 554495]

Group: **D**

Academic year: **2025/2026**

Contents

Project abstract	2
1. Kinematics	4
Robot kinematic model.....	4
Direct Kinematics	4
Inverse Kinematics	5
Jacobian matrix	5
Workspace and Kinematics quality.....	6
1. Workspace mapping and quality	6
2. Kinematics check.....	8
2. Dynamics	10
Robot dynamic model.....	10
Motors sizing	12
3. Task 1	14
Motion planning.....	14
Dynamics and Motor Torque.....	19
Simscape	22
4. Task 2	25
Motion Planning.....	25
Dynamics and Motor Torque.....	31
Simscape	34
5. Lab activities	37
6. EOAT design and manufacturing	39
Critical design characteristics	40
Strength and fatigue resistance:	40
Shape of the end effector:	40
Joining to the robot.....	41
Available tolerancing and surface quality	42
Safety for operators.....	42
7. Conclusions	43
8. MATLAB codes	44
Kinematics	44
Dynamics	52
Task 1	57
Task 2.....	70

Project abstract

The project assigned consists in designing a 4DOF robot according to the kinematics structure shown in the following figure:

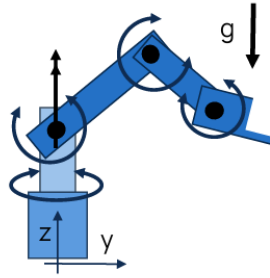


Figure 1

In the first part of the project, it is required to develop a software in MATLAB able to solve the **kinematics** and **dynamics** of the robot, receiving as input all its parameters. Moreover, it is asked to map the "quality" of the robot's kinematics in the workspace using a kinematics index.

In the second part, the software must perform the following tasks:

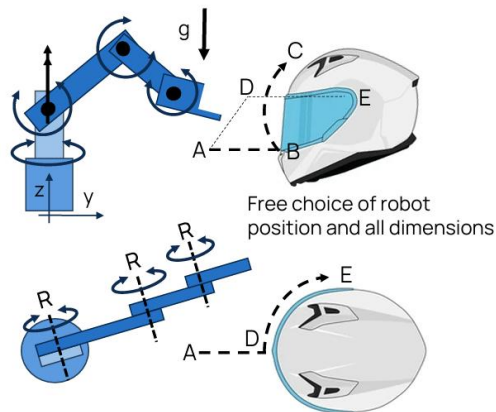


Figure 2

- **The first task** consists in performing the A-B-C-B-A path using the “*Lines&Parabolas*” method in order to open and close the helmet’s visor.
- **The second task** consists in performing the trajectory A-D-E-D-A in order to check the durability of the visor hinges and take a picture of them; the method chosen for the motion planning is the “*trajectory sampling method in operational space*”.

For both the tasks, it is required to compute **the motor torque** of all the motors.

In the third part of the work, a **Simscape Multibody** model of the robot is developed. The purpose of this model is to perform the two trajectories, receiving as input the same robot and tasks parameters of the previous parts. At the end, the **MATLAB** results in terms of motor torques are compared to the Simscape ones and it is checked the consistency among them in order to validate the dynamic model of the robot.

The last part of the project introduces to a practical activity on a real robot. The objective stays in planning the trajectory by means of the software **TMflow** and perform the first task (opening and closing the helmet visor) using one of the **Techman TM** robots (figure 3) available in the Industrial Robotics Laboratory of the University. Moreover, it is necessary to design a EOAT (end-effector), print it using a 3D printer available in the lab and mount it on the TM robot's tool flange.



Figure 3

1. Kinematics

Robot kinematic model

The robot considered is a 4DoF anthropomorphic manipulator made up of four links and four revolute joints. The kinematic structure is schematized as in the figure below:

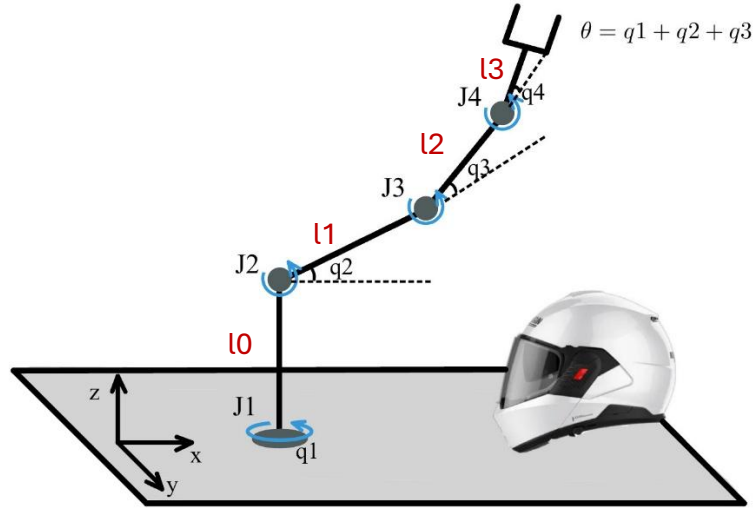


Figure 4

As shown in the picture, the joint J1 rotates around the Z-axis, while J2, J3 and J4 rotate around axes parallel to Y-axis. The reference system X-Y-Z is chosen like depicted in the figure and it is consistent with the MATLAB standard one.

The links lengths (l_0 , l_1 , l_2 , l_3) are parametrized such that they can be changed in order to adapt this model to a set of robots with the same kinematic structure.

Direct Kinematics

The direct kinematics of the robot is computed using a geometric approach. In practice, it provides the end-effector pose by summing the horizontal and vertical contributions of each link. Joint q_1 rotates the planar arm around the vertical axis, generating the (x, y) coordinates. Joints q_2 , q_3 , and q_4 form a planar chain whose cumulative angles determine the radial distance and height:

$$R = l_1 \cos q_2 + l_2 \cos (q_2 + q_3) + l_3 \cos (q_2 + q_3 + q_4)$$

So, the S-coordinates are computed as:

$$x = R \cos q_1$$

$$y = R \sin q_1$$

$$z = l_0 + l_1 \sin q_2 + l_2 \sin (q_2 + q_3) + l_3 \sin (q_2 + q_3 + q_4)$$

$$\theta = q_2 + q_3 + q_4.$$

Inverse Kinematics

The inverse Kinematics of the robot is solved by using a geometric method. Calling p_x, p_y, p_z and θ respectively the S-coordinates in the workspace, the procedure starts with the calculation of the first joint coordinate q_1 , which is obtained directly from the horizontal projection of the desired end-effector position:

$$q_1 = \text{atan2}(p_y, p_x)$$

Once q_1 is fixed, the inverse kinematics problem is reduced to a planar one. It is sufficient to apply the “**wrist method**”, which consists in:

- removing the last joint (l_3) contribution, in order to catch the wrist, point position:

$$R_{target} = \sqrt{p_x^2 + p_y^2} \quad ; \quad R_{wrist} = R_{target} - l_3 \cos(\theta)$$

$$z_{wrist} = p_z - l_0 - l_3 \sin(\theta)$$

- applying the *law of cosines* in order to find the q_3 coordinate, which provides two solutions based on the sin sign:

$$\cos(q_3) = \frac{R_{wrist}^2 + z_{wrist}^2 - l_1^2 - l_2^2}{2l_1l_2}$$

$$\sin(q_3) = \pm \sqrt{1 - \cos^2(q_3)}$$

$$q_3 = \text{atan2}(\sin(q_3), \cos(q_3))$$

- finding q_2 coordinate by decomposing the angle between the shoulder-wrist segment and link l_1 :

$$q_{2z} = \text{atan2}(z_{wrist}, R_{wrist}) \quad ; \quad q_{2R} = \text{atan2}(l_2 \sin(q_3), l_1 + l_2 \cos(q_3))$$

$$q_2 = q_{2z} - q_{2R}$$

- computing finally q_4 by difference between θ and q_3, q_2 :

$$q_4 = \theta - q_2 - q_3$$

Jacobian matrix

The **Jacobian matrix** represents the differential relationship between the joint velocities of the robot and the linear and angular velocities of the end-effector. For a manipulator with configuration $Q = [q_1, q_2, q_3, q_4]^T$, the Jacobian $J(Q)$ must satisfy the following equation:

$$\dot{S} = J(Q) \dot{Q}$$

where $\dot{S}$ contains the translational and rotational velocities of the end-effector.

The Jacobian plays a fundamental role in the project for different reasons; indeed, it is crucial in determining **kinematic singularities**, evaluating the **quality of kinematics** and constructing the **dynamic model** as can be seen in the next sections.

In the project case, the dimension of J is 4×4 , since both the S and Q vector are of dimension 4.

$$J(Q) = \begin{bmatrix} \partial x / \partial q_1 & \partial x / \partial q_2 & \partial x / \partial q_3 & \partial x / \partial q_4; \\ \partial y / \partial q_1 & \partial y / \partial q_2 & \partial y / \partial q_3 & \partial y / \partial q_4; \\ \partial z / \partial q_1 & \partial z / \partial q_2 & \partial z / \partial q_3 & \partial z / \partial q_4; \\ \partial \theta / \partial q_1 & \partial \theta / \partial q_2 & \partial \theta / \partial q_3 & \partial \theta / \partial q_4 \end{bmatrix}$$

Furthermore, in order to get the acceleration profiles of the End - Effector, it is useful to compute the time derivative of $J(Q)$ and write the following expression:

$$\ddot{S} = J(Q) \ddot{Q} + \dot{J}(Q) \dot{Q}$$

This equation shows in which terms the end effector acceleration depends on the joints accelerations and velocities. The expression of the **time derivative of the Jacobian** is shown below:

$$\dot{J}(Q) = \begin{bmatrix} d/dt (\partial x / \partial q_1) & d/dt (\partial x / \partial q_2) & d/dt (\partial x / \partial q_3) & d/dt (\partial x / \partial q_4) \\ d/dt (\partial y / \partial q_1) & d/dt (\partial y / \partial q_2) & d/dt (\partial y / \partial q_3) & d/dt (\partial y / \partial q_4) \\ d/dt (\partial z / \partial q_1) & d/dt (\partial z / \partial q_2) & d/dt (\partial z / \partial q_3) & d/dt (\partial z / \partial q_4) \\ d/dt (\partial \theta / \partial q_1) & d/dt (\partial \theta / \partial q_2) & d/dt (\partial \theta / \partial q_3) & d/dt (\partial \theta / \partial q_4) \end{bmatrix}$$

Workspace and Kinematics quality

1. Workspace mapping and quality

The first goal is to determine the robot's reachable workspace. To do this, it is important to sample the four joint variables q_1, q_2, q_3, q_4 within their mechanical limits. Each sampled joint vector:

$$Q = [q_1 \ q_2 \ q_3 \ q_4]^T$$

is fed into the **direct kinematics equation**, which computes the corresponding end-effector pose:

$$S = \text{ROBOTdir_4R}(Q)$$

Only configurations with a non-negative Z -coordinate (above the base plane) are kept. All valid end-effector positions (X, Y, Z) are stored and later plotted to generate the 3D reachable workspace of the robot.

The workspace can be divided into “**primary workspace**” and “**secondary workspace**.” The primary workspace is defined as the subset of the reachable workspace where the robot exhibits “**good**” **kinematic conditioning**, meaning that the Jacobian has a favourable singular value ratio:

$$\kappa^{-1} = \frac{\sigma_{\min}}{\sigma_{\max}}$$

A configuration is included in the primary workspace if:

$$\kappa^{-1} \geq \kappa_{\text{threshold}}$$

Typically, a threshold such as **0.2** is used. The primary workspace ensures high manipulability and dexterity for high-precision tasks, fast movements, and manipulation requiring fine control.

The **secondary workspace** includes all other reachable points where:

$$\kappa^{-1} < \kappa_{\text{threshold}}$$

These configurations are still reachable, but present “**poor**” **kinematic quality**.

The workspace structure is depicted in the picture below:

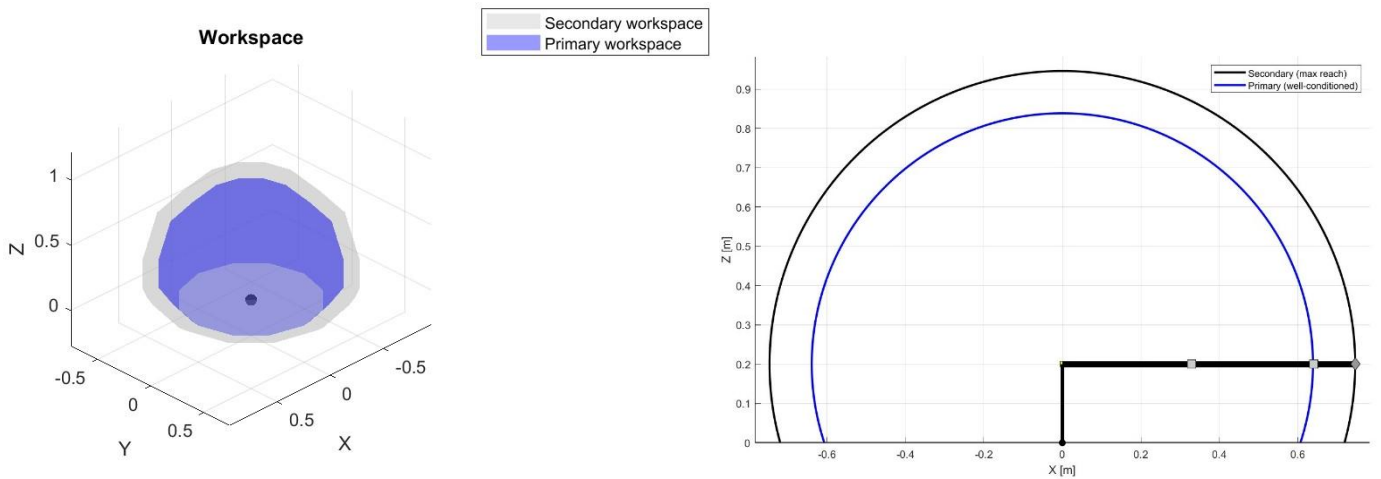


Figure 5

The “well-conditioned” workspace is better represented in the following figure, in which the quality analysis is performed considering several points of the workspace:

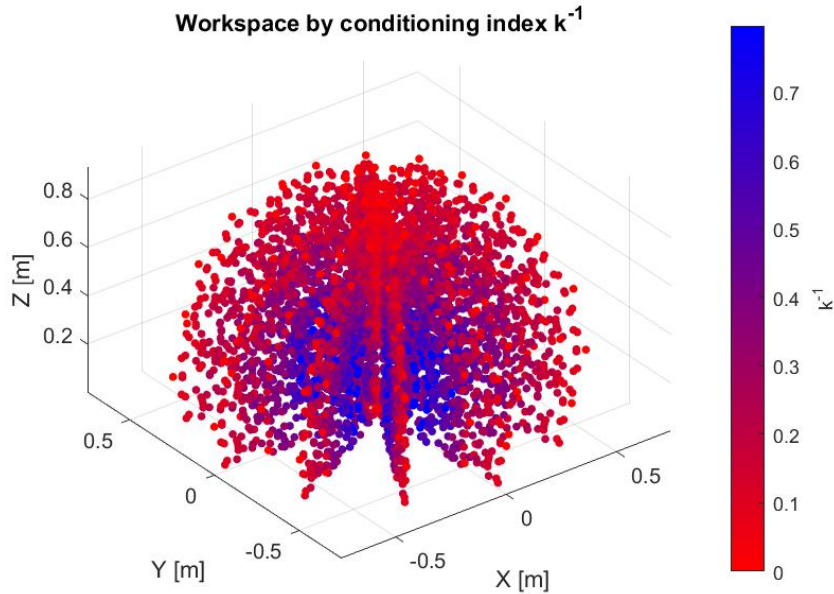


Figure 6

2. Kinematics check

Finally, it is evaluated the **desired end-effector pose**:

$$S = [x \ y \ z \ \theta]^T$$

Two inverse kinematics solutions are computed:

- elbow-up
- elbow-down

Before accepting a solution, the script checks:

- **Geometric reachability:**

$$\sqrt{x^2 + y^2 + (z - l_0)^2} \leq l_1 + l_2 + l_3$$

- **Joint limits feasibility:** each joint value must lie within its mechanical range.

If both checks pass, the direct kinematics is used to verify that the inverse kinematics calculation is consistent. For each IK solution, the Jacobian is recomputed and the same conditioning index k^{-1} is printed to compare the quality of the two configurations.

This step ensures that the robot can achieve the desired pose within its **physical constraints** and evaluates which of the two IK branches is more mechanically advantageous.

In the figure below it is shown an example:

```
S = [0.300; 0.100; 0.400; deg2rad (-10)]; % desired EE pose
```

```
=== Quality index (conditioning) ===
```

```
Solution 1 (elbow-up): k^-1 = 0.5526
```

```
Solution 2 (elbow-down): k^-1 = 0.4437
```

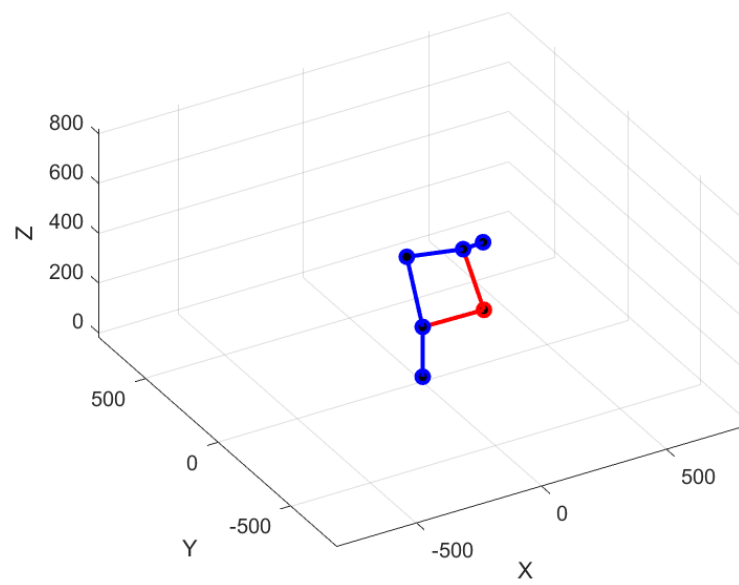


Figure 7: Elbow-down (red), Elbow-up (blue)

2. Dynamics

Robot dynamic model

The dynamic model of the robot is based on the standard formulation of **Newton–Euler dynamics**, extended to account for all link masses, inertias, and the payload. To build the model, each rigid body of the robot (base, link 1, link 2, link 3, and payload) is represented by:

- the mass m
- the location of its centre of mass g
- the relevant inertia tensor's components I

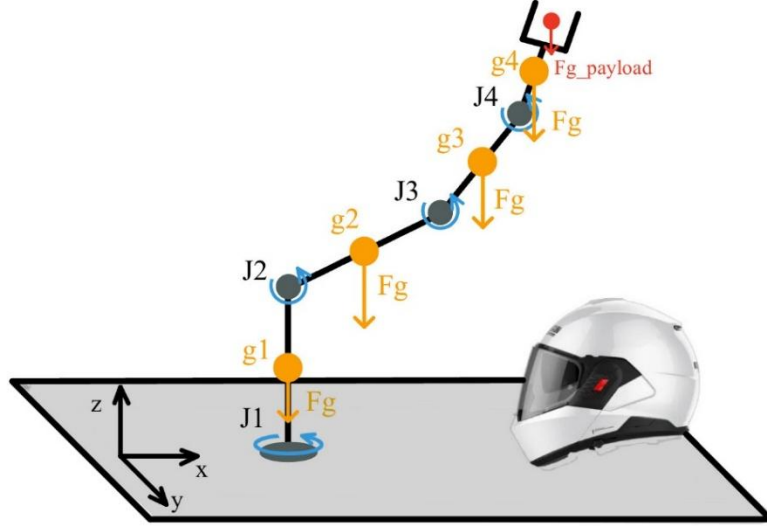


Figure 8

These quantities are collected into a **19×19 diagonal mass matrix**:

$$M = \begin{bmatrix} m_p & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & m_p & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & m_p & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & m_3 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & m_3 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & m_3 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & I_3 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & m_2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & m_2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & m_2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & I_2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & m_1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & m_1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & m_1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & I_1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & m_0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & m_0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & m_0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & I_0 \end{bmatrix}$$

In which m_p, m_3, m_2, m_1, m_0 are respectively the masses of the payload, the third, the second, the first link and of the base.

I_3, I_2, I_1, I_0 are respectively the inertia contributions of the links computed around their rotation axes:

$$I_{0,z} = \frac{1}{2} m_0 r_0^2$$

$$I_{\{1,yz\}} = \left(\frac{1}{4}\right) m_1 r_{\{123\}}^2 + \left(\frac{1}{12}\right) m_1 l_1^2$$

$$I_{\{2,yz\}} = \left(\frac{1}{4}\right) m_2 r_{\{123\}}^2 + \left(\frac{1}{12}\right) m_2 l_2^2$$

$$I_{\{3,yz\}} = \left(\frac{1}{4}\right) m_3 r_{\{123\}}^2 + \left(\frac{1}{12}\right) m_3 l_3^2$$

Moreover, an **external forces vector** is computed which includes **gravity contributions** of each link and payload and could include some **extra forces** if needed (as the reaction force of the helmet visor in task 1). Every force is located in the right position according to the mass matrix as in the example below:

```
Fse (2) = -m_payload*9.81; % Fz payload
Fse (5) = -m3*9.81;       % Fz m3
Fse (9) = -m2*9.81;       % Fz m2
Fse (13) = -m1*9.81;      % Fz m1
Fse (17) = -m0*9.81;      % Fz m0
```

Considering the kinematic model already described before, in order to include the dynamics, it is needed an extended version of the kinematic functions. Therefore, the following functions are included:

- **ROBOTdirdin_4R.m**
- **ROBOTjacdin_4R.m**
- **ROBOTjacPdin_4R.m**

Using the extended Jacobian J_{din} and its derivative $\dot{J}_{din}$, the acceleration of the full extended model is computed as:

$$\ddot{S}_d = \dot{J}_{din}(Q, \dot{Q}) \dot{Q} + J_{din}(Q) \ddot{Q}$$

This acceleration includes the motion of:

- the payload
- the centre of mass of each link
- the rotational dynamics of each body

From this, the **inertial forces** are computed:

$$F_{si} = -M \ddot{S}_d$$

The **total operational forces** are:

$$F_s = F_{se} + F_{si}$$

where F_{se} is the external forces vector and F_{si} is the inertial forces one. Finally, **joint torques** are obtained by projecting the operational forces back into joint space using the transpose of the dynamic Jacobian. So, for each joint:

$$\tau_{joint} = -J_{din}^T F_s$$

Motors sizing

The torques obtained from the dynamic model of the robot correspond to the forces required at each joint to follow the planned trajectory. However, these torques are not delivered directly by the motors, because each actuator is connected to its corresponding link through a **gear transmission** with ratio:

$$rid = \frac{1}{100}$$

However, the **maximum and the minimum reductions** are computed for each task and, in order to check if the choice of the reductor is adequate, it has to remain between the bounds:

$$\tau_{min}, \tau_{max} = \sqrt{J_m} \cdot \frac{\sqrt{\alpha - \beta + 4\dot{\omega}_{r,q} C_{r,q}^*} \pm \sqrt{\alpha - \beta}}{2C_{r,q}^*}$$

In which:

- $J_m = 0.247 \cdot 10^{-4} \text{ Kg} \cdot \text{m}^2$
- $\alpha = \frac{C_n^2}{J_m}$ is the **accelerating factor** of the selected motor
- $C_n = 0,64 \text{ Nm}$ is the **nominal torque** of the selected motor
- $\beta = \left(\frac{rid}{\sqrt{J_m}} \cdot C_{r,q}^* - \frac{\sqrt{J_m}}{rid} \cdot \omega_{r,q} \right)^2 + 2 \cdot (C_{r,q}^* \cdot \dot{\omega}_{r,q} + (\dot{\omega}, C_r^*)_{medio})$ is the **load factor** that depends on the dynamics, the motor's characteristics and the chosen reduction

For each task, a **sizing check** is performed a posteriori. If $\alpha > \beta$ it means that the selected motors are adequate for performing the tasks properly and safely.

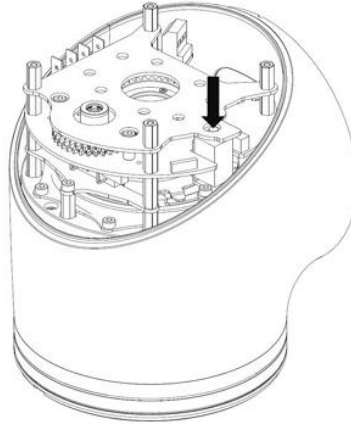


Figure 9: section of a TM5 robot's motor

The reduction is chosen based on the most common reduction ratios used for robotic arms in order to ensure high precision in terms of motion. Anyway, the reduction can be changed in the code to observe the behaviour of the manipulator if subject to a different motor configuration.

In this configuration, the motor rotates faster and delivers less torque, while the gearbox reduces speed and amplifies torque at the joint. To obtain the actual torque that each motor must generate, both the transmission ratio and the motor's rotor inertia must be taken into account.

The **motor torque** for each joint is therefore computed as:

$$\tau_{\text{motor}} = rid \cdot \tau_{\text{joint}} + J_m \cdot \frac{\ddot{\theta}_{\text{joint}}}{rid}$$

where:

- τ_{joint} is the joint torque obtained from the robot dynamics,
- J_m is the rotor inertia of the motor,
- rid is the gear transmission ratio,
- $\ddot{\theta}_{\text{joint}}$ is the joint angular acceleration.

This combined expression is used to compute the complete motor torque profiles for all joints throughout the task execution, ensuring proper motor sizing and allowing a direct comparison with the torques obtained via the *Simscape Multibody* model. Obviously, the maximum velocity and acceleration of the motors are selected depending on the task. This will be described more precisely in the dedicated chapter.

3. Task 1

Motion planning

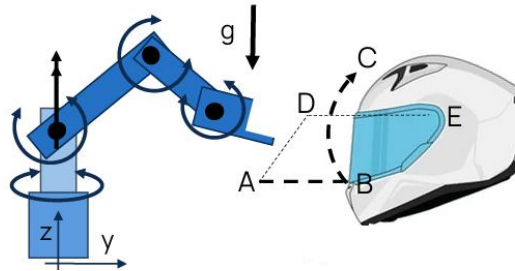


Figure 10

The first task consists in performing the A – B – C – B – A trajectory in order to open and close the helmet's visor. The approach used to plan the motion is the **Lines & Parabolas** method.

The *Lines & Parabolas* method is a time-scaling technique used to generate smooth joint trajectories while respecting kinematic constraints on velocity and acceleration. For each joint, the motion between two consecutive waypoints is composed of constant-acceleration **parabolic blends** connected by a **linear segment** with constant velocity.

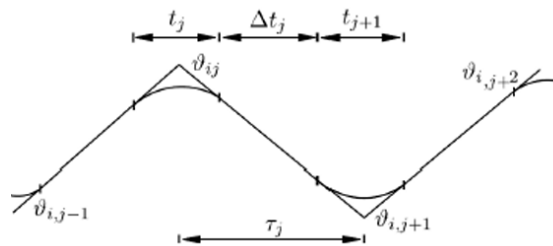


Figure 11: generic trajectory for a joint

Since the method is applied in the **joint space**, the first thing to do is to acquire the q coordinates starting from a set of waypoints taken within the S space. In order to do this, the inverse kinematics function (ROBOTinv_4R.m) is applied for each waypoint chosen for the trajectory.

```
% Waypoints in S-space
X = [250 270 300 340 370 420 500 420 370 340 300 250] * 1e-3; % [m]
Z = [ 75 75 75 150 180 220 230 220 180 150 75 75 ] * 1e-3; % [m]
Teta = deg2rad([0 5 5 -10 -15 -25 -35 -25 -15 -10 5 0]); % [rad]
Y = zeros(size(X));
```

Once the q coordinates are acquired, the next step consists in computing the **minimum actuation time** for each joint which depends on the **maximum velocity** and **acceleration**, both chosen taking into account real numbers for low-medium payload robots, like the one considered for the simulation.

Note: It is important to stress that the velocity and acceleration used for the trajectory planning are referred to the joints, but they are different with respect to the motor ones. For this reason, in order to simulate a real mechanical system made up of a **motor** and a **gearbox**, the maximum velocity and acceleration for the motion planning are reduced by a factor of $rid = 1/100$. This is consistent with the following equations, considering an **ideal reducer**:

$$\dot{\theta}_{joint} = rid \cdot \dot{\theta}_{motor}$$

$$\ddot{\theta}_{joint} = rid \cdot \ddot{\theta}_{motor}$$

As example, it is chosen:

```
% --- MOTORS PARAMETERS ---
motor.A_lim = 600; % max motor acc [rad/s^2]
motor.V_lim = 600; % max motor vel [rad/s]
motor.D_lim = 600; % max motor dec[rad/s^2]
% --- JOINTS PARAMETERS ---
motor.A = motor.A_lim * rid; % max joint acc[rad/s^2]
motor.V = motor.V_lim * rid; % max joint vel [rad/s]
motor.D = motor.D_lim * rid; % max joint dec[rad/s^2]
```

Once the joint parameters are selected, it is possible to compute the **minimum actuation time** using the function `min_rise_time.m`, in which the following equations are applied depending on the kind of profile:

$$T_{rise (parabola)} = \left(\sqrt{\frac{motor.A}{motor.D}} + \sqrt{\frac{motor.D}{motor.A}} \right) \cdot \sqrt{2 \cdot \Delta q \cdot \frac{1}{(motor.A + motor.D)}}$$

$$T_{rise (line)} = \frac{\Delta q}{motor.V} + \frac{motor.V(motor.A + motor.D)}{2 \cdot motor.A \cdot motor.D}$$

Then, the main steps of the method are:

1. Computing the parabolic transition of start and end points to make sure that the end effector reaches the start and end point precisely:

$$t_1 = t_{\{min,1\}} - \sqrt{\left\{ T_{min,1}^2 - 2 \cdot \frac{|\Delta q_1|}{motor.A} \right\}}$$

$$t_{end} = t_{\{min,end\}} - \sqrt{\left\{ T_{min,end}^2 - 2 \cdot \frac{|\Delta q_{end}|}{motor.D} \right\}}$$

2. Find the time for the internal parabolic transition, t_j :

$$t_{\{min,1\}}^* = t_{\{min,1\}} - 0.5 t_1$$

$$t_{\{min,end\}}^* = t_{\{min,1\}} - 0.5 t_{end}$$

3. Determine the time discretization (end and start points of the lines and parabolas):

$$\Delta t_j = t_{min,j} - \frac{t_{j+1}}{2} - \frac{t_j}{2}$$

4. Computing coefficients of lines:

$$\begin{cases} x = a_l \cdot t + b_l \\ \dot{x} = a_l = dq_j/t_{min} \\ \ddot{x} = 0 \end{cases} \longrightarrow \begin{aligned} a_l &= \dot{q} \\ b_l &= (x - a_l \cdot t)|_j \end{aligned}$$

5. Computing coefficients of parabolas:

$$\begin{cases} x = a_p t^2 + b_p t + c_p \\ \dot{x} = 2a_p t + b_p = \dot{q} \\ \ddot{x} = 2a_p = \ddot{q} = motor.A \end{cases} \longrightarrow \begin{aligned} a_p &= \frac{A}{2} = \frac{(a_j - a_{j+1})}{(2(t_{vec,j+1} - t_{vec,j}))} \\ b_p &= (\dot{q} - motor.A \cdot t)|_j \\ c_p &= (x - a_p t^2 - b_p t)|_j \end{aligned}$$

After computing the timing laws and polynomial coefficients, the complete joint trajectories are constructed by sequentially evaluating each parabolic and linear segment over its corresponding time interval. For every joint, position, velocity, and acceleration are computed for each segment using a fixed sampling time.

To ensure continuity and avoid duplicated samples, the initial point of a segment is removed whenever it coincides with the final point of the previous one. The segments are then appended in time to form continuous joint trajectories. Finally, the trajectories of all joints are synchronized and stored in unified vectors, which are used for plotting and for the subsequent dynamic analysis.

The results of the motion planning are depicted in the following plots:

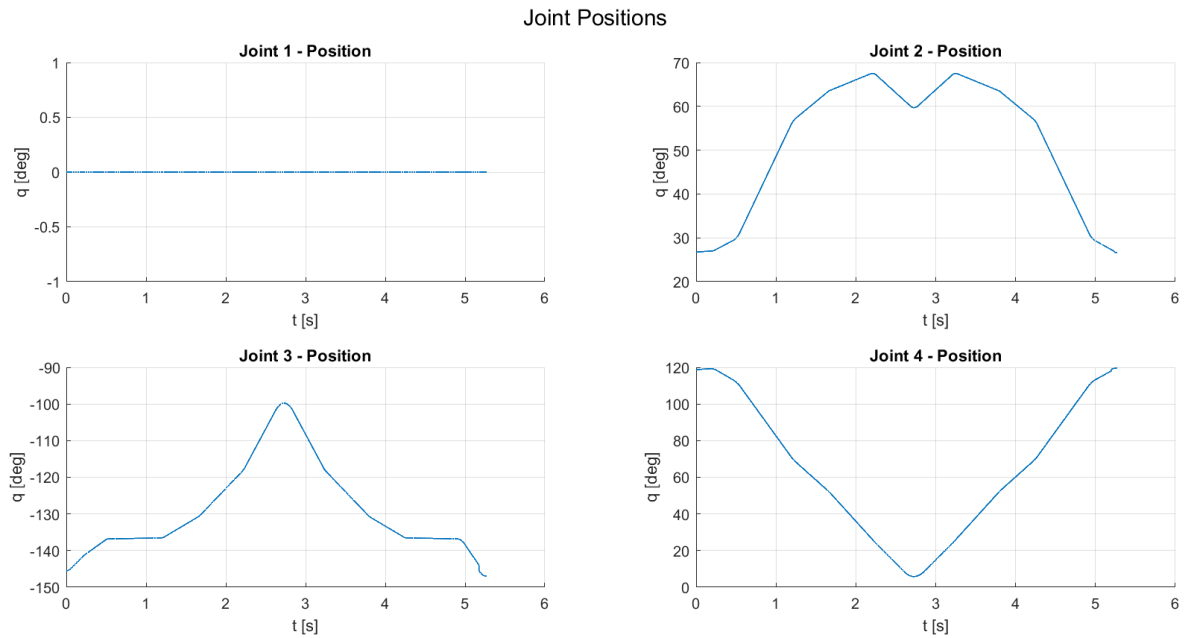


Figure 12: Joints positions task 1

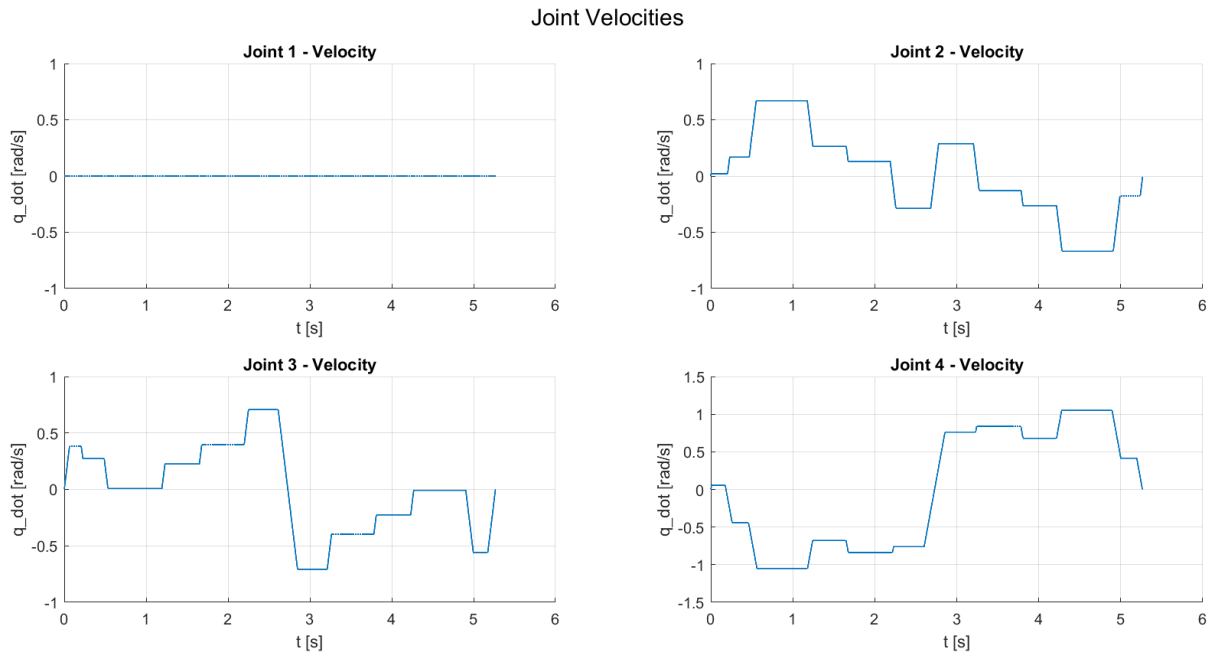


Figure 13: Joints velocities task 1

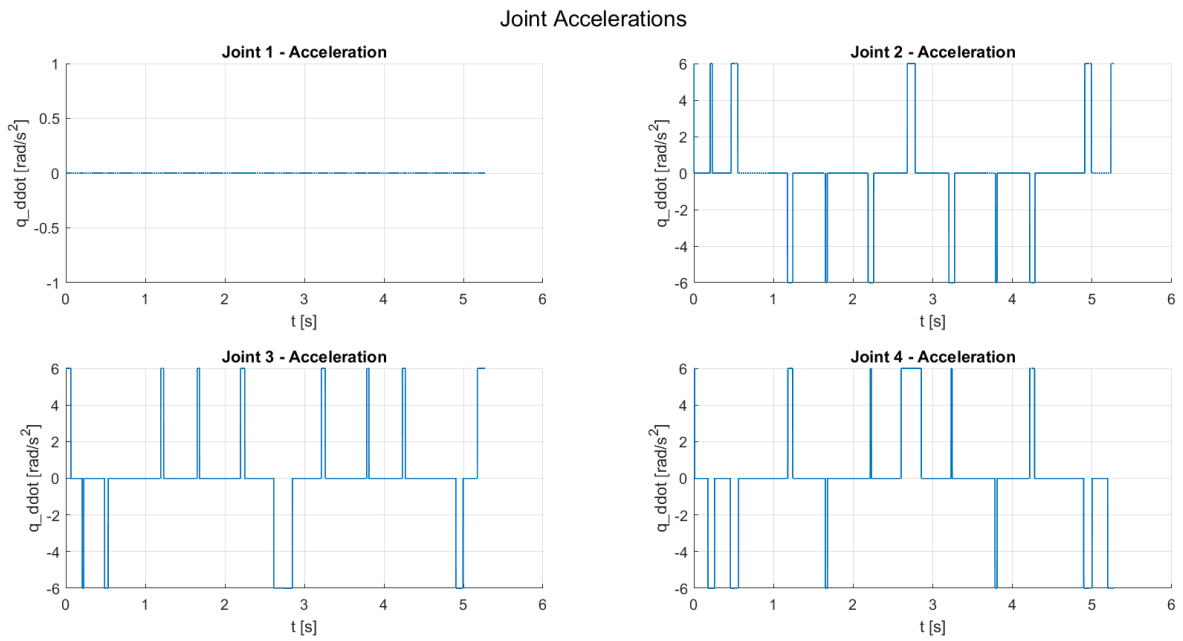


Figure 14: Joints accelerations task 1

The trajectory of the end-effector in S-space can be appreciated in the figure below:

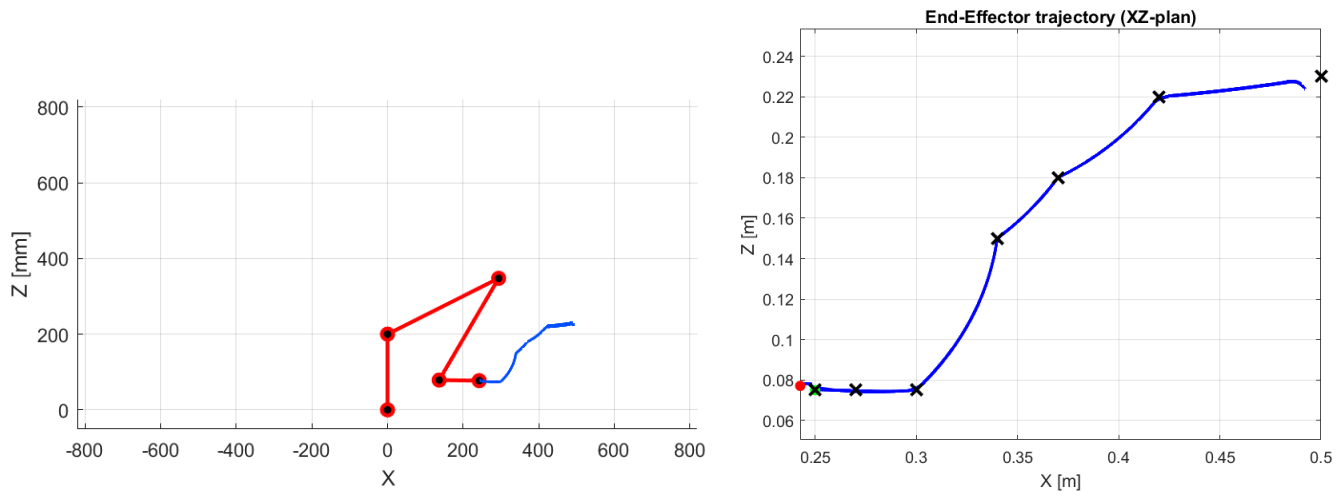


Figure 15: End effector trajectory

Furthermore, a **kinematics quality check** is performed using the same approach described in the *chapter 1*. So, using as reference the kinematic index k^{-1} , some results are printed by the code (minimum and maximum) and a plot which shows how this index changes in time along the trajectory.

=== Quality Index results ===

$k^{-1}_{\min} = 0.3706$

$k^{-1}_{\max} = 0.7781$

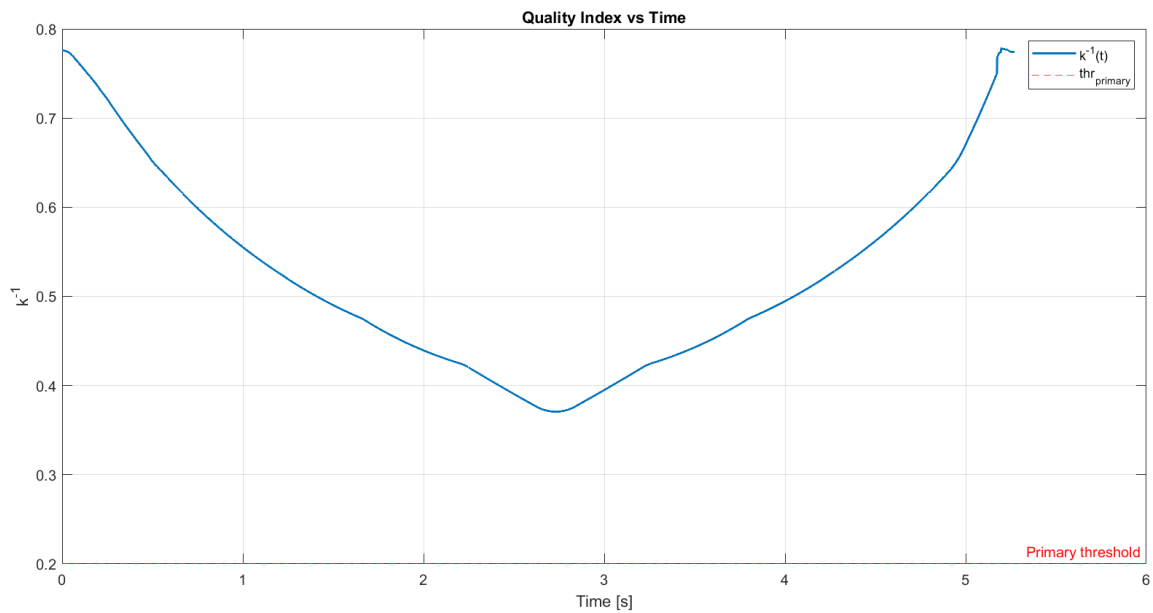


Figure 16: k^{-1} vs Time

Dynamics and Motor Torque

Concerning the dynamics, the method applied is exactly the one described in the *chapter 2*. The only difference is that in this case, in order to have a good approximation of the reality, a **reaction force** of the helmet's visor which is interacting with the manipulator is taken into account.

Moreover, since usually this force is not constant and depends on the kind of helmet, in the project it is considered only in the “worst cases”, so when the visor has to be unlocked in the opening phase and locked in the closing phase. During the remaining parts of the trajectory this force can be neglected for two reasons:

- It is very low in module compared with the robot torque.
- Usually, the visor moves quite autonomously when unlocked and the manipulator gives just little contributions in opening and closing it.

```
% Helmet Visor reaction Force
F_helmet = 4; % [N]
...
% Apply the Helmet Visor reaction force
if abs(Xee - X_helmet) <= dx && abs(Zee - Z_helmet) <= dz...
    && qp_all(4,k)<0

    Fse(2) = Fse(2) - F_helmet;
    F_helmet_on(k) = -F_helmet;

elseif abs(Xee - X_helmet) <= dx && abs(Zee - Z_helmet) <= dz...
    && qp_all(4,k)>0

    Fse(2) = Fse(2) + F_helmet;
    F_helmet_on(k) = F_helmet;
end
```

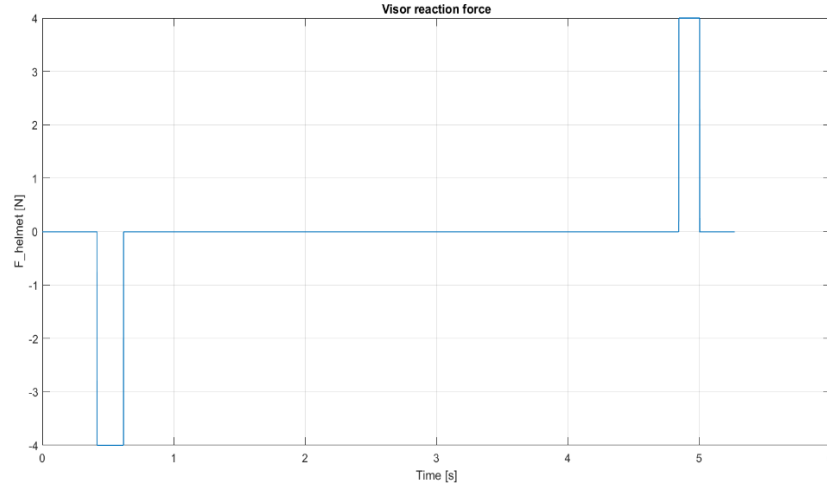


Figure 17: Visor reaction force application vs time

At this point, following the procedure shown in chapter 2, the joints torque based on the task 1 customized dynamics is computed. The results are shown in the figure below:

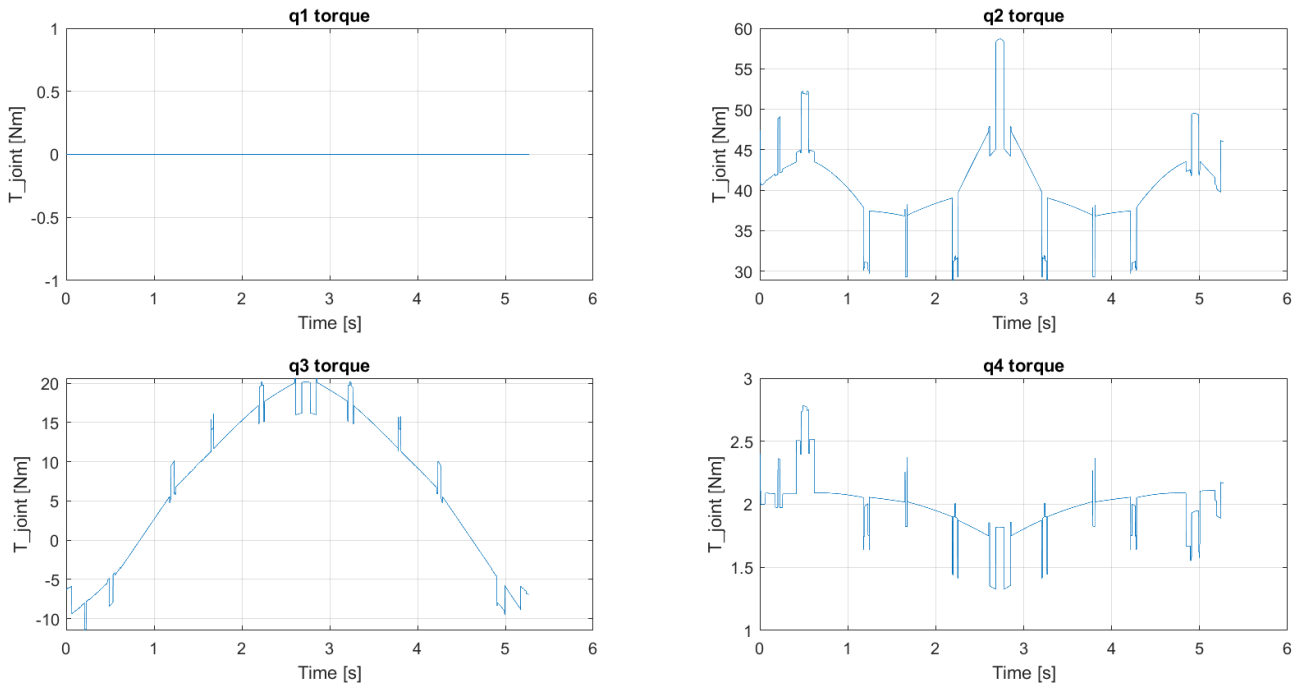


Figure 18: Joints Torques plots

The **motor torques** results, considering a gearbox with a reduction factor of **rid = 1/100** and a motor inertia equal to $J_{mot} = 0.247 \cdot 10^{-4} \text{ kg} \cdot \text{m}^2$, are plotted in the following graphs:

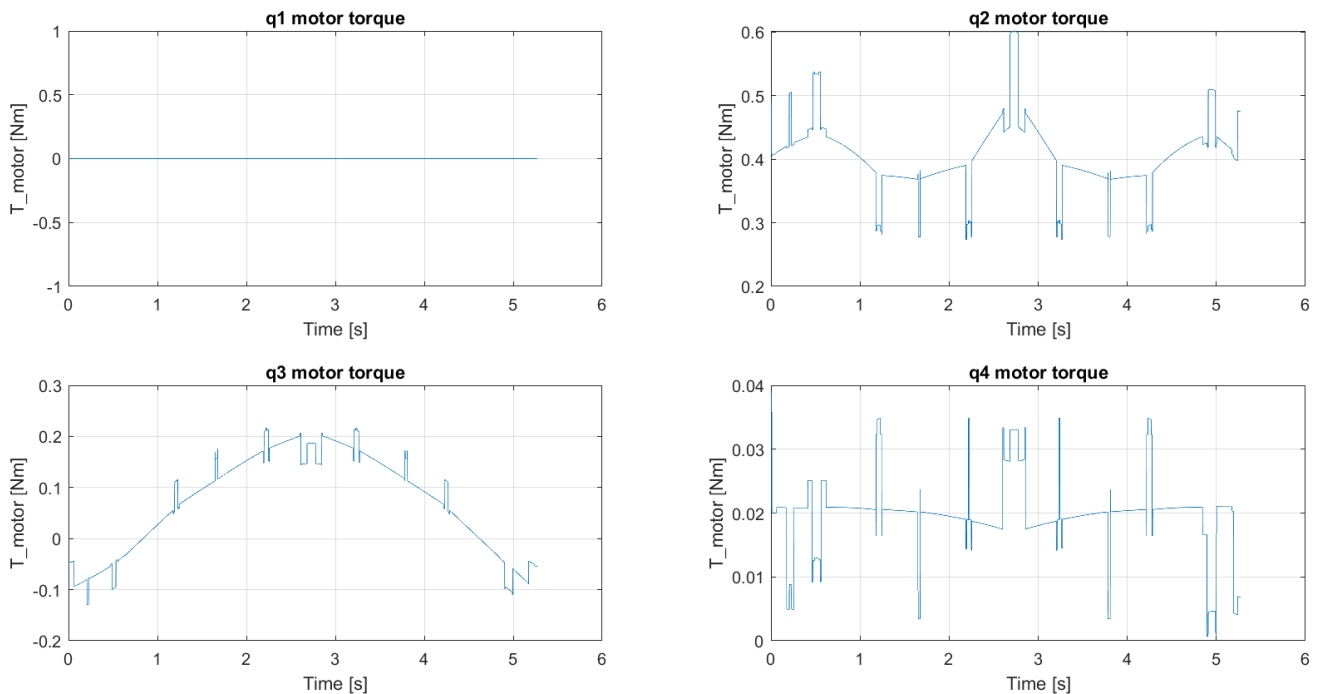


Figure 19: Motor Torques plots

Motor Sizing check

At this point, as anticipated in the previous chapter, it is performed a check a posteriori in order to validate the motor and reduction choices.

First, it is verified that the chosen reduction is adequate and as visible in the Figure 20, it can be stated that it stays in the bounds:

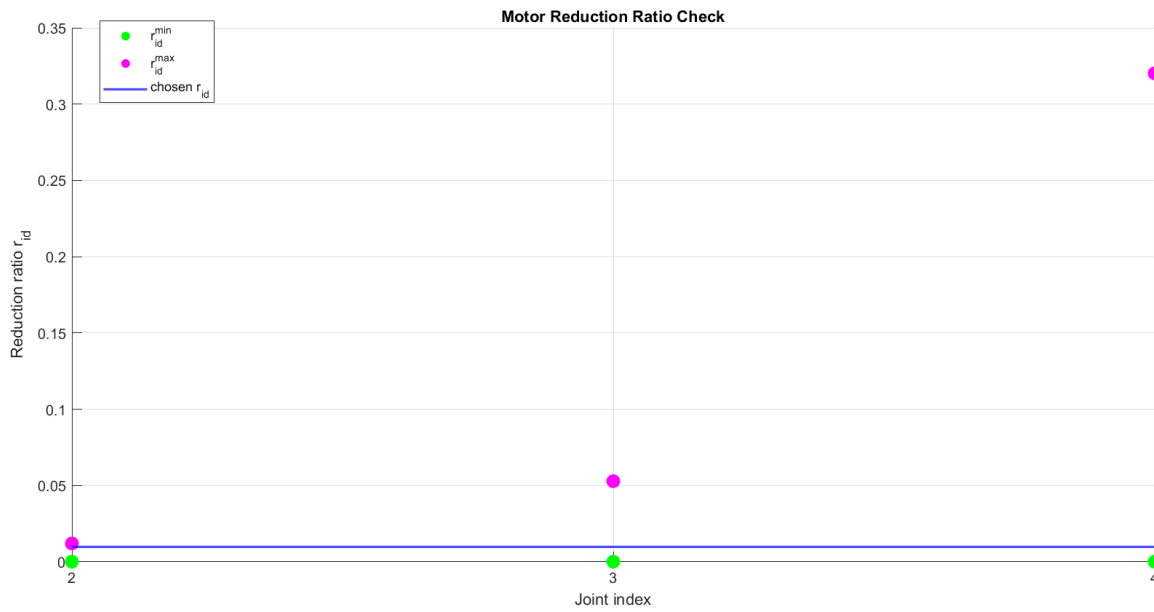


Figure 20

Furthermore, it is verified that the load factor is smaller than the accelerating factor ($\alpha > \beta$) in order to pass the thermal check. In this way it can be stated that the motors are working safely.

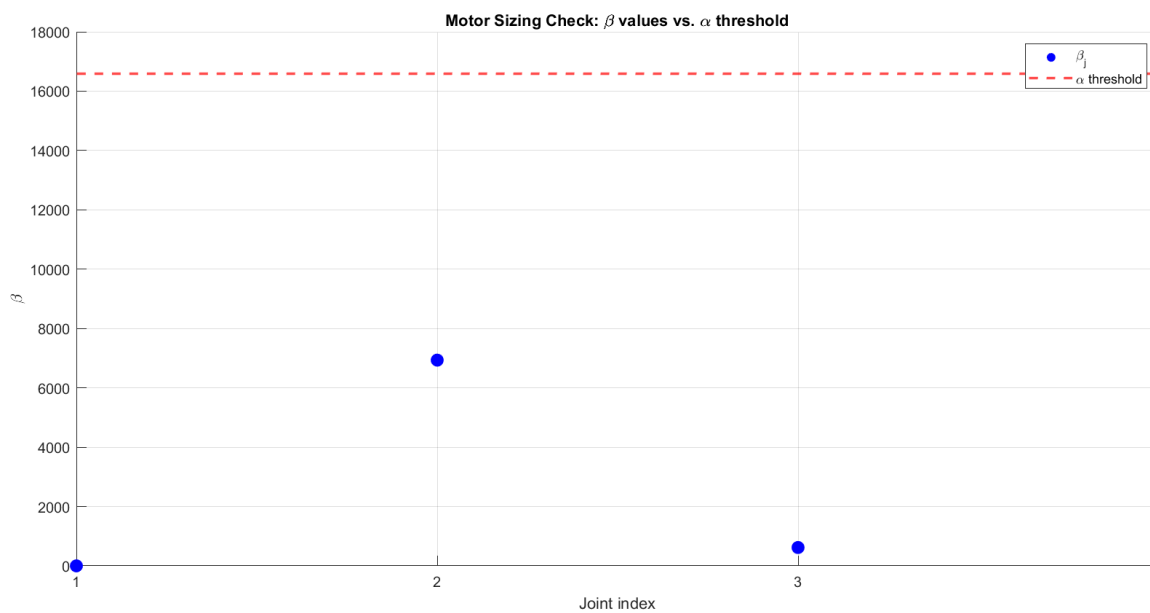


Figure 21

Moreover, The MATLAB code prints:

All joints satisfy the motor sizing condition $\alpha > \beta$.

Simscape

The same robotic system is developed in Simscape Multibody.

The parameters of the robot (lengths and masses) are taken from the MATLAB scripts such that the Simscape model is consistent with the MATLAB one.

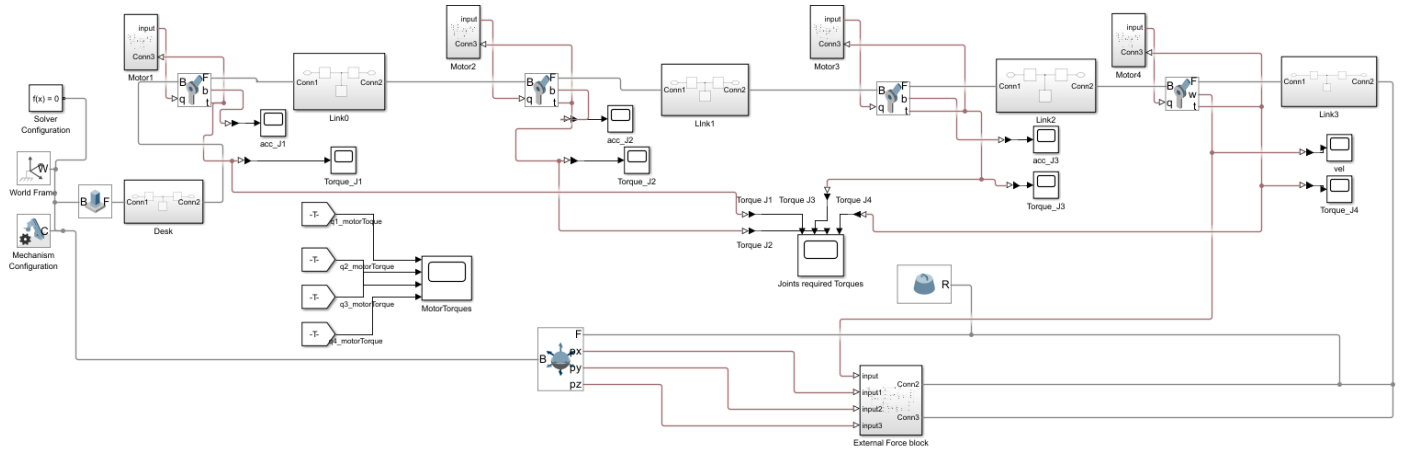


Figure 22: Simscape model - Task 1

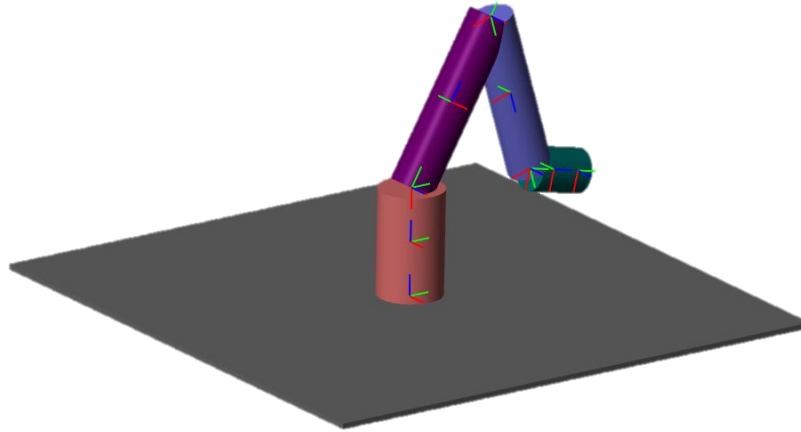


Figure 23: Simscape 3D Robot model – Mechanics Explorer

As visible in the figure 22, every joint is equipped with a Motor Block which contains all blocks in order to simulate the presence of a real motor. The motor receives as input signal the velocity profile imported from MATLAB. Then it computes the **Motor Torque** thanks to block system shown below, according to the equation:

$$\tau_{\text{motor}} = rid \cdot \tau_{\text{joint}} + J_m \cdot \frac{\ddot{\theta}_{\text{joint}}}{rid}$$

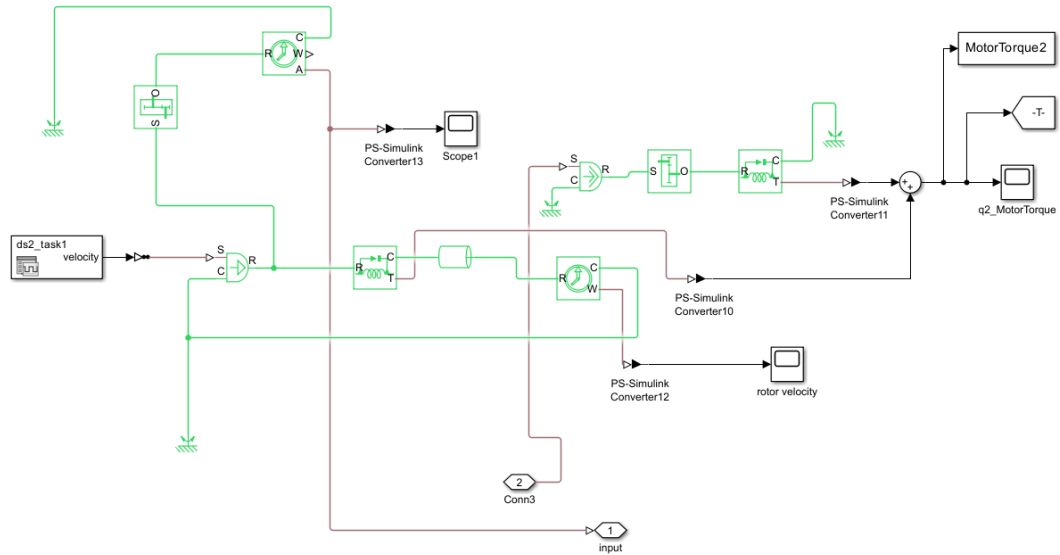


Figure 24: Simscape model – zoom on the Motor Block

Furthermore, with the purpose of considering the **helmet's visor reaction force** following the same logic chosen in MATLAB, another dedicated subsystem is created. In practice, the force is applied two times according to the End - Effector position in the S-space; one time F_{helmet} is positive, and the other time is negative. This is represented in the following picture:

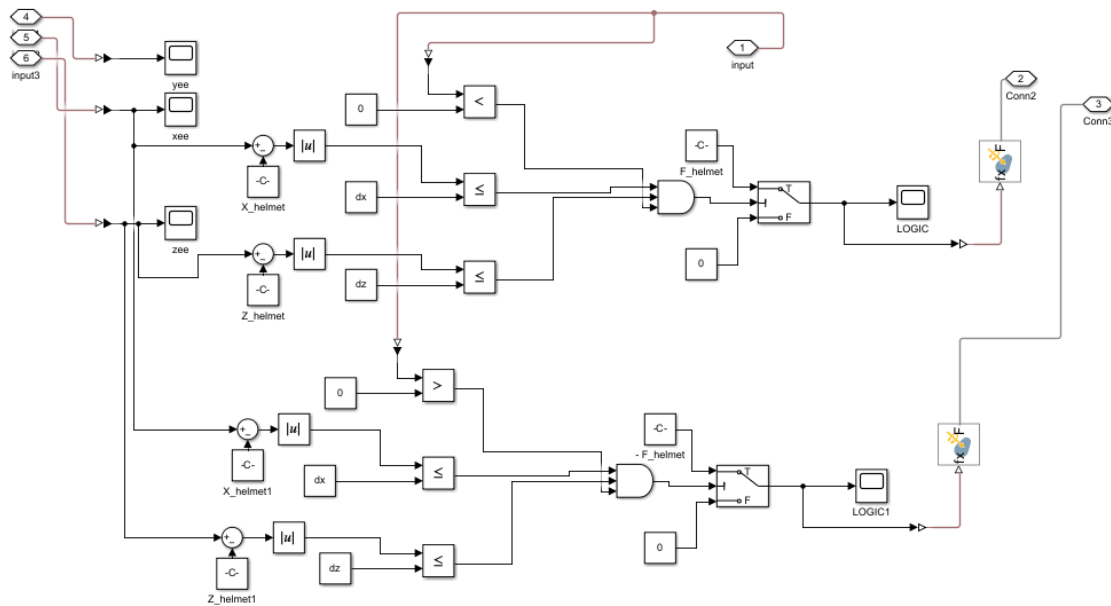


Figure 25: Simscape model – zoom on Reaction Force Logic

Finally, the **Motor Torques** related to each joint result:

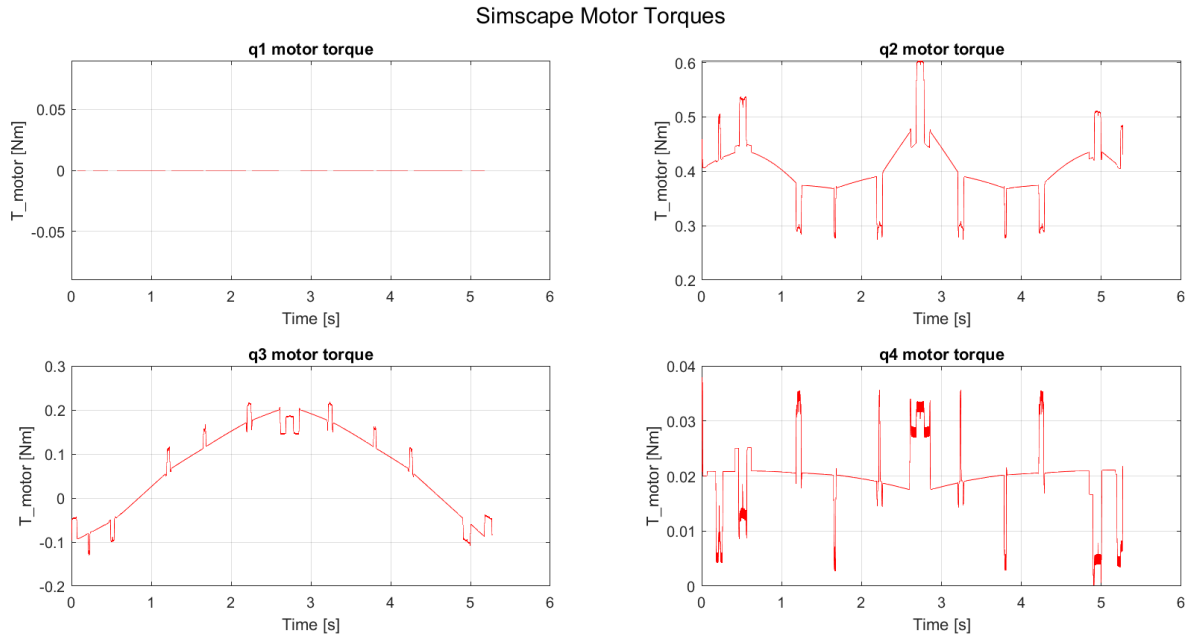


Figure 26: Simscape – Motor Torques

To conclude, in order to **assess the consistency** between the MATLAB model and the Simulink one, it is shown a plot containing both the results in terms of Motor Torques. It is clearly visible in the *figure 27* that the two plots are overlapped, except some ripple given by some disturbances present in the Simulink one.

In general, can be stated that the results are consistent and that the MATLAB model, despite being an approximation, results valid and robust.

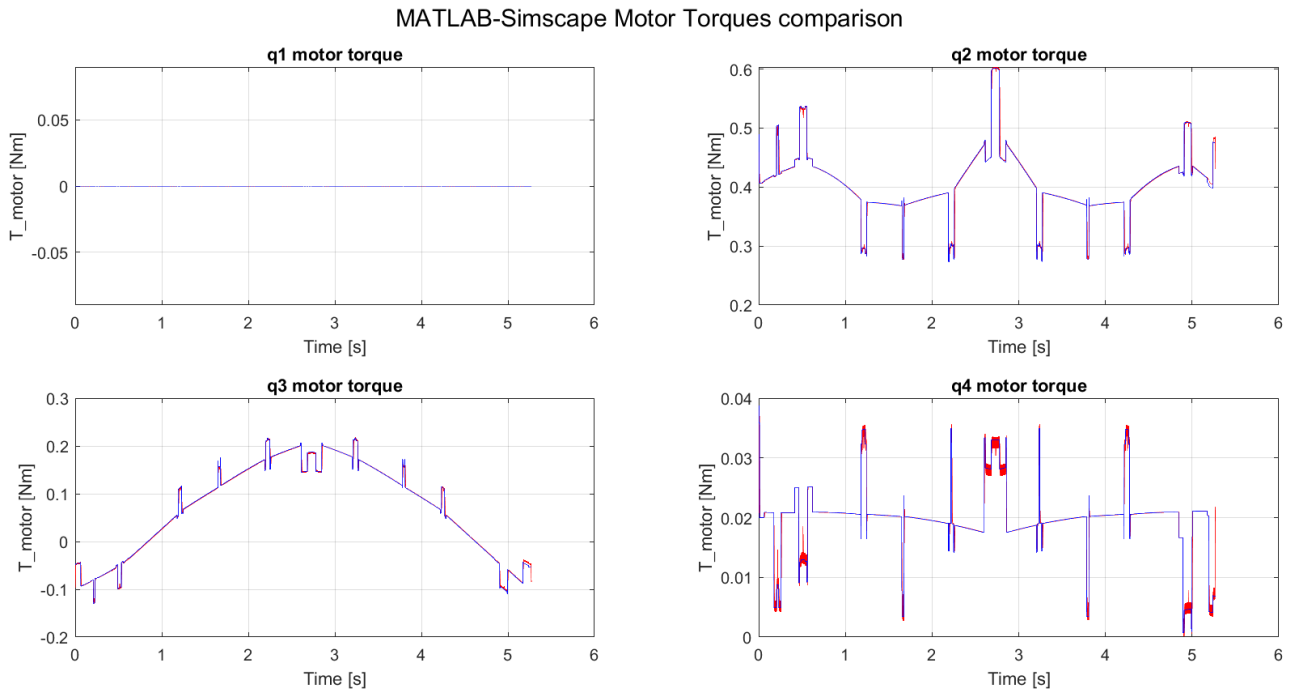


Figure 27: MATLAB and Simscape – Motor Torques comparison

4. Task 2

Motion Planning

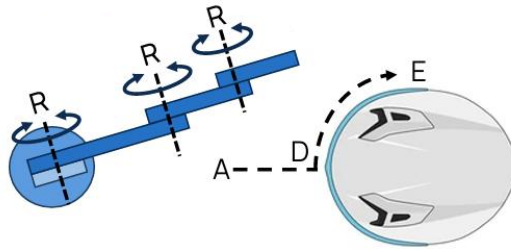


Figure 28: Task 2 trajectory

Concerning the **Task 2**, it is required to plan the trajectory A - D - E - D - A. The aim of the task is to reach one of the hinges of the helmet in order to take a picture of the state of it after performing the opening and closing task.

Since for the Task 1 the motion planning is designed in the joint space using the Lines and Parabolas method, for the task 2 it is chosen to plan the trajectory in the **operational space** (S-space).

$$\mathbf{S}(t) = [x(t) \ y(t) \ z(t) \ \theta(t)]'$$

The method can be defined as “**trajectory sampling method in operational space**”.

The S-Shape motion curve

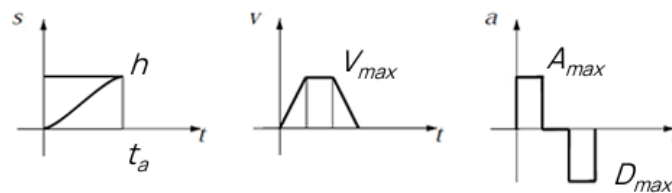


Figure 29: S-Shape generic profile

S-shape motion law with trapezoidal velocity profile is widely used in robotic motion planning to generate smooth trajectories. The motion is divided into three phases: an acceleration phase with constant positive acceleration, a constant-velocity phase, and a deceleration phase with constant negative acceleration. This time law ensures continuity of position and velocity, while limiting the maximum velocity and acceleration. Due to its simplicity and robustness, the trapezoidal motion profile is widely used in robotic applications, as it provides smooth motion execution, reduced excitation of mechanical dynamics, and reliable tracking performance.

This motion law is implemented as a function. Given an initial value, a total displacement, and three switching times, it generates a piecewise trajectory composed of a constant-acceleration phase, a constant-velocity phase, and a constant-deceleration phase, returning position, velocity, and acceleration at each time instant.

Sshape.m

```
function res=Sshape(t,S0,dS,t1,t2,t3)
V=(dS)/(t1/2 + (t2-t1) + (t3-t2)/2);
A=V/t1;
D=V/(t3-t2);
if t<t1
    res.pos=S0+1/2*A*t^2;
    res.vel=A*t;
    res.acc=A;
elseif t<t2
    res.pos=S0+1/2*A*t1^2+V*(t-t1);
    res.vel=V;
    res.acc=0;
else
    res.pos=S0+dS-D*(t3-t)^2/2;
    res.vel=V-D*(t-t2);
    res.acc=-D;
end
end
```

Minimum Actuation Time

The first step of this approach consists in computing the **minimum actuation time** in order to exploit the potential of the motors in terms of velocity and acceleration. In practice, a maximum velocity and acceleration are chosen and based on these, the actuation time is computed.

```
%% --- MOTORS PARAMETERS ---
motor.A_lim = 600; % max motor acc [rad/s^2]
motor.V_lim = 600; % max motor vel [rad/s]
motor.D_lim = 600; % max motor dec[rad/s^2]

%% --- JOINTS PARAMETERS ---
motor.A = motor.A_lim * rid; % max joint acc[rad/s^2]
motor.V = motor.V_lim * rid; % max joint vel [rad/s]
motor.D = motor.D_lim * rid; % max joint dec[rad/s^2]
```

It is used the function `min_rise_time.m` for each joint and finally the maximum value of it is chosen. This one is used in the S-Shape motion curve for the profile calculation.

Note: the motion planning is done in the operational space, so it can happen that, due to the non-linear relations between the Q and S spaces, the joints velocity and acceleration profiles could not respect precisely the motor limits. For this reason, the actuation time is **scaled of a 10%** safety factor, according to sampling method guidelines.

```
%% --- S-Shape time configuration ---
T_AD = T_min_act * 1.1; % A->D line [s]
T_DE = T_min_act * 1.1; % D->E arch [s]
T_ED = T_min_act * 1.1; % E->D arch [s]
T_DA = T_min_act * 1.1; % D->A line [s]
```

Trajectory planning

The trajectory planning is performed in the task space:

$$\mathbf{S}(t) = [x(t) \quad y(t) \quad z(t) \quad \theta(t)]^T$$

by combining point-to-point motions along linear segments and curved motions along circular arcs. All trajectory segments are generated using a trapezoidal (S-shape) velocity profile, ensuring continuity of position and velocity.

- Point to Point trajectory planning (A – D)

For a point-to-point motion between two task-space configurations S_A and S_D , a straight-line path is defined by linear interpolation. Each task-space component is expressed as:

$$s(t) = s_A + \Delta s \sigma(t) \quad \Delta s = s_D - s_A$$

where $\sigma(t)$ is the normalized trapezoidal motion law with:

$$\sigma(0) = 0 \quad \sigma(T) = 1$$

The corresponding velocity and acceleration are:

$$\dot{s}(t) = \Delta s \dot{\sigma}(t) \quad \ddot{s}(t) = \Delta s \ddot{\sigma}(t)$$

Note: The same approach is used for the returning phase (D-A).

- Circular trajectory planning (D – E)

For curved motions, the path is defined as a circular arc lying in the xy -plane. Let (C_x, C_y) be the centre of the circle and R the radius. The angular coordinate $\phi(t)$ is used as the path parameter and evolves according to a trapezoidal velocity profile:

$$\phi(t) = \phi_0 + \Delta\phi \sigma(t) \quad \Delta\phi = \phi_f - \phi_0$$

The Cartesian position is obtained through the geometric mapping:

$$\begin{aligned} x(t) &= C_x + R \cos \phi(t) \\ y(t) &= C_y + R \sin \phi(t) \end{aligned}$$

Using the chain rule, the Cartesian velocity components are:

$$\begin{aligned} \dot{x}(t) &= -R \sin \phi(t) \dot{\phi}(t) \\ \dot{y}(t) &= R \cos \phi(t) \dot{\phi}(t) \end{aligned}$$

with:

$$\dot{\phi}(t) = \Delta\phi \dot{\sigma}(t)$$

The Cartesian accelerations are obtained as:

$$\begin{aligned}\ddot{x}(t) &= -R\cos \phi(t) \dot{\phi}^2(t) - R\sin \phi(t) \ddot{\phi}(t) \\ \ddot{y}(t) &= -R\sin \phi(t) \dot{\phi}^2(t) + R\cos \phi(t) \ddot{\phi}(t)\end{aligned}$$

where:

$$\ddot{\phi}(t) = \Delta\phi \ddot{\sigma}(t)$$

These expressions account for both the tangential and centripetal components of the acceleration along the circular path.

During circular motion in the xy -plane, the **remaining task-space coordinates z and θ** are interpolated linearly with the same trapezoidal time scaling:

$$\begin{aligned}z(t) &= z_0 + \Delta z \sigma(t) \\ \theta(t) &= \theta_0 + \Delta \theta \sigma(t)\end{aligned}$$

Note: The same approach is used for the returning phase (E-D).

Finally, all the single trajectory segments are linked in order to obtain unique Time and S vectors.

The next step consists in applying the **Inverse Kinematics** to acquire the joints coordinates along all the trajectory and creating a unique q vector.

The results of the motion planning are depicted in the following plots:

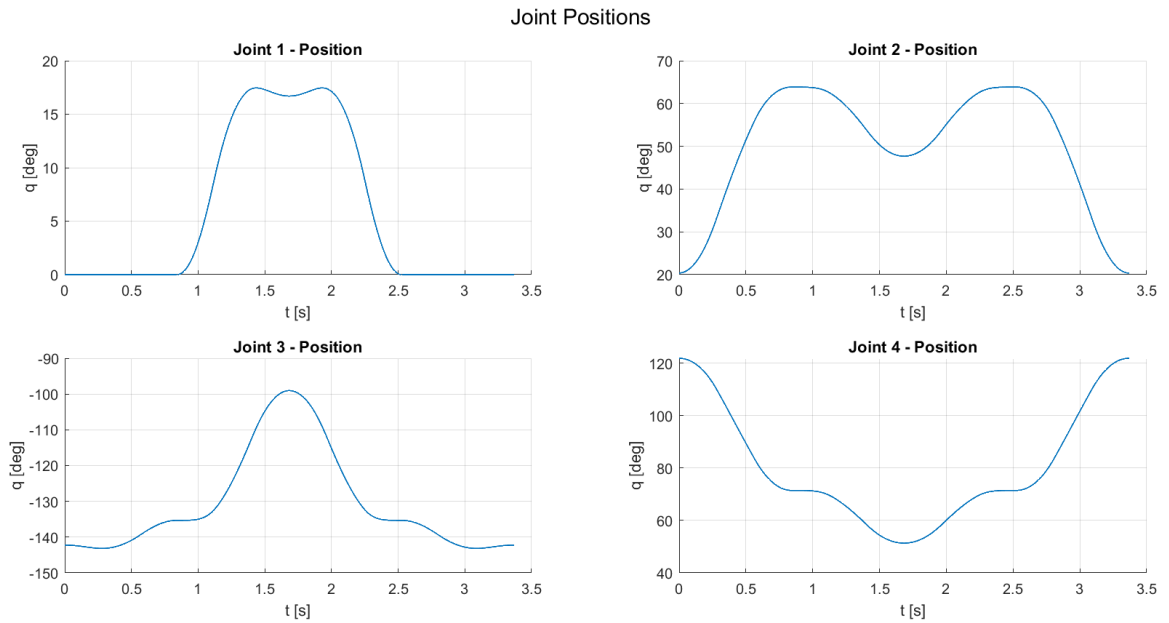


Figure 30: Joints positions task 2

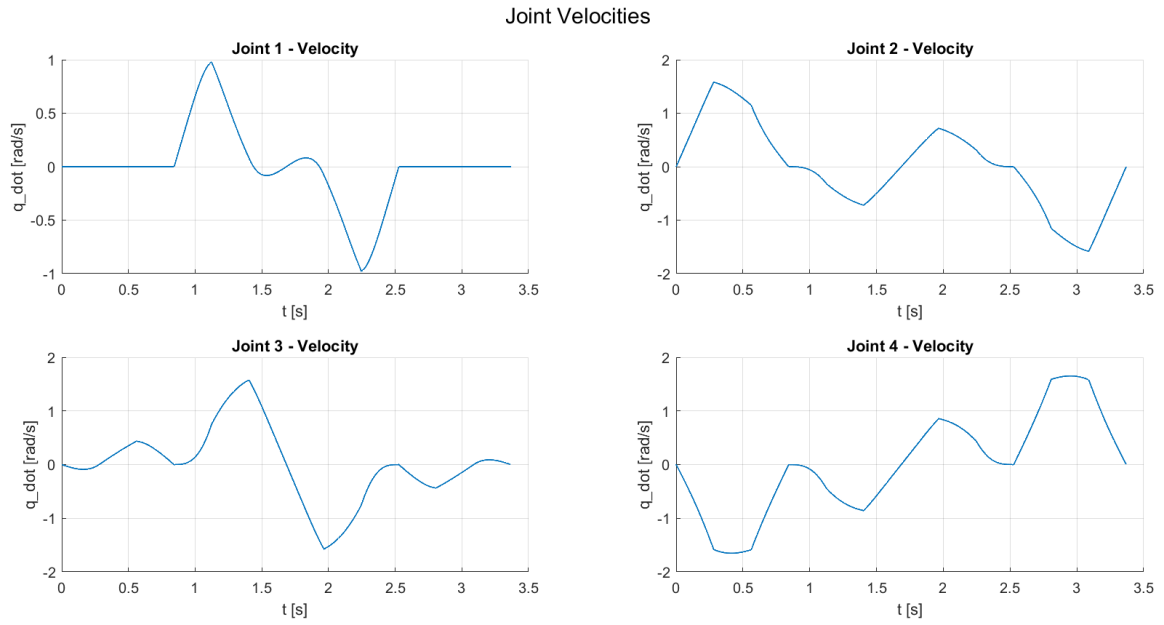


Figure 31: Joints velocities task 2

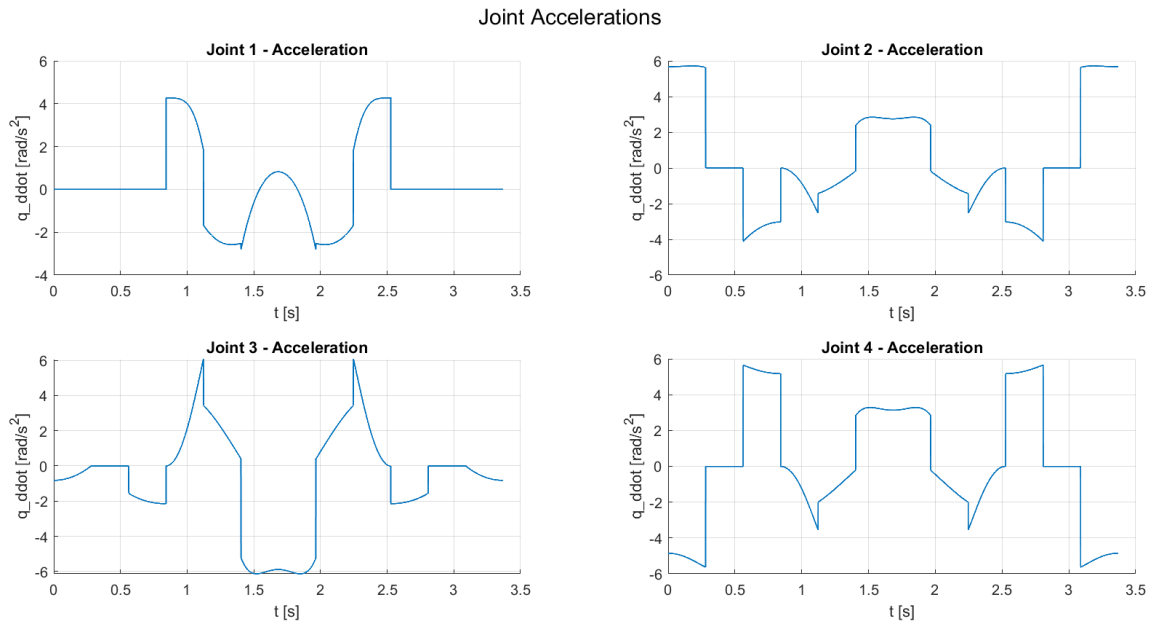


Figure 32: Joints velocities task 2

The trajectory of the end-effector in S-space can be appreciated in the figure below:

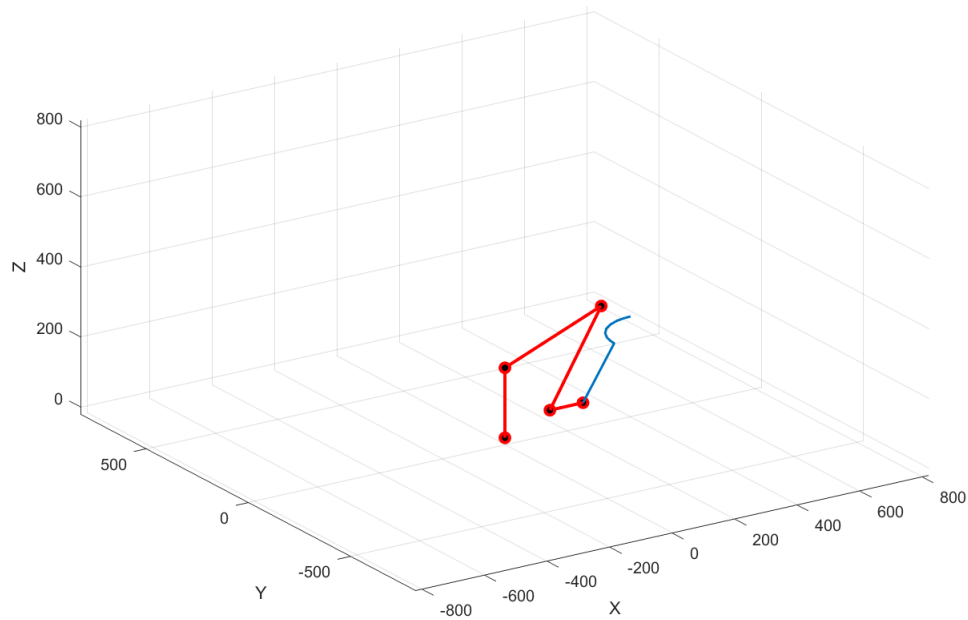


Figure 33: Robot animation with trajectory - task 2

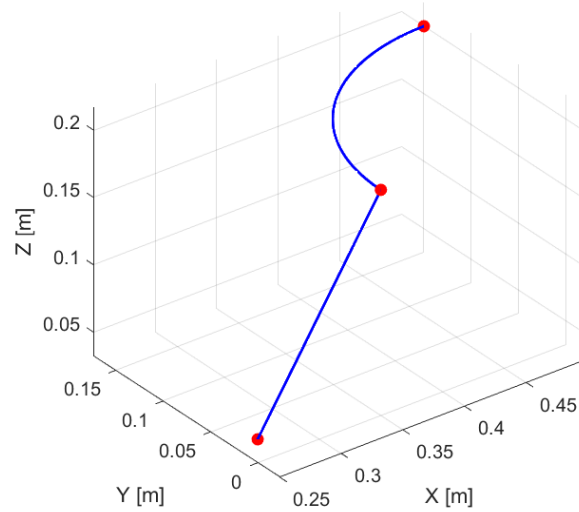


Figure 34: EE trajectory and waypoints - task 2

Furthermore, a **kinematics quality check** is performed using the same approach described in the *chapter 1*. So, using as reference the kinematic index k^{-1} , some results are printed by the code (minimum and maximum) and a plot which shows how this index changes in time along the trajectory.

=== Quality Index results ===

$k^{-1}_{\min} = 0.3321$

$k^{-1}_{\max} = 0.7539$

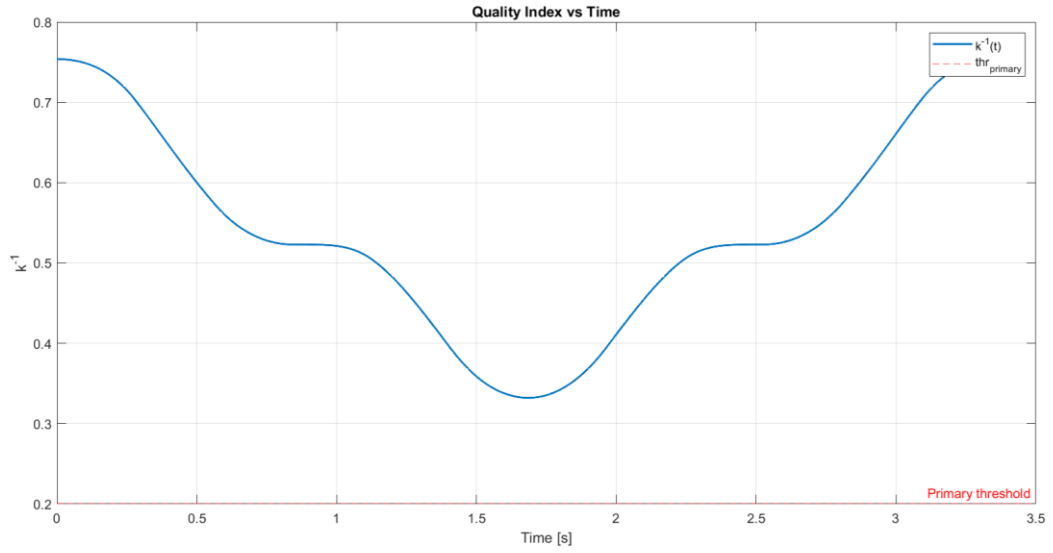


Figure 35: k^{-1} vs Time

Dynamics and Motor Torque

At this point, following the procedure shown in chapter 2, the joints torque based on the task 2 customized dynamics is computed. The results are shown in the figure below:

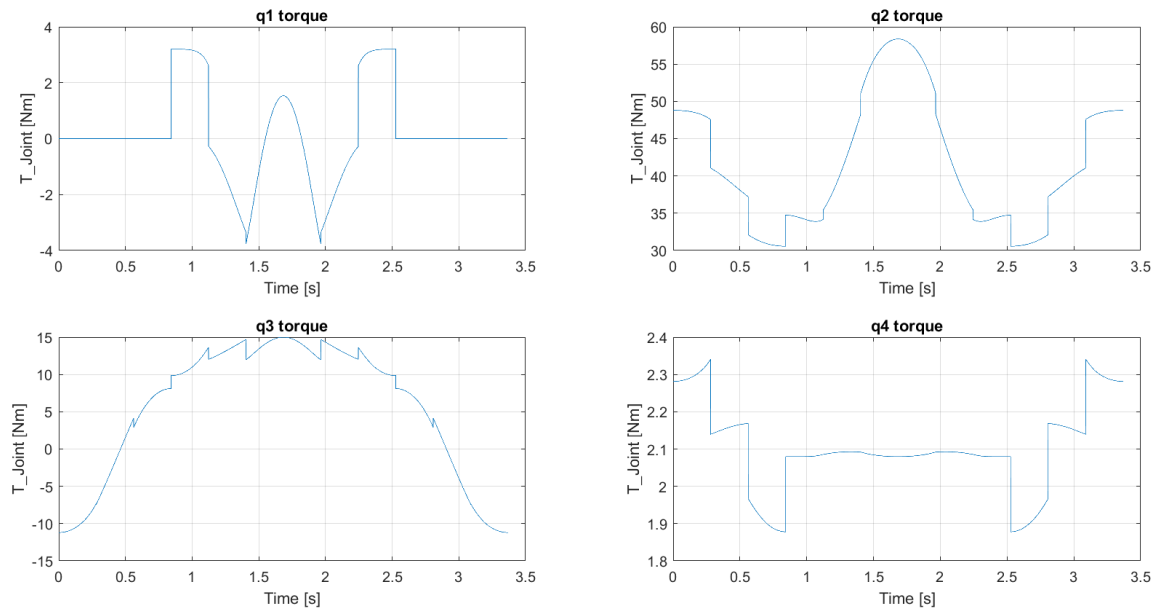


Figure 36: Joints Torques – Task 2

The **motor torques** results, considering a gearbox with a reduction factor of **rid = 1/100** and a motor inertia equal to $J_m = 0.247 \cdot 10^{-4} \text{ kg} \cdot \text{m}^2$, are plotted in the following graphs:

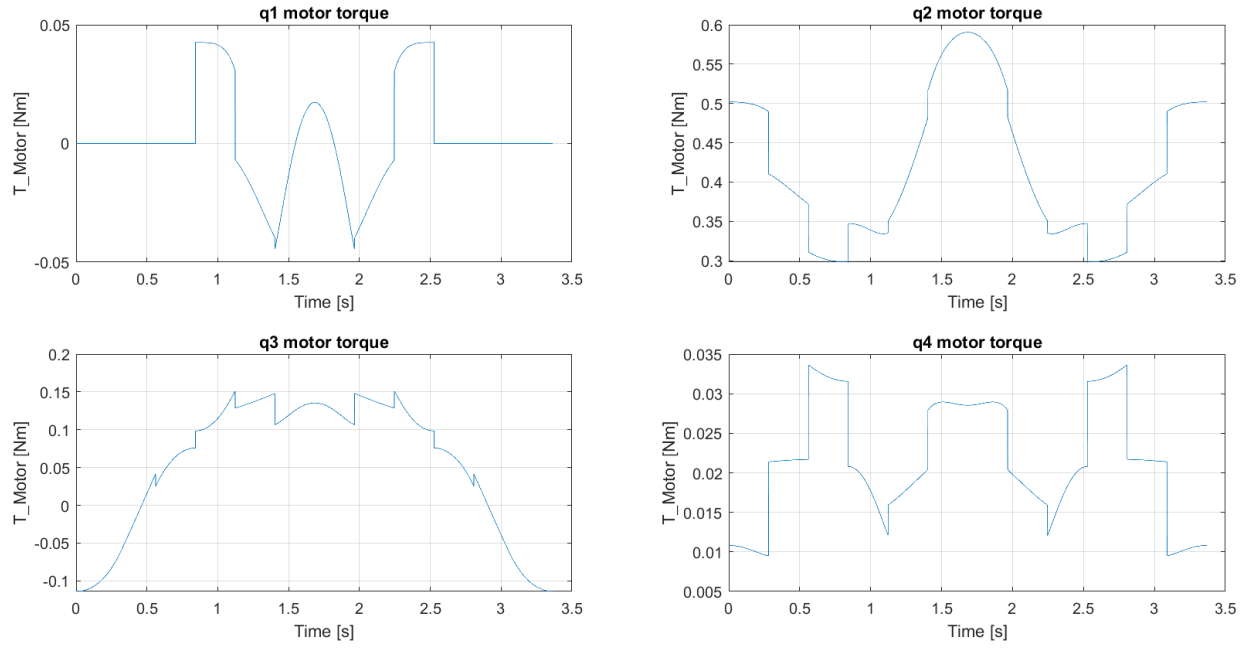


Figure 37: Motor Torques – Task 2

Motor Sizing check

As in the previous chapter, it is performed a check a posteriori in order to validate the motor and reduction choices.

First, it is verified that the chosen reduction is adequate and as visible in the Figure 38, it can be stated that it stays in the bounds:

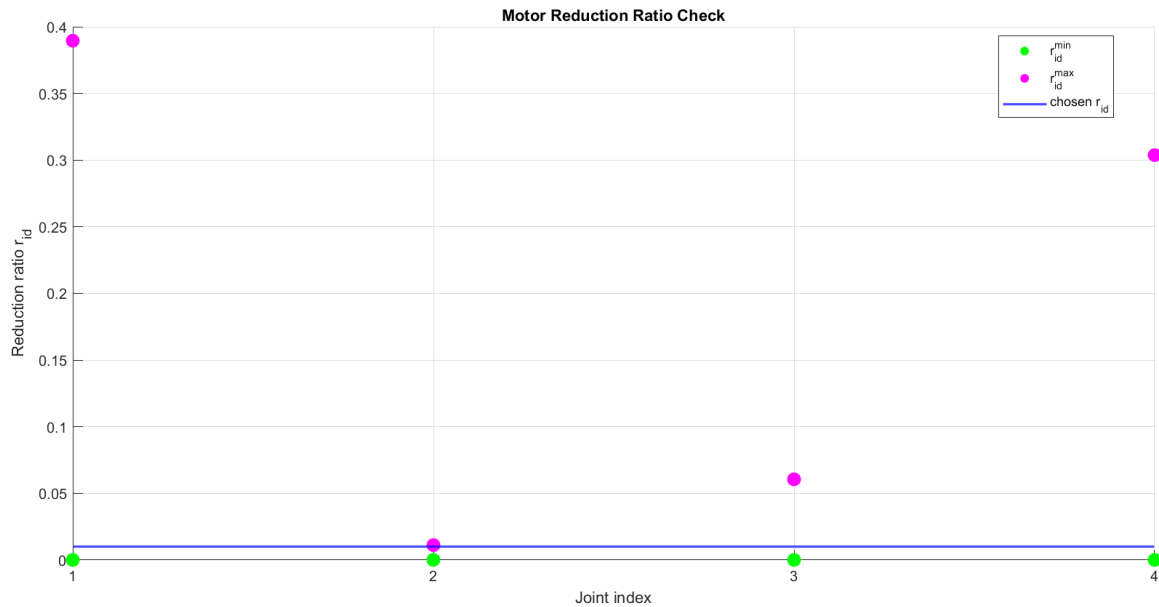


Figure 38

Furthermore, it is verified that the load factor is smaller than the accelerating factor ($\alpha > \beta$) in order to pass the thermal check. In this way it can be stated that the motors are working safely.

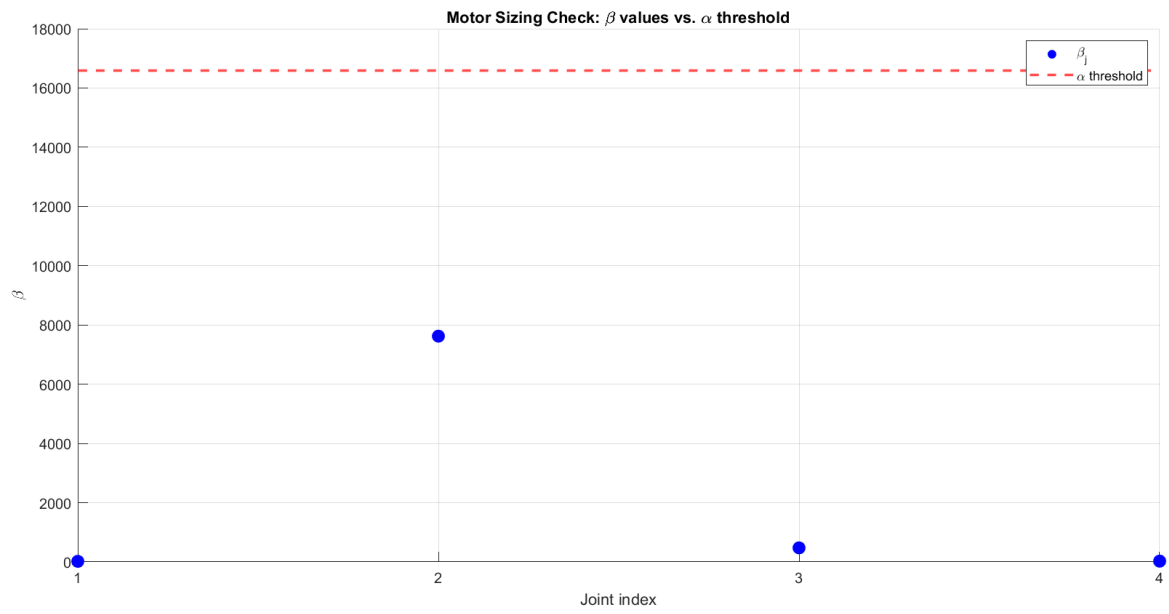


Figure 39

Moreover, The MATLAB code prints:

All joints satisfy the motor sizing condition $\alpha > \beta$.

Simscape

The same robotic system is developed in Simscape Multibody.

The parameters of the robot (lengths and masses) are taken from the MATLAB scripts such that the Simscape model is consistent with the MATLAB one.

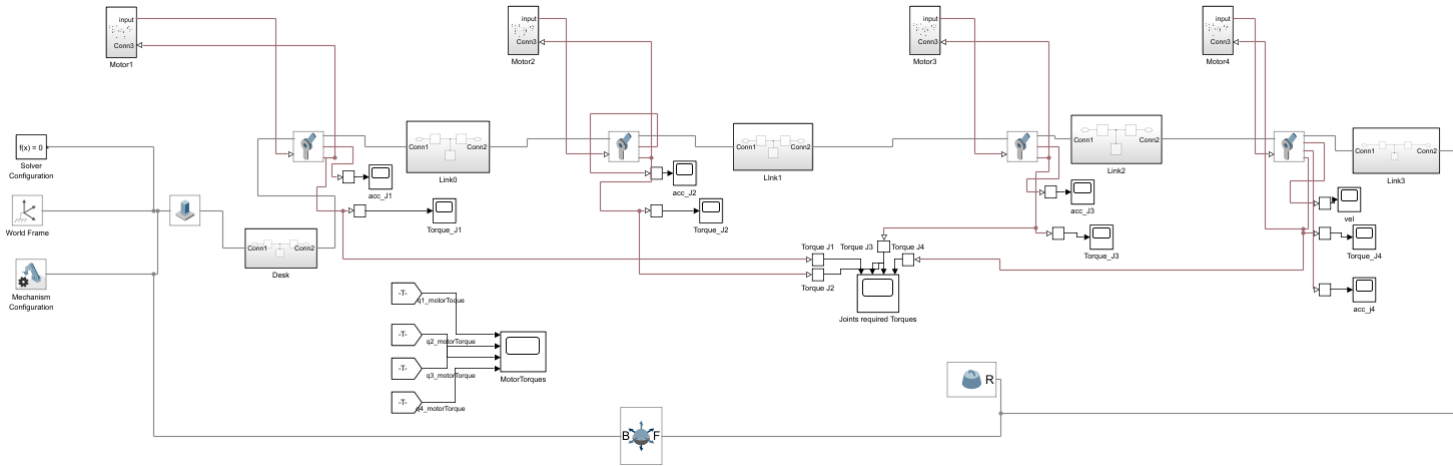


Figure 40: Simscape model - Task 2

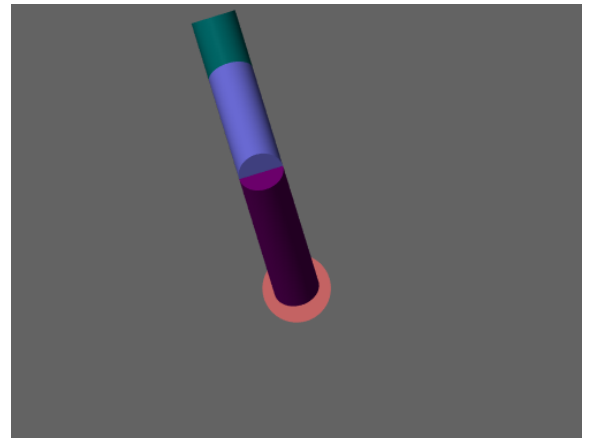
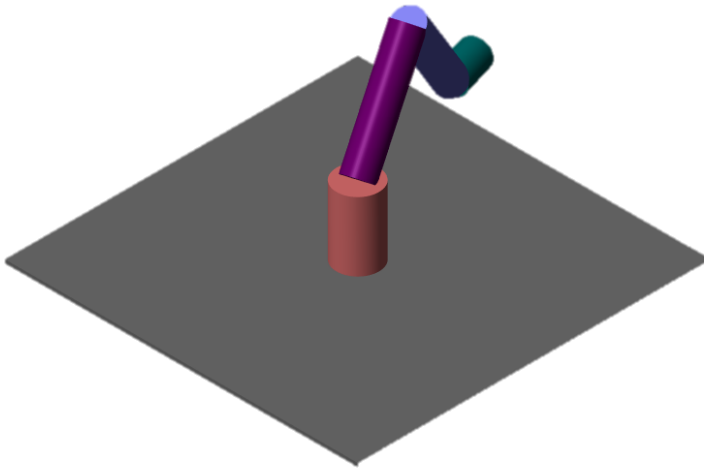


Figure 41: Simscape 3D Robot model – Mechanics Explorer

As visible in the figure 40, every joint is equipped with a Motor Block which contains all blocks in order to simulate the presence of a real motor. The motor receives as input signal the velocity profile imported from MATLAB. Then it computes the **Motor Torque** thanks to block system shown below, according to the equation:

$$\tau_{\text{motor}} = rid \cdot \tau_{\text{joint}} + J_m \cdot \frac{\ddot{\theta}_{\text{joint}}}{rid}$$

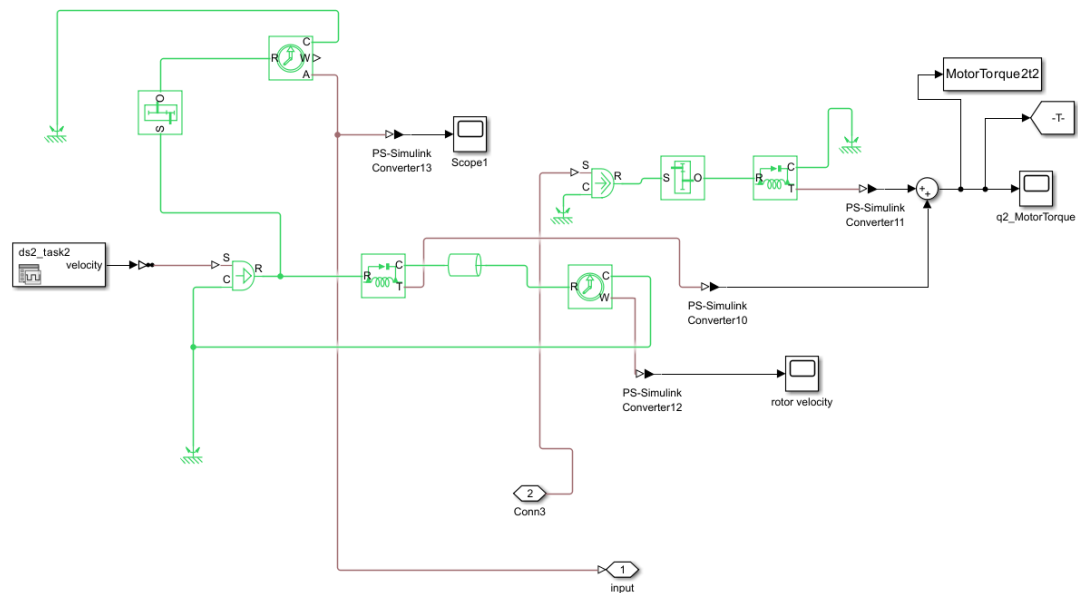


Figure 42: Simscape 3D Robot model – Mechanics Explorer

Finally, the **Motor Torques** related to each joint result:

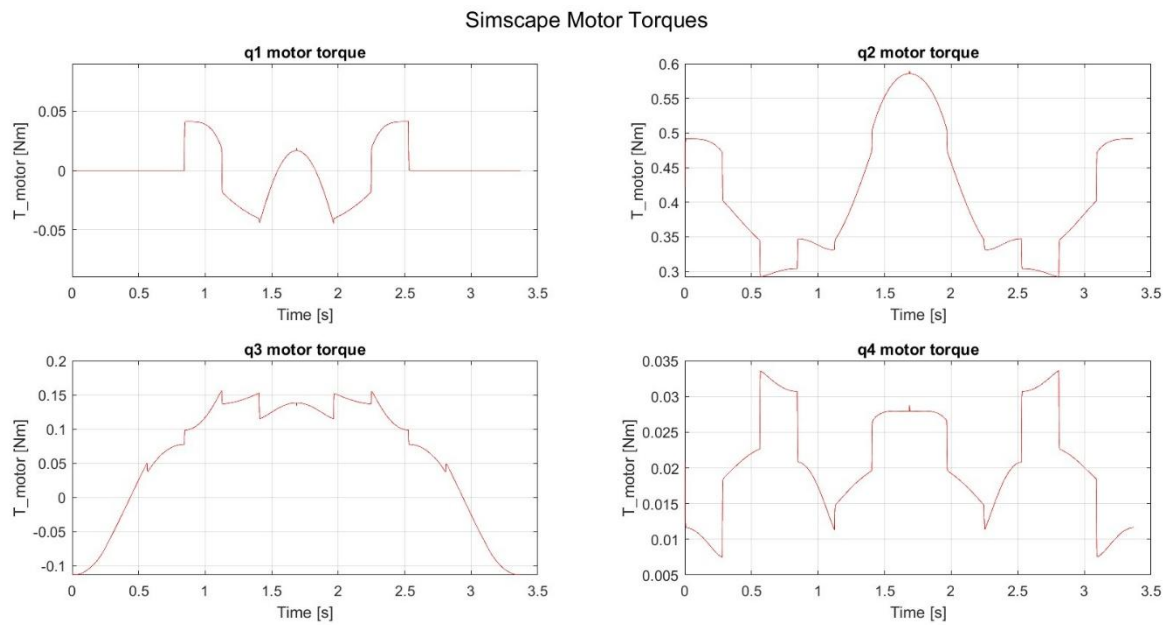


Figure 43: Simscape – Motor Torques

To conclude, in order to assess the consistency between the MATLAB model and the Simulink one, it is shown a plot containing both the results in terms of Motor Torque. It is clearly visible in the *figure 44* that the two plots are overlapped.

In general, can be stated that the results are consistent and that the MATLAB model, despite being an approximation, results valid and robust.

MATLAB-Simscape Motor Torques comparison

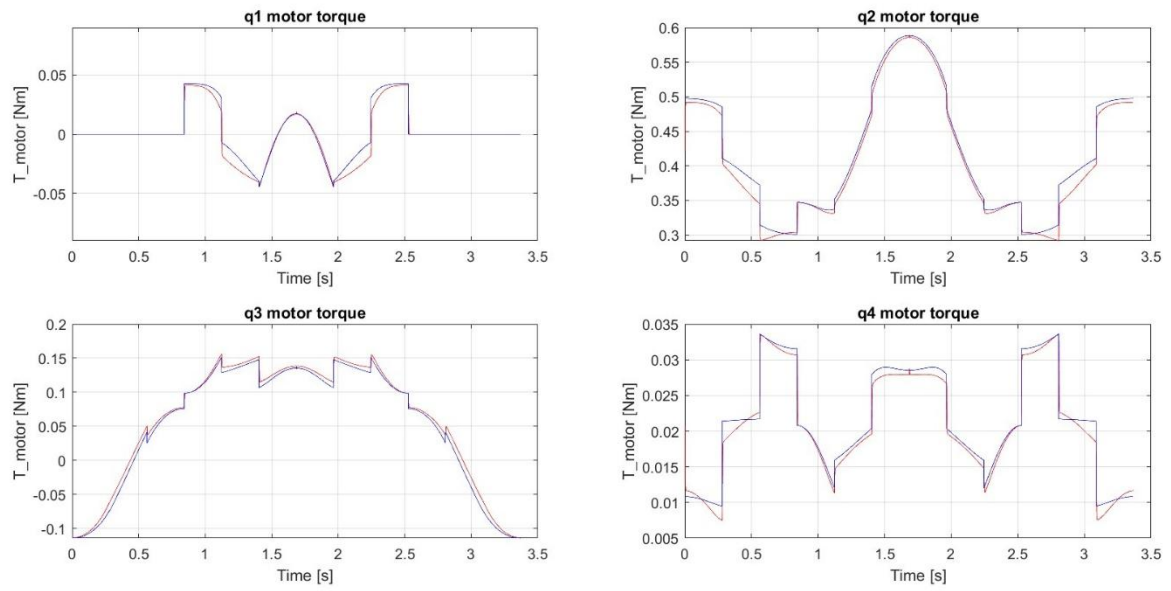


Figure 44: MATLAB and Simscape – Motor Torques comparison

5. Lab activities

The last part of the project introduces to a practical activity on a real robot. This activity consists in performing the tasks using one of the robots available in the Industrial Robotics Laboratory of the University of Pavia.

In particular, the robot used for the activity is the *TM Techman robot*.



Figure 45: *TM5 robot*

The robot is programmed via the **TMflow** software. There are two different solutions in planning the trajectory using the software:

- The first one consists in writing **lines of code**;
- The second one consists in using a **flow chart** approach building a **block diagram** in which every block represents a point in the space or a particular instruction that must be respected by the robot.

Since the task to be performed allows to use the second method, that one is chosen. The flow chart is shown in the following pictures:

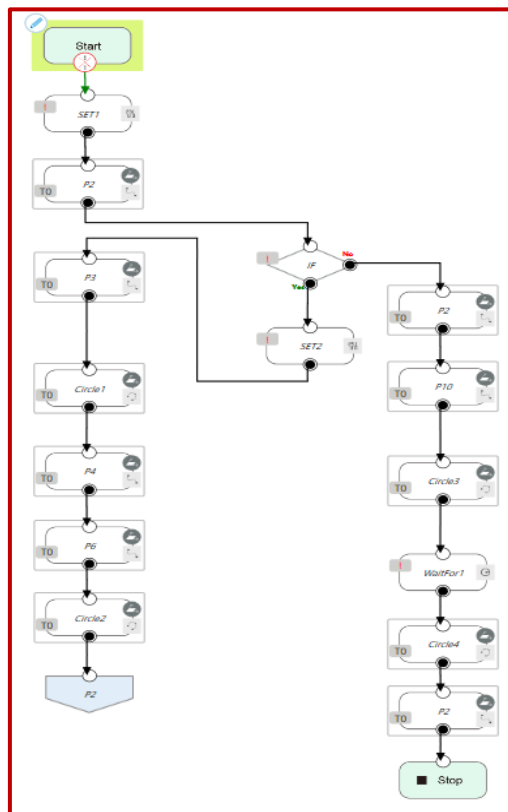


Figure 46: *TM flowchart*

Some pictures from the lab activity:



Figure 47: robot setup

Furthermore, the **EOAT** visible in the pictures is designed and printed using a 3D printer. More details about it are provided in the *Chapter 6*



Figure 48: helmet's entry point detail

6. EOAT design and manufacturing

In robotics, an **end effector** is the device or tool that the robot uses to interact with the environment to perform the desired task. The design of an end effector is a critical part; its adequacy can determine the success or failure in performing the required tasks.

For this project the required task is that of opening and closing a helmet visor repetitively with the goal of conducting quality control activities. For this purpose, the main considerations for the design of the end effector are:

- Rigidity and strength of the part must be sufficient to withstand the forces and repetitive strain of the task.
- Shape of the end effector and its adequacy to the required task.
- Contact point with the subject (in this case a NOLAN helmet).
- Joining to the robot (fasteners, fittings...).
- Available tolerancing and surface quality offered by the manufacturing method (3D printing).
- Ease of assembly and disassembly.
- Safety for operators.

Considering these points the following EOAT design has been developed:

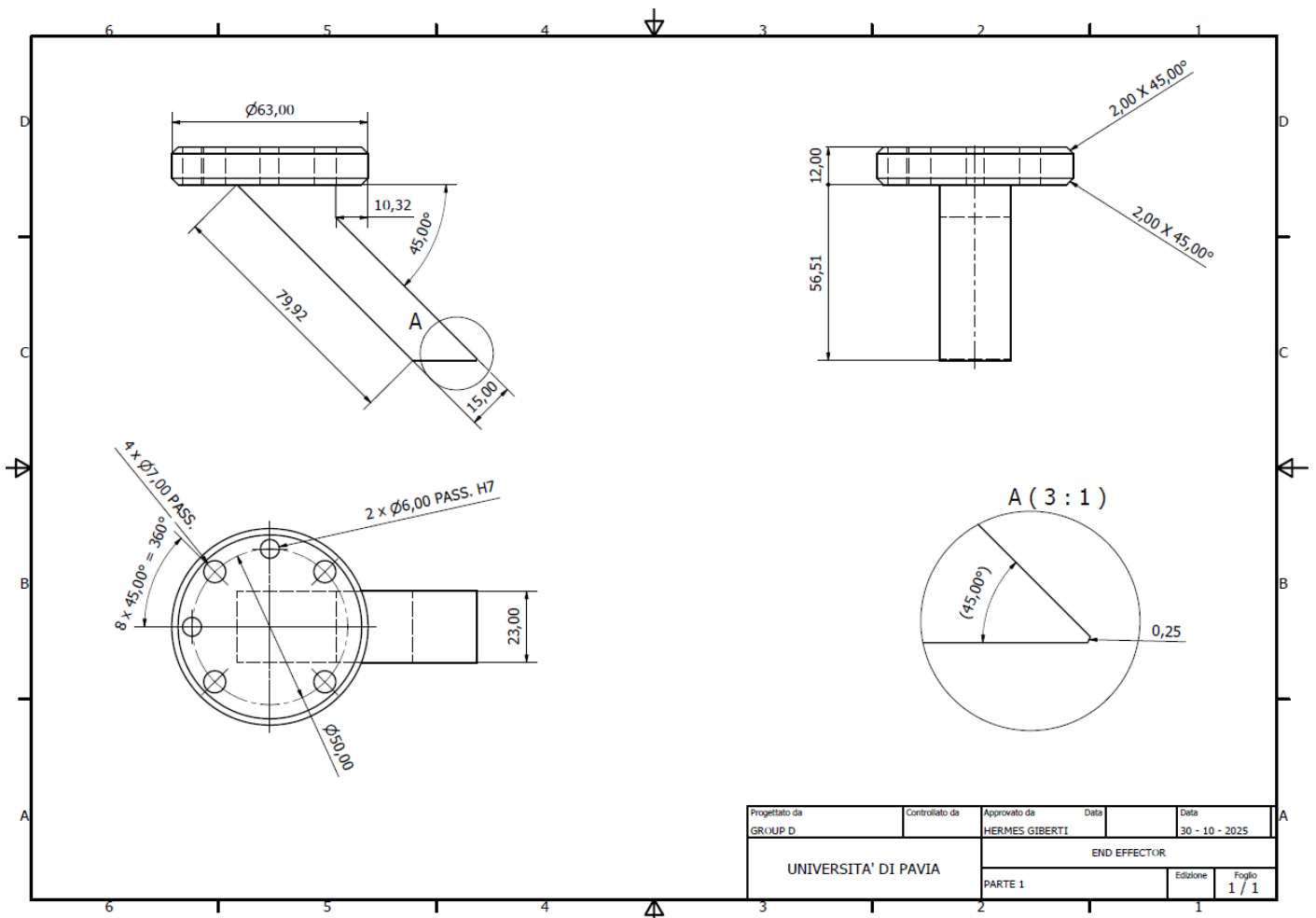


Figure 49: Drawing of the end effector

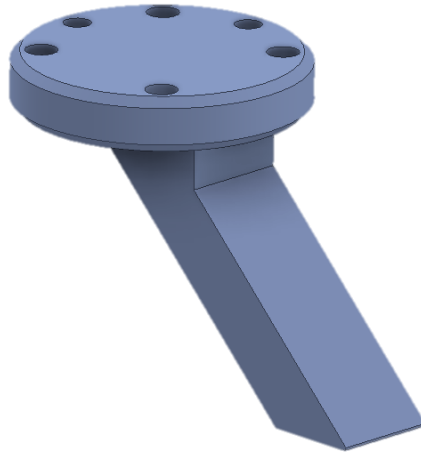


Figure 50: EOAT.ipt

Critical design characteristics

Strength and fatigue resistance:

Considering this point some of the dimensions have been over dimensioned, this is the case of the cylindrical base and the finger. Through testing, it has been observed that the estimations made were enough to withstand the performance of the task repeatedly.

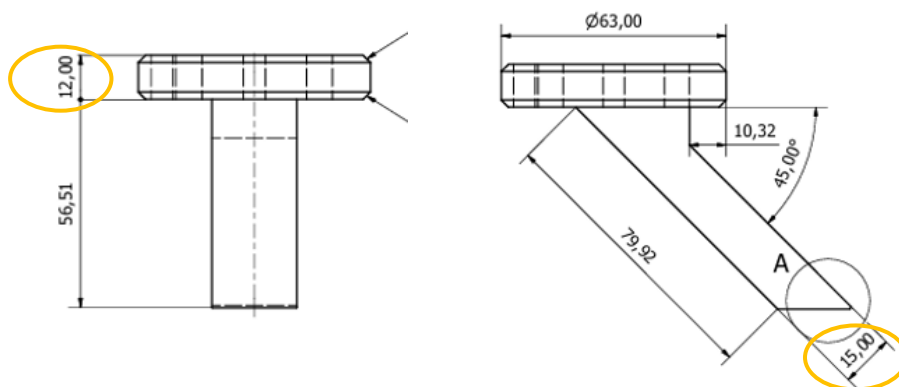


Figure 51: dimensions taken into consideration

Shape of the end effector:

An important consideration is to make sure that the geometry of the EOAT fits the task.

For this reason, the EOAT:

- has the same angle at the tip of the finger as the entry point of the helmet visor. This way it fits perfectly when lifting the visor and does not put unnecessary strain on the helmet or EOAT. When examining the helmet, the appropriate measurements were taken to make sure that the design of the finger is consistent with said angle ($\approx 45^\circ$).

- the finger itself goes down vertically at a 45° angle; this way it is more straightforward to approach the end effector and place it in the correct pose without colliding against the robot base or testing table. The length of the finger has been designed long enough so that the contact with the helmet is done by the tip of the finger and not any other part of the robot.

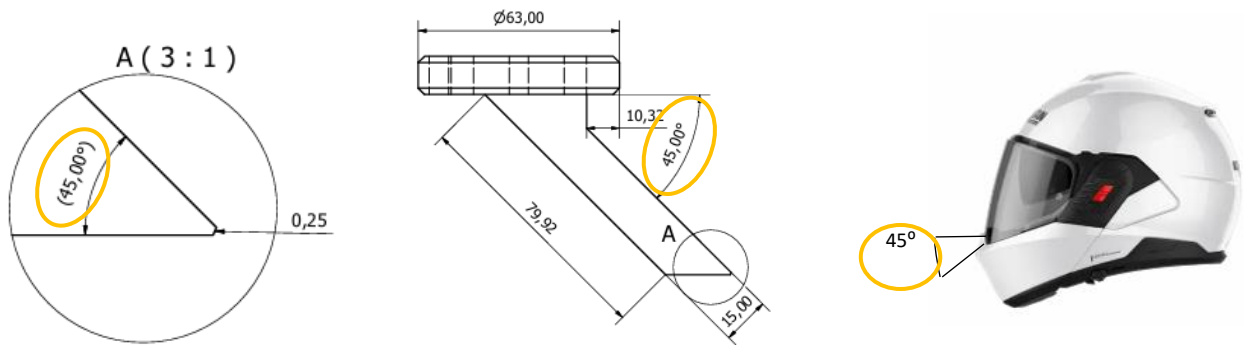


Figure 52: Fitting of the EOAT into the helmet visor

Joining to the robot

Taking into consideration the geometry of the connecting flange of the robot with the EOAT the following design decisions have been made:

- 4 x 6mm diameter through holes placed on a circumference of 50mm diameter to fit the 4 M6x20 screws, the length of which is consistent with the base girth.
- 2 x 7mm H7 holes for alignment (optional but indicated in the TM5-700 robot arm manual).
- Diameter of the base consistent with robot flange (not critical, but better aesthetically).

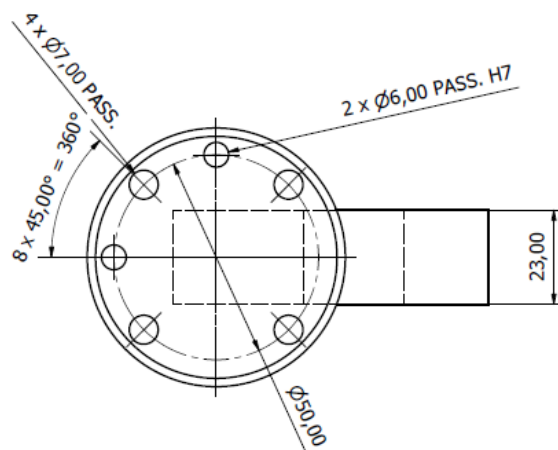


Figure 53: EOAT flange



Figure 54: EOAT flange and EE

Available tolerancing and surface quality

Considering that the EOAT is produced via additive manufacturing no special tolerancing or surface qualities have been added since the standard tolerance for this kind of manufacturing method is around ± 0.5 mm. For this reason, all fittings and holes are designed to leave enough space considering the available tolerances.

However, a special consideration has been done regarding the manufacturing method, the angle of the finger in respect to the base has been kept at 45° not to need additional supports imposed by the slicing software.

Safety for operators

Regarding the safety features of the EOAT, a chamfer has been added to the tip of the finger to avoid having any sharp corners that could potentially be a hazard when interacting with operators.

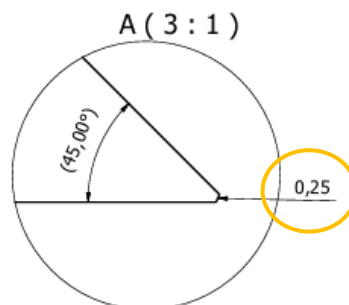


Figure 55: Chamfer on the EOAT

7. Conclusions

This project provided an overview of a structured approach to motion planning, dynamic analysis and simulation in robotics.

Different planning strategies were implemented and analysed, highlighting how the choice of trajectory representation, either in joint space or in cartesian space, directly affects the resulting kinematic and dynamic behaviour of the system.

The use of simulation tools proved essential to evaluate robot motion, actuator loads and kinematic performance before real execution, enabling a clear understanding of the underlying physical phenomena and model assumptions.

Moreover, the laboratory activity carried out on the real robot provided an important link between theory and practice, showing how simulation-based planning can be effectively transferred to real systems while accounting for unavoidable discrepancies due to hardware limitations and unmodeled effects.

Overall, the project demonstrates the importance of simulation as a fundamental tool in robotic system design, validation and experimentation.

8. MATLAB codes

Kinematics

ROBOTdir_4R.m

```
% --- Direct Kinematics ---

function S = ROBOTdir_4R(Q,L)

% Links lengths
l0=L(1);
l1=L(2);
l2=L(3);
l3=L(4);

S= zeros(4,1);

% Abbreviations
c1 = cos(Q(1)); s1 = sin(Q(1));
c2 = cos(Q(2)); s2 = sin(Q(2));
c3 = cos(Q(3)); s3 = sin(Q(3));
c4 = cos(Q(4)); s4 = sin(Q(4));
Q23 = Q(2) + Q(3);
Q234 = Q(2) + Q(3) + Q(4);
c23 = cos(Q23); s23 = sin(Q23);
c234 = cos(Q234); s234 = sin(Q234);

% Pose of the EE

R = l1 * c2 + l2 * c23 + l3 * c234; %radial distance

S(1) = c1 * R; % X-coordinate
S(2) = s1 * R; % Y-coordinate
S(3) = l0 + l1 * s2 + l2 * s23 + l3 * s234; % Z-coordinate
S(4) = Q234; % Theta (EE orientation)

end
```

ROBOTinv_4R.m

```
% --- Inverse Kinematics ---

function Q = ROBOTinv_4R(S, L, sol)

% Links lengths
l0=L(1);
l1=L(2);
l2=L(3);
l3=L(4);

% Pose of the EE
p_x = S(1);
p_y = S(2);
p_z = S(3);
teta = S(4);

Q = zeros(4,1);

% Joint q1 coordinate
Q(1) = atan2(p_y, p_x);
```

```

R_target = sqrt(p_x^2 + p_y^2); % horizontal distance of the EE from the base axis
R_wrist = R_target - l3 * cos(teta); % horizontal distance of the wrist from the base axis

pw_z = p_z - l0 - l3*sin(teta); % vertical position of the wrist w.r.t. joint-2 frame

% elbow joint q3 from the law of cosines
c3 = (R_wrist^2 + pw_z^2 - l1^2 - l2^2)/(2*l1*l2);
s3_sgn = sqrt(1-c3^2);

if sol > 0
    s3 = s3_sgn; % elbow-up solution
else
    s3 = -s3_sgn; % elbow-down solution
end

q2_z = atan2(pw_z, R_wrist); % angle between horizontal axis and wrist vector

q2_R = atan2(l2 * s3, l1 + l2 * c3); % internal angle of triangle

% Joint q2 coordinate
Q(2) = q2_z - q2_R;

% Joint q3 coordinate
Q(3) = atan2(s3, c3);

% Joint q4 coordinate
Q(4) = teta - Q(2) - Q(3);

end

```

ROBOTjac_4R.m

```

function J = ROBOTjac_4R(Q,L)

% Links lengths
l0=L(1);
l1=L(2);
l2=L(3);
l3=L(4);
q1 = Q(1); q2 = Q(2); q3 = Q(3); q4 = Q(4);

% Abbreviations
c1 = cos(q1); s1 = sin(q1);
Q23 = q2 + q3;
Q234 = q2 + q3 + q4;
c2 = cos(q2); s2 = sin(q2);
c23 = cos(Q23); s23 = sin(Q23);
c234 = cos(Q234); s234 = sin(Q234);

% Partial computations
R = l1 * c2 + l2 * c23 + l3 * c234;
dR_dq2 = -l1 * s2 - l2 * s23 - l3 * s234;
dR_dq3 = -l2 * s23 - l3 * s234;
dR_dq4 = -l3 * s234;

dpz_dq2 = l1 * c2 + l2 * c23 + l3 * c234;
dpz_dq3 = l2 * c23 + l3 * c234;
dpz_dq4 = l3 * c234;

% Jacobian matrix construction
J = zeros(4, 4);

J(1, 1) = -s1 * R;
J(2, 1) = c1 * R;

```

```

J(3, 1) = 0;
J(4, 1) = 0;

J(1, 2) = c1 * dR_dq2;
J(2, 2) = s1 * dR_dq2;
J(3, 2) = dpz_dq2;
J(4, 2) = 1;

J(1, 3) = c1 * dR_dq3;
J(2, 3) = s1 * dR_dq3;
J(3, 3) = dpz_dq3;
J(4, 3) = 1;

J(1, 4) = c1 * dR_dq4;
J(2, 4) = s1 * dR_dq4;
J(3, 4) = dpz_dq4;
J(4, 4) = 1;

```

End

ROBOTjacP_4R.m

```

function Jdot = ROBOTjacP_4R(Q,Q_dot,L)

l0 = L(1); l1=L(2); l2=L(3); l3=L(4);
q1 = Q(1); q2 = Q(2); q3 = Q(3); q4 = Q(4);
q1_dot = Q_dot(1); q2_dot = Q_dot(2); q3_dot = Q_dot(3); q4_dot = Q_dot(4);

% Trigonometric Abbreviations
c1 = cos(q1); s1 = sin(q1);
Q23 = q2 + q3; Q234 = q2 + q3 + q4;
c2 = cos(q2); s2 = sin(q2);
c23 = cos(Q23); s23 = sin(Q23);
c234 = cos(Q234); s234 = sin(Q234);

% Partial computations
R = l1 * c2 + l2 * c23 + l3 * c234;

% Derivate Prime (Coefficienti in J)
dR_dq2 = -l1 * s2 - l2 * s23 - l3 * s234;
dR_dq3 = -l2 * s23 - l3 * s234;
dR_dq4 = -l3 * s234;

% Second order partial derivatives
d2R_dq2_dqi = [-l1*c2 - l2*c23 - l3*c234, -l2*c23 - l3*c234, -l3*c234];
d2R_dq3_dqi = [-l2*c23 - l3*c234, -l2*c23 - l3*c234, -l3*c234];
d2R_dq4_dqi = [-l3*c234, -l3*c234, -l3*c234];
d2pz_dq2_dqi = [dR_dq2, dR_dq3, dR_dq4];
d2pz_dq3_dqi = [-l2*s23 - l3*s234, -l2*s23 - l3*s234, -l3*s234];
d2pz_dq4_dqi = [-l3*s234, -l3*s234, -l3*s234];

% JacobianP matrix construction
Jdot= zeros(4,4);

J_dot(1, 1) = (-c1*R)*q1_dot + (-s1*dR_dq2)*q2_dot + (-s1*dR_dq3)*q3_dot + (-s1*dR_dq4)*q4_dot;
J_dot(2, 1) = (-s1*R)*q1_dot + (c1*dR_dq2)*q2_dot + (c1*dR_dq3)*q3_dot + (c1*dR_dq4)*q4_dot;
J_dot(3, 1) = 0;
J_dot(4, 1) = 0;

```

```

J_dot(1, 2) = (-s1*dR_dq2)*q1_dot + c1*(d2R_dq2_dqi(1)*q2_dot + d2R_dq2_dqi(2)*q3_dot +
d2R_dq2_dqi(3)*q4_dot);
J_dot(2, 2) = (c1*dR_dq2)*q1_dot + s1*(d2R_dq2_dqi(1)*q2_dot + d2R_dq2_dqi(2)*q3_dot +
d2R_dq2_dqi(3)*q4_dot);
J_dot(3, 2) = d2pz_dq2_dqi(1)*q2_dot + d2pz_dq2_dqi(2)*q3_dot + d2pz_dq2_dqi(3)*q4_dot;
J_dot(4, 2) = 0;

J_dot(1, 3) = (-s1*dR_dq3)*q1_dot + c1*(d2R_dq3_dqi(1)*q2_dot + d2R_dq3_dqi(2)*q3_dot +
d2R_dq3_dqi(3)*q4_dot);
J_dot(2, 3) = (c1*dR_dq3)*q1_dot + s1*(d2R_dq3_dqi(1)*q2_dot + d2R_dq3_dqi(2)*q3_dot +
d2R_dq3_dqi(3)*q4_dot);
J_dot(3, 3) = d2pz_dq3_dqi(1)*q2_dot + d2pz_dq3_dqi(2)*q3_dot + d2pz_dq3_dqi(3)*q4_dot;
J_dot(4, 3) = 0;

J_dot(1, 4) = (-s1*dR_dq4)*q1_dot + c1*(d2R_dq4_dqi(1)*q2_dot + d2R_dq4_dqi(2)*q3_dot +
d2R_dq4_dqi(3)*q4_dot);
J_dot(2, 4) = (c1*dR_dq4)*q1_dot + s1*(d2R_dq4_dqi(1)*q2_dot + d2R_dq4_dqi(2)*q3_dot +
d2R_dq4_dqi(3)*q4_dot);
J_dot(3, 4) = d2pz_dq4_dqi(1)*q2_dot + d2pz_dq4_dqi(2)*q3_dot + d2pz_dq4_dqi(3)*q4_dot;
J_dot(4, 4) = 0;

end

```

test_robot_4R_Conditioning.m

```

clear; close all; clc;

%% --- ROBOT PARAMETERS ---
l0 = 200e-3;
l1 = 329e-3;
l2 = 311.50e-3;
l3 = 106e-3;

L = [l0; l1; l2; l3];

%% --- JOINTS LIMITS ---
q1_min = deg2rad(-180);    q1_max = deg2rad(180);
q2_min = deg2rad(-120);    q2_max = deg2rad(120);
q3_min = deg2rad(-140);    q3_max = deg2rad(140);
q4_min = deg2rad(-180);    q4_max = deg2rad(180);

%% 1) KINEMATICS TEST

S = [0.300; 0.100; 0.400; deg2rad(-10)]; % desired EE pose

% Workspace range check
max_reach = l1 + l2 + l3;
if sqrt(S(1)^2 + S(2)^2 + (S(3)-l0)^2) > max_reach
    error("ERROR: Target S is outside the maximum workspace radius.");
end

Q1 = ROBOTinv_4R(S, L, 1); % elbow-up
Q2 = ROBOTinv_4R(S, L, -1); % elbow-down

Q1_deg = rad2deg(Q1);
Q2_deg = rad2deg(Q2);

% Joint limit check
joint_limits = [q1_min q1_max;
                q2_min q2_max;
                q3_min q3_max;
                q4_min q4_max];

if any(Q1 < joint_limits(:,1) | Q1 > joint_limits(:,2))
    error("ERROR: Desired pose S is NOT reachable within joint limits (solution 1).");
end
if any(Q2 < joint_limits(:,1) | Q2 > joint_limits(:,2))

```



```

    error("ERROR: Desired pose S is NOT reachable within joint limits (solution 2).");
end

% Direct Kinematics check
S1 = ROBOTdir_4R(Q1, L);
S2 = ROBOTdir_4R(Q2, L);

%% --- QUALITY INDEX ---

% Jacobians
J1 = ROBOTjac_4R(Q1, L);
J2 = ROBOTjac_4R(Q2, L);

% Linear parts
Jv1 = J1(1:3,:);
Jv2 = J2(1:3,:);

% Singular values
s1 = svd(Jv1);
s2 = svd(Jv2);

% Conditioning index  $k^{-1}$ 
k1 = min(s1)/max(s1);
k2 = min(s2)/max(s2);

fprintf("\n=== Quality index (conditioning) ===\n");
fprintf("Solution 1 (elbow-up):  $k^{-1}$  = %.4f\n", k1);
fprintf("Solution 2 (elbow-down):  $k^{-1}$  = %.4f\n", k2);

%% --- PLOT ROBOT IN 3D SPACE ---
f1=figure;
f1.WindowStyle='normal';
hold on;
grid on;
view(3); % 3D view
l11=line('XData', [0 0 0], ...
        'YData', [0 0 0], ...
        'ZData', [0 0 0],...
        'linestyle', '-', 'linewidth', 2, 'color', 'b',...
        'marker', 'o', 'markersize', 6, 'markerfacecolor', 'k'); %1st sol

l12=line('XData', [0 0 0], ...
        'YData', [0 0 0], ...
        'ZData', [0 0 0],...
        'linestyle', '-', 'linewidth', 2, 'color', 'b',...
        'marker', 'o', 'markersize', 6, 'markerfacecolor', 'k'); %2nd sol

PlotRobot2(Q1,L,l0,'r',f1,l11); % draw the 1 sol.
PlotRobot2(Q2,L,l0,'b',f1,l12); % draw the 2 sol.

%% 2) WORKSPACE AND CONDITIONING

% Sampling
Nq1 = 13;      Nq2 = 13;
Nq3 = 9;       Nq4 = 9;

q1_vec = linspace(q1_min, q1_max, Nq1);
q2_vec = linspace(q2_min, q2_max, Nq2);
q3_vec = linspace(q3_min, q3_max, Nq3);
q4_vec = linspace(q4_min, q4_max, Nq4);

X = []; Y = []; Z = []; Kinv = [];

for q1 = q1_vec
for q2 = q2_vec
for q3 = q3_vec
for q4 = q4_vec

    Q = [q1; q2; q3; q4];

```

```

S_fk = ROBOTdir_4R(Q, L);

if S_fk(3) < 0
    continue;
end

J = ROBOTjac_4R(Q, L);
Jv = J(1:3,:);

s = svd(Jv);
smax = max(s);
smin = min(s);

if smax < 1e-8
    continue;
end

k_inv = smin/smax;

X(end+1) = S_fk(1);
Y(end+1) = S_fk(2);
Z(end+1) = S_fk(3);
Kinv(end+1) = k_inv;

end
end
end
end

% legend
ncol = 256;
cmap = [linspace(1,0,ncol)' zeros(ncol,1) linspace(0,1,ncol)'];

kmin = min(Kinv);
kmax = max(Kinv);

%% --- FIGURE: WORKSPACE 3D COLORED ---
figure;
scatter3(X, Y, Z, 15, Kinv, 'filled');
grid on; axis equal;
xlabel("X [m]"); ylabel("Y [m]"); zlabel("Z [m]");
title("Workspace by conditioning index  $k^{-1}$ ");
colormap(cmap);
cb = colorbar;
cb.Label.String = " $k^{-1}$ ";

%% 3) WORKSPACE MAPPING

thr_primary = 0.2;
idx_primary = find(Kinv >= thr_primary);

Xp = X(idx_primary);
Yp = Y(idx_primary);
Zp = Z(idx_primary);

K_all = convhull(X, Y, Z);
K_primary = convhull(Xp, Yp, Zp);

%% --- PLOTS ---
figure; hold on; grid on; axis equal; view(135,30);
xlabel('X'); ylabel('Y'); zlabel('Z');
title("Workspace");

% Secondary workspace
trisurf(K_all, X, Y, Z, 'FaceColor',[0.7 0.7 0.7], ...
        'FaceAlpha',0.3, 'EdgeColor','none');

% Primary workspace
trisurf(K_primary, Xp, Yp, Zp, 'FaceColor',[0 0 1], ...

```

```

    'FaceAlpha',0.4, 'EdgeColor','none');
surf(Xs, Ys, Zs, 'FaceColor',[1 0 0], ...
    'FaceAlpha',0.4, 'EdgeColor','none');

plot3(0,0,0,'ko','MarkerFaceColor','k');

legend("Secondary workspace", ...
    "Primary workspace");

```

PlotArea4R_Catalog.m

```

function PlotArea4R_Catalog(L, fig, X, Y, Z, Kinv, thr_primary)

figure(fig); clf; hold on; grid on; axis equal;

l0 = L(1); l1 = L(2); l2 = L(3); l3 = L(4);

% --- safety filter (even if already filtered in main) ---
idx = (Z >= 0);
X = X(idx); Y = Y(idx); Z = Z(idx); Kinv = Kinv(idx);

% --- distance from shoulder (0,0,l0) ---
r_xy = hypot(X, Y); % sqrt(X^2 + Y^2)
rho = hypot(r_xy, (Z - l0)); % sqrt(X^2 + Y^2 + (Z-l0)^2)

% --- Secondary radius (theoretical max reach) ---
Rout = l1 + l2 + l3;

% --- Primary radius from sampled data (well-conditioned) ---
idxp = (Kinv >= thr_primary);
Rpri = max(rho(idxp));
if isempty(Rpri)
    Rpri = NaN; % no primary points
end

% --- draw circles in X-Z section centered at shoulder (0,l0) ---
cx = 0; cz = l0;
fi = linspace(0, 2*pi, 400);

% Secondary (black)
plot(cx + Rout*cos(fi), cz + Rout*sin(fi), 'k-', 'LineWidth', 2);

% Primary (blue) - only if exists
if ~isnan(Rpri)
    plot(cx + Rpri*cos(fi), cz + Rpri*sin(fi), 'b-', 'LineWidth', 2);
end

% --- stylized robot stretched along +X at z=l0 ---
plot(0, 0, 'ko', 'MarkerFaceColor','k', 'MarkerSize', 6); % base
plot(0, l0, 'ks', 'MarkerFaceColor',[1 1 0], 'MarkerSize', 7); % shoulder
plot([0 0], [0 l0], 'k-', 'LineWidth', 3); % column

p1 = [0; l0];
p2 = [l1; l0];
p3 = [l1+l2; l0];
p4 = [l1+l2+l3; l0];

plot([p1(1) p2(1)], [p1(2) p2(2)], 'k-', 'LineWidth', 6);
plot([p2(1) p3(1)], [p2(2) p3(2)], 'k-', 'LineWidth', 6);
plot([p3(1) p4(1)], [p3(2) p4(2)], 'k-', 'LineWidth', 6);

plot(p2(1), p2(2), 'ks', 'MarkerFaceColor',[0.8 0.8 0.8], 'MarkerSize', 10);
plot(p3(1), p3(2), 'ks', 'MarkerFaceColor',[0.8 0.8 0.8], 'MarkerSize', 10);
plot(p4(1), p4(2), 'kd', 'MarkerFaceColor',[0.6 0.6 0.6], 'MarkerSize', 9);

```

```

% --- floor and axes ---
yline(0,'k-','LineWidth',1.2);
xlim([-1.05*Rout, 1.05*Rout]);
ylim([0, l0 + 1.05*Rout]);

xlabel('X [m]'); ylabel('Z [m]');
% legend (simple)
if ~isnan(Rpri)
    legend('Secondary (max reach)', 'Primary (well-conditioned)', ...
           'Location', 'northeast');
else
    legend('Secondary (max reach)', 'Location', 'northeast');
end
hold off
end

```

PlotRobot2.m

```

function PlotRobot2(Q,L,l0,col,fig,l1)
q1=Q(1); q2=Q(2); q3=Q(3); q4=Q(4);
l0=L(1); l1=L(2); l2=L(3); l3=L(4);

x0=0;
y0=0;
z0=l0;

%position of the end of the first link
R1 = l1 * cos(q2);
x1=R1*cos(q1);
y1=R1*sin(q1);
z1=z0+l1*sin(q2);

%position of the end of the second link
R2 = R1 + l2 * cos(q2+q3);
x2 = cos(q1) * R2;
y2 = sin(q1) * R2;
z2=z1+l2*sin(q2+q3);

%position of the end effector
R3 = R2 + l3 * cos(q2+q3+q4);
x3 = cos(q1) * R3;
y3 = sin(q1) * R3;
z3=z2+l3*sin(q2+q3+q4);

figure(fig); grid on;
set(l1,'XData', [0 x0*1e3 x1*1e3 x2*1e3 x3*1e3],...
     'YData', [0 y0*1e3 y1*1e3 y2*1e3 y3*1e3], 'ZData',...
     [0 z0*1e3 z1*1e3 z2*1e3 z3*1e3], 'color', col);
xlabel('X'); ylabel('Y'); zlabel('Z');
axis equal;

app=(l1+l2+l3)*1.1*1e3;
xlim([-app app]);
ylim([-app app]);
zlim([-20 app]);

drawnow;

end

```

Dynamics

ROBOTdirdin_4R.m

```
function S_din = ROBOTdirdin_4R(Q,L)

% direct kinematics
l0 = L(1); l1=L(2); l2=L(3); l3=L(4);
g1 = L(5); g2 = L(6); g3 = L(7);
g0=l0/2;
S= zeros(19,1);

%abbreviations
c1 = cos(Q(1)); s1 = sin(Q(1));
c2 = cos(Q(2)); s2 = sin(Q(2));
c3 = cos(Q(3)); s3 = sin(Q(3));
c4 = cos(Q(4)); s4 = sin(Q(4));
Q23 = Q(2) + Q(3);
Q234 = Q(2) + Q(3) + Q(4);
c23 = cos(Q23); s23 = sin(Q23);
c234 = cos(Q234); s234 = sin(Q234);
%pose of the EE
R = l1 * c2 + l2 * c23 + l3 * c234;

S(1) = c1 * R;
S(2) = l0 + l1 * s2 + l2 * s23 + l3 * s234; %z
S(3) = s1 * R;

S(4) = c1 * (R - (g3 * c234));
S(5) = l0 + l1 * s2 + l2 * s23 + g3 * s234;
S(6) = s1 * (R - (g3 * c234));
S(7) = Q234;

S(8) = c1 * (l1 * c2 + g2 * c23);
S(9) = l0 + l1 * s2 + g2 * s23;
S(10) = s1 * (l1 * c2 + g2 * c23);
S(11) = Q23;

S(12) = c1 * (g1 * c2);
S(13) = l0 + g1 * s2;
S(14) = s1 * (g1 * c2);
S(15) = Q2;

S(16) = 0;
S(17) = g0;
S(18) = 0;
S(19) = Q(1);

end
```

ROBOTjacdin_4R.m

```
function J_din = ROBOTjacdin_4R(Q,L)

% Lenghts
l0 = L(1); l1=L(2); l2=L(3); l3=L(4);
g1 = L(5); g2 = L(6); g3 = L(7);
q1 = Q(1); q2 = Q(2); q3 = Q(3); q4 = Q(4);

% Abbreviations
c1 = cos(q1); s1 = sin(q1);
c2 = cos(q2); s2 = sin(q2);
Q23 = q2 + q3;
Q234 = q2 + q3 + q4;
c23 = cos(Q23); s23 = sin(Q23);
c234 = cos(Q234); s234 = sin(Q234);
R = l1 * c2 + l2 * c23 + l3 * c234;
```

```

R_prime = R - (g3 * c234);
R_double_prime = l1 * c2 + g2 * c23;
R_triple_prime = g1 * c2;

dR_dQ2 = -l1 * s2 - l2 * s23 - l3 * s234;
dR_dQ3 = -l2 * s23 - l3 * s234;
dR_dQ4 = -l3 * s234;

dR_prime_dQ3 = dR_dQ3 + g3 * s234;
dR_prime_dQ4 = dR_dQ4 + g3 * s234;

dR_double_prime_dQ2 = -l1 * s2 - g2 * s23;
dR_double_prime_dQ3 = -g2 * s23;

dR_triple_prime_dQ2 = -g1 * s2;

% --- Jacobian (19x4) ---
J_din = zeros(19, 4);

J_din(1, 1) = -s1 * R;
J_din(1, 2) = c1 * dR_dQ2;
J_din(1, 3) = c1 * dR_dQ3;
J_din(1, 4) = c1 * dR_dQ4;

J_din(2, 1) = 0;
J_din(2, 2) = l1 * c2 + l2 * c23 + l3 * c234; % = R
J_din(2, 3) = l2 * c23 + l3 * c234;
J_din(2, 4) = l3 * c234;

J_din(3, 1) = c1 * R;
J_din(3, 2) = s1 * dR_dQ2;
J_din(3, 3) = s1 * dR_dQ3;
J_din(3, 4) = s1 * dR_dQ4;

J_din(4, 1) = -s1 * R_prime;
J_din(4, 2) = c1 * dR_dQ2;
J_din(4, 3) = c1 * dR_prime_dQ3;
J_din(4, 4) = c1 * dR_prime_dQ4;

J_din(5, 1) = 0;
J_din(5, 2) = l1 * c2 + l2 * c23 + g3 * c234;
J_din(5, 3) = l2 * c23 + g3 * c234;
J_din(5, 4) = g3 * c234;

J_din(6, 1) = c1 * R_prime;
J_din(6, 2) = s1 * dR_dQ2;
J_din(6, 3) = s1 * dR_prime_dQ3;
J_din(6, 4) = s1 * dR_prime_dQ4;

J_din(7, :) = [0, 1, 1, 1];

J_din(8, 1) = -s1 * R_double_prime;
J_din(8, 2) = c1 * dR_double_prime_dQ2;
J_din(8, 3) = c1 * dR_double_prime_dQ3;
J_din(8, 4) = 0;

J_din(9, 1) = 0;
J_din(9, 2) = l1 * c2 + g2 * c23;
J_din(9, 3) = g2 * c23;
J_din(9, 4) = 0;

J_din(10, 1) = c1 * R_double_prime;
J_din(10, 2) = s1 * dR_double_prime_dQ2;
J_din(10, 3) = s1 * dR_double_prime_dQ3;
J_din(10, 4) = 0;

J_din(11, :) = [0, 1, 1, 0];

J_din(12, 1) = -s1 * R_triple_prime;
J_din(12, 2) = c1 * dR_triple_prime_dQ2;

```

```

J_din(12, 3) = 0;
J_din(12, 4) = 0;

J_din(13, 1) = 0;
J_din(13, 2) = g1 * c2;
J_din(13, 3) = 0;
J_din(13, 4) = 0;

J_din(14, 1) = c1 * R_triple_prime;
J_din(14, 2) = s1 * dR_triple_prime_dQ2;
J_din(14, 3) = 0;
J_din(14, 4) = 0;

J_din(15, :) = [0, 1, 0, 0];

J_din(16, :) = [0, 0, 0, 0];

J_din(17, :) = [0, 0, 0, 0];

J_din(18, :) = [0, 0, 0, 0];

J_din(19, :) = [1, 0, 0, 0];

end

```

ROBOTjacPdin_4R.m

```

function J_dot_din = ROBOTjacPdin_4R(Q, Q_dot, L)

% Lengths
l0 = L(1); l1 = L(2); l2 = L(3); l3 = L(4);
g1 = L(5); g2 = L(6); g3 = L(7);
q1 = Q(1); q2 = Q(2); q3 = Q(3); q4 = Q(4);
q_dot1 = Q_dot(1); q_dot2 = Q_dot(2); q_dot3 = Q_dot(3); q_dot4 = Q_dot(4);

% Abbreviations
c1 = cos(q1); s1 = sin(q1);
c2 = cos(q2); s2 = sin(q2);
Q23 = q2 + q3; Q234 = q2 + q3 + q4;
c23 = cos(Q23); s23 = sin(Q23);
c234 = cos(Q234); s234 = sin(Q234);

c1_dot = -s1 * q_dot1; s1_dot = c1 * q_dot1;
c2_dot = -s2 * q_dot2; s2_dot = c2 * q_dot2;
Q23_dot = q_dot2 + q_dot3;
c23_dot = -s23 * Q23_dot; s23_dot = c23 * Q23_dot;
Q234_dot = Q23_dot + q_dot4;
c234_dot = -s234 * Q234_dot; s234_dot = c234 * Q234_dot;
R = l1 * c2 + l2 * c23 + l3 * c234;
R_prime = R - (g3 * c234);
R_double_prime = l1 * c2 + g2 * c23;
R_triple_prime = g1 * c2;
R_dot = l1 * c2_dot + l2 * c23_dot + l3 * c234_dot;
R_prime_dot = R_dot - g3 * c234_dot;
R_double_prime_dot = l1 * c2_dot + g2 * c23_dot;
R_triple_prime_dot = g1 * c2_dot;
dR_dQ2 = -l1 * s2 - l2 * s23 - l3 * s234;
dR_dQ3 = -l2 * s23 - l3 * s234;
dR_dQ4 = -l3 * s234;
dR_prime_dQ3 = dR_dQ3 + g3 * s234;
dR_prime_dQ4 = dR_dQ4 + g3 * s234;
dR_double_prime_dQ2 = -l1 * s2 - g2 * s23;
dR_double_prime_dQ3 = -g2 * s23;
dR_triple_prime_dQ2 = -g1 * s2;

dR_dQ2_dot = -l1 * s2_dot - l2 * s23_dot - l3 * s234_dot;
dR_dQ3_dot = -l2 * s23_dot - l3 * s234_dot;
dR_dQ4_dot = -l3 * s234_dot;

```

```

dR_prime_dQ3_dot = dR_dQ3_dot + g3 * s234_dot;
dR_prime_dQ4_dot = dR_dQ4_dot + g3 * s234_dot;

dR_double_prime_dQ2_dot = -l1 * s2_dot - g2 * s23_dot;
dR_double_prime_dQ3_dot = -g2 * s23_dot;

dR_triple_prime_dQ2_dot = -g1 * s2_dot;

% Jacobian derivative
J_dot_din = zeros(19, 4);

J_dot_din(1, 1) = -s1_dot * R - s1 * R_dot;
J_dot_din(1, 2) = c1_dot * dR_dQ2 + c1 * dR_dQ2_dot;
J_dot_din(1, 3) = c1_dot * dR_dQ3 + c1 * dR_dQ3_dot;
J_dot_din(1, 4) = c1_dot * dR_dQ4 + c1 * dR_dQ4_dot;

J_dot_din(2, 1) = 0;
J_dot_din(2, 2) = R_dot;
J_dot_din(2, 3) = l2 * c23_dot + l3 * c234_dot;
J_dot_din(2, 4) = l3 * c234_dot;

J_dot_din(3, 1) = c1_dot * R + c1 * R_dot;
J_dot_din(3, 2) = s1_dot * dR_dQ2 + s1 * dR_dQ2_dot;
J_dot_din(3, 3) = s1_dot * dR_dQ3 + s1 * dR_dQ3_dot;
J_dot_din(3, 4) = s1_dot * dR_dQ4 + s1 * dR_dQ4_dot;

J_dot_din(4, 1) = -s1_dot * R_prime - s1 * R_prime_dot;
J_dot_din(4, 2) = c1_dot * dR_dQ2 + c1 * dR_dQ2_dot;
J_dot_din(4, 3) = c1_dot * dR_prime_dQ3 + c1 * dR_prime_dQ3_dot;
J_dot_din(4, 4) = c1_dot * dR_prime_dQ4 + c1 * dR_prime_dQ4_dot;

J_dot_din(5, 1) = 0;
J_dot_din(5, 2) = l1 * c2_dot + l2 * c23_dot + g3 * c234_dot;
J_dot_din(5, 3) = l2 * c23_dot + g3 * c234_dot;
J_dot_din(5, 4) = g3 * c234_dot;

J_dot_din(6, 1) = c1_dot * R_prime + c1 * R_prime_dot;
J_dot_din(6, 2) = s1_dot * dR_dQ2 + s1 * dR_dQ2_dot;
J_dot_din(6, 3) = s1_dot * dR_prime_dQ3 + s1 * dR_prime_dQ3_dot;
J_dot_din(6, 4) = s1_dot * dR_prime_dQ4 + s1 * dR_prime_dQ4_dot;

J_dot_din(7, :) = [0, 0, 0, 0];

J_dot_din(8, 1) = -s1_dot * R_double_prime - s1 * R_double_prime_dot;
J_dot_din(8, 2) = c1_dot * dR_double_prime_dQ2 + c1 * dR_double_prime_dQ2_dot;
J_dot_din(8, 3) = c1_dot * dR_double_prime_dQ3 + c1 * dR_double_prime_dQ3_dot;
J_dot_din(8, 4) = 0;

J_dot_din(9, 1) = 0;
J_dot_din(9, 2) = l1 * c2_dot + g2 * c23_dot;
J_dot_din(9, 3) = g2 * c23_dot;
J_dot_din(9, 4) = 0;

J_dot_din(10, 1) = c1_dot * R_double_prime + c1 * R_double_prime_dot;
J_dot_din(10, 2) = s1_dot * dR_double_prime_dQ2 + s1 * dR_double_prime_dQ2_dot;
J_dot_din(10, 3) = s1_dot * dR_double_prime_dQ3 + s1 * dR_double_prime_dQ3_dot;
J_dot_din(10, 4) = 0;

J_dot_din(11, :) = [0, 0, 0, 0];

J_dot_din(12, 1) = -s1_dot * R_triple_prime - s1 * R_triple_prime_dot;
J_dot_din(12, 2) = c1_dot * dR_triple_prime_dQ2 + c1 * dR_triple_prime_dQ2_dot;
J_dot_din(12, 3) = 0;
J_dot_din(12, 4) = 0;

J_dot_din(13, 1) = 0;
J_dot_din(13, 2) = g1 * c2_dot;
J_dot_din(13, 3) = 0;
J_dot_din(13, 4) = 0;

```



```

J_dot_din(14, 1) = c1_dot * R_triple_prime + c1 * R_triple_prime_dot;
J_dot_din(14, 2) = s1_dot * dR_triple_prime_dQ2 + s1 * dR_triple_prime_dQ2_dot;
J_dot_din(14, 3) = 0;
J_dot_din(14, 4) = 0;

J_dot_din(15, :) = [0, 0, 0, 0];

J_dot_din(16, :) = [0, 0, 0, 0];
J_dot_din(17, :) = [0, 0, 0, 0];
J_dot_din(18, :) = [0, 0, 0, 0];

J_dot_din(19, :) = [0, 0, 0, 0];
end

```

Task 1

min_rise_time.m

```
function [T_rise] = min_rise_time(motor,dq)

[T_lim,dq_lim] = limit_case(motor)

if dq < dq_lim %case1

    T_rise = ((sqrt(motor.A/motor.D)+sqrt(motor.D/motor.A))*...
        sqrt(2*dq*1/(motor.A+motor.D)));
else
    T_rise = dq/motor.V+motor.V*(motor.A+motor.D)/(2*motor.A*motor.D); %case2
end

end
```

limit_case.m

```
function [T_lim,dq_lim] = limit_case(motor)

T_lim = motor.V * (1/motor.A+1/motor.D);
dq_lim = 0.5 * motor.V^2 * (1/motor.A+1/motor.D);

end
```

LinesParabolas_task1.m

```
clearvars
close all
clc

%% --- ROBOT PARAMETERS ---
% Link lengths [m]
l0 = 200e-3;
l1 = 329e-3;
l2 = 311.50e-3;
l3 = 106e-3;
r0 = 60e-3;
r123 = 40e-3;

% Joint limits
q1_min = deg2rad(-180);    q1_max = deg2rad(180);
q2_min = deg2rad(-120);    q2_max = deg2rad(120);
q3_min = deg2rad(-180);    q3_max = deg2rad(180);
q4_min = deg2rad(-180);    q4_max = deg2rad(180);

joint_limits = [q1_min q1_max;
                q2_min q2_max;
                q3_min q3_max;
                q4_min q4_max];

% Center of mass lengths [m]
g1=l1/2;
g2=l2/2;
g3=l3/2;

% masses [kg]
m0 = 17;
m1 = 10;
m2 = 9;
m3 = 3;
m_payload=0.5;
```

```

% Inertias [kg*m^2]
I0_xy = (1/4)*m0*r0^2 + (1/12)*m0*l0^2;
I0_z = (1/2)*m0*r0^2;
I1_yz = (1/4)*m1*r123^2 + (1/12)*m1*l1^2;
I1_x = (1/2)*m1*r123^2;
I2_yz = (1/4)*m2*r123^2 + (1/12)*m2*l2^2;
I2_x = (1/2)*m2*r123^2;
I3_yz = (1/4)*m3*r123^2 + (1/12)*m3*l3^2;
I3_x = (1/2)*m3*r123^2;

% Mass Matrix
M=diag([m_payload, m_payload, m_payload, m3, m3, m3, I3_yz,...
        m2, m2, m2, I2_yz, m1, m1, m1, I1_yz, m0, m0, m0, ...
        I0_z]);

% External forces
Fse_base = zeros(19,1);

Fse_base(2) = -m_payload*9.81; % Fz payload
Fse_base(5) = -m3*9.81;        % Fz m3
Fse_base(9) = -m2*9.81;        % Fz m2
Fse_base(13) = -m1*9.81;       % Fz m1
Fse_base(17) = -m0*9.81;       % Fz m0

% Helmet Visor reaction Force
F_helmet = 4; % [N]

%% --- Lines&Parabolas Motion Planning ---

% Waypoints in S-space
X = [250 270 300 340 370 420 500 420 370 340 300 250] * 1e-3;
Z = [ 75 75 75 150 180 220 230 220 180 150 75 75] * 1e-3;
Teta = deg2rad([0 5 5 -10 -15 -25 -35 -25 -15 -10 5 0]);
Y = zeros(size(X));

L=[l0;l1;l2;l3;g1;g2;g3]; % [m] %lengths vector

nP = numel(X);
S_points = [X;
            Y;
            Z;
            Teta];

% S-coordinates of helmet approaching and tolerances
X_helmet = X(3); % [m]
Z_helmet = Z(3); % [m]
dx = 0.01; % [m]
dz = 0.01; % [m]

%% Inverse Kinematics

nJ = 4; % number of joints
Q = zeros(nJ, nP);

for i = 1:nP
    Q(:,i) = ROBOTinv_4R(S_points(:,i), L, -1);
end

disp('Q (joint values per waypoint) [deg]:');
disp(rad2deg(Q));

% --- Joint limits check on waypoints ---

for i = 1:nP
    if any(Q(:,i) < joint_limits(:,1) | Q(:,i) > joint_limits(:,2))
        error('ERROR: waypoint %d NOT reachable within joint limits.', i);
    end
end

```

```

q1 = Q(1,:);
q2 = Q(2,:);
q3 = Q(3,:);
q4 = Q(4,:);

Q = [q1; q2; q3; q4];

dq1=diff(q1);
dq2=diff(q2);
dq3=diff(q3);
dq4=diff(q4);

dQ=[dq1;dq2;dq3;dq4];
dQ_abs= abs(dQ);

%% --- MOTORS PARAMETERS ---
motor.A_lim = 600; % max motor acc [rad/s^2]
motor.V_lim = 600; % max motor vel [rad/s]
motor.D_lim = 600; % max motor dec[rad/s^2]

% Servo motor HG-KN 23 (B)J (brake+resolver)
Cn =0.64; % Nominal Torque [Nm]
mot_inertia = 0.247*10^-4; % [kg*m^2]
rid = 1/100;
alpha = Cn^2/mot_inertia; % Accererating Factor
%% --- JOINTS PARAMETERS ---
motor.A = motor.A_lim * rid; % max joint acc[rad/s^2]
motor.V = motor.V_lim * rid; % max joint vel [rad/s]
motor.D = motor.D_lim * rid; % max joint dec[rad/s^2]

%% Simulation sampling time
dt = 0.001;

%% 0) Calculate minimum actuation time
rise_time=zeros(4,size(dQ,2));
for i=1:size(dQ,2)
    rise_time(1,i)=min_rise_time(motor,dQ_abs(1,i));
    rise_time(2,i)=min_rise_time(motor,dQ_abs(2,i));
    rise_time(3,i)=min_rise_time(motor,dQ_abs(3,i));
    rise_time(4,i)=min_rise_time(motor,dQ_abs(4,i));
end

T_min_act= max(rise_time);

%% 1) Auxiliary point's transition time
t1= T_min_act(1)-sqrt(T_min_act(1)^2-2*dQ_abs(:,1)/motor.A);
tend=T_min_act(end)-sqrt(T_min_act(end)^2-2*dQ_abs(:,end)/motor.D);

%% 2) Internal parabolic transition tj;
min_act_time_joints = [T_min_act;T_min_act;T_min_act;T_min_act];
min_act_time_joints(:,1)=T_min_act(1)-0.5*t1;
min_act_time_joints(:,end)=T_min_act(end)-0.5*tend;

q_dot=dQ./min_act_time_joints;

t_j = abs(diff(q_dot,[],2) / motor.A);

t_j=[t1,t_j,tend];

%% 3) Time_vector
dt_j = zeros(4, size(min_act_time_joints,2));
for i=1:size(min_act_time_joints,2)
    dt_j(:,i)= min_act_time_joints(:,i)-0.5*(t_j(:,i)+t_j(:,i+1));
end

dt_j=[[0;0;0;0],dt_j];

```

```

t_vec= zeros(4,length(t_j)+length(dt_j));
for i=1:length(t_j)
    t_vec(:,i*2-1)=dt_j(:,i);
    t_vec(:,i*2)=t_j(:,i);
end

t_vec=cumsum(t_vec,2);

%% 4) Line coefficients
t_line_vec= cumsum([t1/2,min_act_time_joints],2);

al=q_dot;
bl= Q(:,2:end)-al.*t_line_vec(:,2:end);

%% 5) Parabolic coefficient
q_dot_ext= [[0;0;0;0], q_dot,[0; 0;0;0] ];
Q_end_parabola =[ al, al(:,end)].*t_vec(:, 2:2:end) +[bl,bl(:,end)];

ap = diff(q_dot_ext, [], 2)./(2*t_j);
bp = [q_dot, [0;0;0;0]] -ap*2.*t_vec(:, 2:2:end);
cp = (Q_end_parabola -ap.*t_vec(:, 2:2:end).^2-bp.*t_vec(:, 2:2:end));

static_joints = all(dQ_abs == 0, 2);
if any(static_joints)
    al(static_joints,:) = 0;
    ap(static_joints,:) = 0;
    bp(static_joints,:) = 0;
    cp(static_joints,:) = Q(static_joints,1);
    t_j(static_joints,:) = 1;
end

%% 6) Complete Trajectory for each Joint

nPar = size(ap,2);      % n.of parabolic branches

% -- Joint 1 --
t1      = [];
q1_traj = [];
qp1_traj= [];
qpp1_traj=[];

for k = 1:nPar

    t_start_p = t_vec(1, 2*k-1);
    t_end_p   = t_vec(1, 2*k);
    tp = t_start_p:dt:t_end_p;

    if ~isempty(t1) && ~isempty(tp)
        tp = tp(2:end);
    end

    q_p   = ap(1,k).*tp.^2 + bp(1,k).*tp + cp(1,k);
    qp_p  = 2*ap(1,k).*tp + bp(1,k);
    qpp_p = 2*ap(1,k)*ones(size(tp));

    t1      = [t1,      tp];
    q1_traj = [q1_traj, q_p];
    qp1_traj = [qp1_traj, qp_p];
    qpp1_traj=[qpp1_traj,qpp_p];

    % --- Line between parabolas k and k+1 ---
    if k < nPar
        t_start_l = t_vec(1, 2*k);
        t_end_l   = t_vec(1, 2*k+1);
        t1 = t_start_l:dt:t_end_l;

        if ~isempty(t1)
            t1 = t1(2:end);
        end
    end
end

```

```

end

q_1 = a1(1,k).*t1 + b1(1,k);
qp_1 = a1(1,k)*ones(size(t1));
qpp_1 = zeros(size(t1));

t1 = [t1, t1];
q1_traj = [q1_traj, q_1];
qp1_traj = [qp1_traj, qp_1];
qpp1_traj=[qpp1_traj,qpp_1];
end
end

% -- Joint 2 --
t2 = [];
q2_traj = [];
qp2_traj= [];
qpp2_traj=[];

for k = 1:nPar

    t_start_p = t_vec(2, 2*k-1);
    t_end_p = t_vec(2, 2*k);
    tp = t_start_p:dt:t_end_p;

    if ~isempty(t2) && ~isempty(tp)
        tp = tp(2:end);
    end

    q_p = ap(2,k).*tp.^2 + bp(2,k).*tp + cp(2,k);
    qp_p = 2*ap(2,k).*tp + bp(2,k);
    qpp_p = 2*ap(2,k)*ones(size(tp));

    t2 = [t2, tp];
    q2_traj = [q2_traj, q_p];
    qp2_traj = [qp2_traj, qp_p];
    qpp2_traj=[qpp2_traj,qpp_p];

    if k < nPar
        t_start_l = t_vec(2, 2*k);
        t_end_l = t_vec(2, 2*k+1);
        t1 = t_start_l:dt:t_end_l;

        if ~isempty(t1)
            t1 = t1(2:end);
        end

        q_1 = a1(2,k).*t1 + b1(2,k);
        qp_1 = a1(2,k)*ones(size(t1));
        qpp_1 = zeros(size(t1));

        t2 = [t2, t1];
        q2_traj = [q2_traj, q_1];
        qp2_traj = [qp2_traj, qp_1];
        qpp2_traj=[qpp2_traj,qpp_1];
    end
end

% -- Joint 3 --
t3 = [];
q3_traj = [];
qp3_traj= [];
qpp3_traj=[];

for k = 1:nPar

    t_start_p = t_vec(3, 2*k-1);
    t_end_p = t_vec(3, 2*k);
    tp = t_start_p:dt:t_end_p;

```

```

if ~isempty(t3) && ~isempty(tp)
    tp = tp(2:end);
end

q_p = ap(3,k).*tp.^2 + bp(3,k).*tp + cp(3,k);
qp_p = 2*ap(3,k).*tp + bp(3,k);
qpp_p = 2*ap(3,k)*ones(size(tp));

t3 = [t3, tp];
q3_traj = [q3_traj, q_p];
qp3_traj = [qp3_traj, qp_p];
qpp3_traj=[qpp3_traj,qpp_p];

if k < nPar
    t_start_l = t_vec(3, 2*k);
    t_end_l = t_vec(3, 2*k+1);
    t1 = t_start_l:dt:t_end_l;

    if ~isempty(t1)
        t1 = t1(2:end);
    end

    q_l = a1(3,k).*t1 + b1(3,k);
    qp_l = a1(3,k)*ones(size(t1));
    qpp_l = zeros(size(t1));

    t3 = [t3, t1];
    q3_traj = [q3_traj, q_l];
    qp3_traj = [qp3_traj, qp_l];
    qpp3_traj=[qpp3_traj,qpp_l];
end
end

% -- Joint 4 --
t4 = [];
q4_traj = [];
qp4_traj= [];
qpp4_traj=[];

for k = 1:nPar

    t_start_p = t_vec(4, 2*k-1);
    t_end_p = t_vec(4, 2*k);
    tp = t_start_p:dt:t_end_p;

    if ~isempty(t4) && ~isempty(tp)
        tp = tp(2:end);
    end

    q_p = ap(4,k).*tp.^2 + bp(4,k).*tp + cp(4,k);
    qp_p = 2*ap(4,k).*tp + bp(4,k);
    qpp_p = 2*ap(4,k)*ones(size(tp));

    t4 = [t4, tp];
    q4_traj = [q4_traj, q_p];
    qp4_traj = [qp4_traj, qp_p];
    qpp4_traj=[qpp4_traj,qpp_p];

    if k < nPar
        t_start_l = t_vec(4, 2*k);
        t_end_l = t_vec(4, 2*k+1);
        t1 = t_start_l:dt:t_end_l;

        if ~isempty(t1)
            t1 = t1(2:end);
        end

        q_l = a1(4,k).*t1 + b1(4,k);
        qp_l = a1(4,k)*ones(size(t1));
        qpp_l = zeros(size(t1));

```

```

        t4      = [t4,      t1];
        q4_traj = [q4_traj, q_1];
        qp4_traj = [qp4_traj, qp_1];
        qpp4_traj=[qpp4_traj,qpp_1];
    end
end

%% 6b) Construction of a unique t and q vector

% joint 1 as reference
t_all = t1;
N = length(t_all);

% Matrices 4xN
q_all  = zeros(4, N);
qp_all = zeros(4, N);
qpp_all = zeros(4, N);

% --- Joint 1 ---
for k = 1:N
    q_all(1,k)  = q1_traj(k);
    qp_all(1,k) = qp1_traj(k);
    qpp_all(1,k) = qpp1_traj(k);
end

% --- Joint 2 ---
N2 = length(t2);
for k = 1:N
    if k <= N2
        q_all(2,k)  = q2_traj(k);
        qp_all(2,k) = qp2_traj(k);
        qpp_all(2,k) = qpp2_traj(k);
    else
        q_all(2,k)  = q2_traj(N2);
        qp_all(2,k) = qp2_traj(N2);
        qpp_all(2,k) = qpp2_traj(N2);
    end
end

% --- Joint 3 ---
N3 = length(t3);
for k = 1:N
    if k <= N3
        q_all(3,k)  = q3_traj(k);
        qp_all(3,k) = qp3_traj(k);
        qpp_all(3,k) = qpp3_traj(k);
    else
        q_all(3,k)  = q3_traj(N3);
        qp_all(3,k) = qp3_traj(N3);
        qpp_all(3,k) = qpp3_traj(N3);
    end
end

% --- Joint 4 ---
N4 = length(t4);
for k = 1:N
    if k <= N4
        q_all(4,k)  = q4_traj(k);
        qp_all(4,k) = qp4_traj(k);
        qpp_all(4,k) = qpp4_traj(k);
    else
        q_all(4,k)  = q4_traj(N4);
        qp_all(4,k) = qp4_traj(N4);
        qpp_all(4,k) = qpp4_traj(N4);
    end
end
end

```



```

%% --- Kinematics quality index  $k^{-1}$  along trajectory ---

nt = numel(t_all);
Kinv = zeros(1, nt);

for k = 1:nt
    Qk = q_all(:,k);

    J = ROBOTjac_4R(Qk, L);

    Jv = J(1:3,:);

    s = svd(Jv);
    smax = max(s);
    smin = min(s);

    if smax < 1e-8
        Kinv(k) = 0;
    else
        Kinv(k) = smin/smax;
    end
end

% Results
fprintf('\n=== Quality Index results ===\n');
fprintf('k^{-1}_min = %.4f\n', min(Kinv)); % min  $K^{-1}$ 
fprintf('k^{-1}_max = %.4f\n', max(Kinv)); % max  $K^{-1}$ 

figure;
plot(t_all, Kinv, 'LineWidth', 1.5);
grid on;
xlabel('Time [s]');
ylabel('k^{-1}');
title('Quality Index vs Time');
%  $K^{-1}$  threshold
thr_primary = 0.2;
yline(thr_primary, '--r', 'Primary threshold');
legend('k^{-1}(t)', 'thr_{primary}');

%-----%

%% --- TORQUES ---
nt = numel(t_all);
q1t_save = zeros(1, nt);
q2t_save = zeros(1, nt);
q3t_save = zeros(1, nt);
q4t_save = zeros(1, nt);
qt_save = zeros(4, nt);

q1tm_save = zeros(1, nt);
q2tm_save = zeros(1, nt);
q3tm_save = zeros(1, nt);
q4tm_save = zeros(1, nt);

F_helmet_on = zeros(1, nt);
for k = 1:nt
    t_helmet = t_all(k);
    qk = q_all(:,k); % 4x1
    qdk = qp_all(:,k); % 4x1
    qddk = qpp_all(:,k); % 4x1

    % Dynamic Jacobian
    J_din = ROBOTjacdin_4R(qk, L);
    J_dot_din = ROBOTjacPdin_4R(qk, qdk, L);

    % Extended accelerations
    Sdpp = J_dot_din*qdk + J_din*qddk;

```

```

% Inertial forces
Fsi = -M*Sdpp;

% External forces
Fse = Fse_base;

% Check the EE position
Se = ROBOTdir_4R(qk, L);
Xee = Se(1);
Zee = Se(3);

% Apply the Helmet Visor reaction force
if abs(Xee - X_helmet) <= dx && abs(Zee - Z_helmet) <= dz...
    && qp_all(4,k)<0

    Fse(2) = Fse(2) - F_helmet;
    F_helmet_on(k) = -F_helmet;

elseif abs(Xee - X_helmet) <= dx && abs(Zee - Z_helmet) <= dz...
    && qp_all(4,k)>0

    Fse(2) = Fse(2) + F_helmet;
    F_helmet_on(k) = F_helmet;
end

% Total Forces
Fs = Fse + Fsi;

% Joints Torques
Fcq = -J_din.' * Fs;

q1t_save(k) = Fcq(1);
q2t_save(k) = Fcq(2);
q3t_save(k) = Fcq(3);
q4t_save(k) = Fcq(4);
qt_save(:,k) = Fcq;

app_1(k) = q1t_save(k) * qpp_all(1,k);
app_2(k) = q2t_save(k) * qpp_all(2,k);
app_3(k) = q3t_save(k) * qpp_all(3,k);
app_4(k) = q4t_save(k) * qpp_all(4,k);
app(:,k) = Fcq .* qpp_all(:,k);

% Motor Torques
q1tm_save(k) = q1t_save(k)*rid + mot_inertia*qpp_all(1,k)*(1/rid);
q2tm_save(k) = q2t_save(k)*rid + mot_inertia*qpp_all(2,k)*(1/rid);
q3tm_save(k) = q3t_save(k)*rid + mot_inertia*qpp_all(3,k)*(1/rid);
q4tm_save(k) = q4t_save(k)*rid + mot_inertia*qpp_all(4,k)*(1/rid);
end

% Load Factor check
beta = zeros(1,nJ);
rid_opt = zeros(1,nJ);
rid_max = zeros(1,nJ);
rid_min = zeros(1,nJ);

for p=1:nJ

    wrq(p) = rms(qp_all(p,:));
    dwrq(p) = rms(qpp_all(p,:));
    Crsq(p) = rms(qt_save(p,:));
    dwCm(p) = mean(app(p,:));

    rid_opt = sqrt(mot_inertia*dwrq(p)/Crsq(p));

    beta(p) = ((rid/sqrt(mot_inertia)) * Crsq(p) - (sqrt(mot_inertia)/rid) * wrq(p))^2 +
2*(Crsq(p)*dwrq(p) + dwCm(p));

```

```

    rid_max(p) = (sqrt(mot_inertia)*(sqrt(alpha-beta(p)+4*dwrq(p)*Crsq(p))...
        + sqrt(alpha - beta(p)))/(2*Crsq(p));

    rid_min(p) = (sqrt(mot_inertia)*(sqrt(alpha-beta(p)+4*dwrq(p)*Crsq(p))...
        - sqrt(alpha - beta(p)))/(2*Crsq(p));

end

figure; hold on; grid on;

joints = 1:4;

% Punti beta
scatter(joints, beta, 80, 'b', 'filled');

% Threshold alpha
yline(alpha, 'r--', 'LineWidth', 1.8);

xlabel('Joint index');
ylabel('\beta');
title('Motor Sizing Check: \beta values vs. \alpha threshold');
xticks(joints);

legend('\beta_j', '\alpha threshold');

if all(beta < alpha)
    fprintf("All joints satisfy the motor sizing condition  $\alpha > \beta$ .\n");
else
    fprintf("WARNING: Some joints violate  $\alpha > \beta$ .\n");
    idx = find(beta >= alpha);
    fprintf("Joints which do not respect the condition: ");
    fprintf("%d ", idx);
    fprintf("\n");
end

figure; hold on; grid on;

joints = 1:4;

% Plot rid_min
scatter(joints, rid_min, 90, 'g', 'filled');

% Plot rid_max
scatter(joints, rid_max, 90, 'm', 'filled');

% rid
yline(rid, 'b-', 'LineWidth', 1.6);

xlabel('Joint index');
ylabel('Reduction ratio r_{id}');
title('Motor Reduction Ratio Check');
xticks(joints);

legend('r_{id}^{\min}', 'r_{id}^{\max}', 'chosen r_{id}', ...
    'Location', 'best');

%% --- TORQUES PLOTS ---

figure; % Joints Torques plots
subplot(2,2,1); plot(t_all,q1t_save); grid on; xlabel('Time [s]'); ylabel('T_{joint [Nm]}'); title('q1 torque');
subplot(2,2,2); plot(t_all,q2t_save); grid on; xlabel('Time [s]'); ylabel('T_{joint [Nm]}'); title('q2 torque');
subplot(2,2,3); plot(t_all,q3t_save); grid on; xlabel('Time [s]'); ylabel('T_{joint [Nm]}'); title('q3 torque');
subplot(2,2,4); plot(t_all,q4t_save); grid on; xlabel('Time [s]'); ylabel('T_{joint [Nm]}'); title('q4 torque');

```

```

figure; % Motor Torques plots
grid on;
subplot(2,2,1); plot(t_all,q1tm_save); xlabel('Time [s]'); grid on; ylabel('T\_motor [Nm]'); title('q1
motor torque');
subplot(2,2,2); plot(t_all,q2tm_save); xlabel('Time [s]'); grid on; ylabel('T\_motor [Nm]'); title('q2
motor torque');
subplot(2,2,3); plot(t_all,q3tm_save); xlabel('Time [s]'); grid on; ylabel('T\_motor [Nm]'); title('q3
motor torque');
subplot(2,2,4); plot(t_all,q4tm_save); xlabel('Time [s]'); grid on; ylabel('T\_motor [Nm]'); title('q4
motor torque');

figure; % Helmet's Visor Reaction Force Logic
plot(t_all, F_helmet_on, 'LineWidth', 1);
grid on;
xlabel('Time [s]');
ylabel('F\_helmet [N]');
title('Visor reaction force');

figure; % Pos
sgtitle('Joint Positions');

for j = 1:4
    subplot(2,2,j); hold on; grid on;
    plot(t_all, rad2deg(q_all(j,:)), 'LineWidth', 1.2);
    xlabel('t [s]');
    ylabel('q [deg]');
    title(sprintf('Joint %d - Position', j));
end

figure; % Vel
sgtitle('Joint Velocities');

for j = 1:4
    subplot(2,2,j); hold on; grid on;
    plot(t_all, qp_all(j,:), 'LineWidth', 1.2);
    xlabel('t [s]');
    ylabel('q\_dot [rad/s]');
    title(sprintf('Joint %d - Velocity', j));
end

figure; % Acc
sgtitle('Joint Accelerations');

for j = 1:4
    subplot(2,2,j); hold on; grid on;
    plot(t_all, qpp_all(j,:), 'LineWidth', 1.2);
    xlabel('t [s]');
    ylabel('q\_ddot [rad/s^2]');
    title(sprintf('Joint %d - Acceleration', j));
end

%% --- EE Trajectory in S-space ---

S_all = zeros(4, nt);
for k = 1:nt
    S_all(:,k) = ROBOTdir_4R(q_all(:,k), L);
end

figure;
plot(S_all(1,:), S_all(3,:), 'b-', 'LineWidth', 2); hold on;
plot(S_all(1,1), S_all(3,1), 'go', 'MarkerFaceColor', 'g'); % start
plot(S_all(1,end), S_all(3,end), 'ro', 'MarkerFaceColor', 'r'); % end

for p = 1:nP
    Sp = ROBOTdir_4R(Q(:,p), L);
    plot(Sp(1), Sp(3), 'kx', 'MarkerSize', 10, 'LineWidth', 2); % waypoints
end

```

```

grid on; axis equal; box on;
xlabel('X [m]');
ylabel('Z [m]');
title('End-Effector trajectory (XZ-plan)');

%% --- Animation ---
f1 = figure;
f1.WindowStyle='normal';
hold on;
grid on;
xlabel('X [mm]');
ylabel('Z [mm]');
axis equal;

app = (l1 + l2 + l3)*1.1*1e3;
xlim([-app app]);
ylim([-app app]);

% Linea robot (già la usi)
l11 = line('XData', [0 0], ...
           'YData', [0 0], ...
           'linestyle', '-', 'linewidth',2, 'color','r', ...
           'marker','o', 'markersize',6, 'markerfacecolor','k');

% Linea SCIA (vuota all'inizio)
trail = line('XData', [], ...
            'YData', [], ...
            'Color', [0 0.3 1], ...
            'LineWidth', 1.5);

% vettori per salvare la scia accumulata
trail_X = [];
trail_Z = [];

for k = 1:size(q_all,2)

    % --- Plot robot istantaneo ---
    PlotRobot3(q_all(:,k), L, l0, 'r', f1, l11);

    % --- Aggiorna SCIA ---
    Se = ROBOTdir_4R(q_all(:,k), L); % end-effector in [X;Y;Z;theta]
    Xe = Se(1)*1e3; % in mm
    Ze = Se(3)*1e3;

    trail_X(end+1) = Xe;
    trail_Z(end+1) = Ze;

    set(trail, 'XData', trail_X, 'YData', trail_Z);

    drawnow;
end

%% --Simscape utilities--

%Signal editor

%joint1
tt=t_all;
ww1=qp_all(1,:);
signal1 = Simulink.SimulationData.Signal;
signal1.Name = 'vel1_task1';
signal1.Values = timeseries(ww1*(1/rid), tt);
ds1_task1 = Simulink.SimulationData.Dataset;
ds1_task1 = addElement(ds1_task1, signal1, 'velocity');
save('vel1_task1.mat', 'ds1_task1');

%joint2
tt=t_all;
ww2=qp_all(2,:);

```

```

signal2 = Simulink.SimulationData.Signal;
signal2.Name = 'vel2_task1';
signal2.Values = timeseries(ww2*(1/rid), tt);
ds2_task1 = Simulink.SimulationData.Dataset;
ds2_task1 = addElement(ds2_task1, signal2, 'velocity');
save('vel2_task1.mat', 'ds2_task1');

%joint3
tt=t_all;
ww3=qp_all(3,:);
signal3 = Simulink.SimulationData.Signal;
signal3.Name = 'vel3_task1';
signal3.Values = timeseries(ww3*(1/rid), tt);
ds3_task1 = Simulink.SimulationData.Dataset;
ds3_task1 = addElement(ds3_task1, signal3, 'velocity');
save('vel3_task1.mat', 'ds3_task1');

%joint4
tt=t_all;
ww4=qp_all(4,:);
signal4 = Simulink.SimulationData.Signal;
signal4.Name = 'vel4_task1';
signal4.Values = timeseries(ww4*(1/rid), tt);
ds4_task1 = Simulink.SimulationData.Dataset;
ds4_task1 = addElement(ds4_task1, signal4, 'velocity');
save('vel4_task1.mat', 'ds4_task1');

% START SIMULATION
sim('Simscape_task1_motors.slx');

% To-Workspace
figure;
sgtitle('Simscape Motor Torques');
subplot(2,2,1); plot(MotorTorque1.Time, MotorTorque1.Data,'color','r'); grid on; xlabel('Time [s]');
ylabel('T\_motor [Nm]'); title('q1 motor torque'); ylim([-0.09 0.09]);
subplot(2,2,2); plot(MotorTorque2.Time, MotorTorque2.Data,'color','r'); grid on; xlabel('Time [s]');
ylabel('T\_motor [Nm]'); title('q2 motor torque');
subplot(2,2,3); plot(MotorTorque3.Time, MotorTorque3.Data,'color','r'); grid on; xlabel('Time [s]');
ylabel('T\_motor [Nm]'); title('q3 motor torque');
subplot(2,2,4); plot(MotorTorque4.Time, MotorTorque4.Data,'color','r'); grid on; xlabel('Time [s]');
ylabel('T\_motor [Nm]'); title('q4 motor torque');

figure; % Matlab/SIMSCAPE comparison
sgtitle('MATLAB-Simscape Motor Torques comparison');
subplot(2,2,1); plot(MotorTorque1.Time, MotorTorque1.Data,'color','r'); grid on; xlabel('Time [s]');
ylabel('T\_motor [Nm]'); title('q1 motor torque'); ylim([-0.09 0.09]); hold on;
subplot(2,2,2); plot(MotorTorque2.Time, MotorTorque2.Data,'color','r'); grid on; xlabel('Time [s]');
ylabel('T\_motor [Nm]'); title('q2 motor torque'); hold on;
subplot(2,2,3); plot(MotorTorque3.Time, MotorTorque3.Data,'color','r'); grid on; xlabel('Time [s]');
ylabel('T\_motor [Nm]'); title('q3 motor torque'); hold on;
subplot(2,2,4); plot(MotorTorque4.Time, MotorTorque4.Data,'color','r'); grid on; xlabel('Time [s]');
ylabel('T\_motor [Nm]'); title('q4 motor torque'); hold on;
subplot(2,2,1); plot(t_all,q1tm_save,'color','b'); xlabel('Time [s]'); grid on; ylabel('T\_motor [Nm]');
title('q1 motor torque');
subplot(2,2,2); plot(t_all,q2tm_save,'color','b'); xlabel('Time [s]'); grid on; ylabel('T\_motor [Nm]');
title('q2 motor torque');
subplot(2,2,3); plot(t_all,q3tm_save,'color','b'); xlabel('Time [s]'); grid on; ylabel('T\_motor [Nm]');
title('q3 motor torque');
subplot(2,2,4); plot(t_all,q4tm_save,'color','b'); xlabel('Time [s]'); grid on; ylabel('T\_motor [Nm]');
title('q4 motor torque');

```

Task 2

Sshape.m

```
function res=Sshape(t,S0,dS,t1,t2,t3)
V=(dS)/(t1/2 + (t2-t1) + (t3-t2)/2);
A=V/t1;
D=V/(t3-t2);
if t<t1
    res.pos=S0+1/2*A*t^2;
    res.vel=A*t;
    res.acc=A;
elseif t<t2
    res.pos=S0+1/2*A*t1^2+V*(t-t1);
    res.vel=V;
    res.acc=0;
else
    res.pos=S0+dS-D*(t3-t)^2/2;
    res.vel=V-D*(t-t2);
    res.acc=-D;
end
end
```

Sshape_task2.m

```
clear; close all; clc;

%% --- ROBOT PARAMETERS ---

% Link lengths [m]
l0 = 200e-3;
l1 = 329e-3;
l2 = 311.50e-3;
l3 = 106e-3;
r0 = 60e-3;
r123 = 40e-3;

% Joint limits
q1_min = deg2rad(-180);    q1_max = deg2rad(180);
q2_min = deg2rad(-180);    q2_max = deg2rad(180);
q3_min = deg2rad(-180);    q3_max = deg2rad(180);
q4_min = deg2rad(-180);    q4_max = deg2rad(180);

joint_limits = [q1_min q1_max;
                q2_min q2_max;
                q3_min q3_max;
                q4_min q4_max];

% Center of mass lengths [m]
g1 = l1/2;
g2 = l2/2;
g3 = l3/2;

%masses [kg]
m0 = 17;
m1 = 10;
m2 = 9;
m3 = 3;
m_payload=0.5;

% Inertias [kg*m^2]
I0_xy = (1/4)*m0*r0^2 + (1/12)*m0*l0^2;
I0_z  = (1/2)*m0*r0^2;
I1_yz = (1/4)*m1*r123^2 + (1/12)*m1*l1^2;
I1_x  = (1/2)*m1*r123^2;
I2_yz = (1/4)*m2*r123^2 + (1/12)*m2*l2^2;
I2_x  = (1/2)*m2*r123^2;
I3_yz = (1/4)*m3*r123^2 + (1/12)*m3*l3^2;
```

```

I3_x = (1/2)*m3*r123^2;

%% --- MOTORS PARAMETERS ---
motor.A_lim = 600; % max motor acc [rad/s^2]
motor.V_lim = 600; % max motor vel [rad/s]
motor.D_lim = 600; % max motor dec[rad/s^2]

% Servo motor HG-KN 23 (B)J (brake+resolver)
Cn =0.64; % Nominal Torque [Nm]
mot_inertia = 0.247*10^-4; % [kg*m^2]
rid = 1/100;
alpha = Cn^2/mot_inertia; % Accererating Factor
%% --- JOINTS PARAMETERS ---
motor.A = motor.A_lim * rid; % max joint acc[rad/s^2]
motor.V = motor.V_lim * rid; % max joint vel [rad/s]
motor.D = motor.D_lim * rid; % max joint dec[rad/s^2]

L = [l0; l1; l2; l3; g1; g2; g3];

% Masses diagonal matrix
M=diag([m_payload, m_payload,m_payload, m3, m3, m3, I3_yz,...
        m2, m2, m2, I2_yz, m1, m1, m1, I1_yz, m0, m0, ...
        I0_z]);

%% External forces vector

Fse = zeros(19,1); % extended version of external forces
Fse(2) = -m_payload*9.81; % Fz payload
Fse(5) = -m3*9.81; % Fz m3
Fse(9) = -m2*9.81; % Fz m2
Fse(13) = -m1*9.81; % Fz m1
Fse(17) = -m0*9.81; % Fz m0

nJ = 4; % n. of Joints

%% --- Waypoints on the S-space---

A = [0.25; 0.00; 0.05; deg2rad(0)];
D = [0.35; 0.00; 0.20; deg2rad(0)];
E = [0.50; 0.15; 0.20; deg2rad(0)];

%% Sampling time
dt = 0.001;

%% Calculate minimum actuation time
% Inverse kinematics for A, D, E (elbow-up)
QA = ROBOTinv_4R(A, L, -1);
QD = ROBOTinv_4R(D, L, -1);
QE = ROBOTinv_4R(E, L, -1);

dq_AD = abs(QD - QA); % A -> D
dq_DE = abs(QE - QD); % D -> E
dq_ED = abs(QD - QE); % E -> D
dq_DA = abs(QA - QD); % D -> A

dQ=[dq_AD;dq_DE;dq_ED;dq_DA];
dQ_abs= abs(dQ);

rise_time=zeros(4,size(dQ,2));
for i=1:size(dQ,2)
    rise_time(1,i)=min_rise_time(motor,dQ_abs(1,i));
    rise_time(2,i)=min_rise_time(motor,dQ_abs(2,i));
    rise_time(3,i)=min_rise_time(motor,dQ_abs(3,i));
    rise_time(4,i)=min_rise_time(motor,dQ_abs(4,i));
end

```



```

T_min_act= max(rise_time);
disp('Min Actuation Time:');
disp(T_min_act);

%% --- S-Shape time configuration ---
T_AD = T_min_act * 1.1; % A->D line [s]
T_DE = T_min_act * 1.1; % D->E arch [s]
T_ED = T_min_act * 1.1; % E->D arch [s]
T_DA = T_min_act * 1.1; % D->A line [s]

t_AD = 0:dt:T_AD;
t_DE = 0:dt:T_DE;
t_ED = 0:dt:T_ED;
t_DA = 0:dt:T_DA;

N_AD = length(t_AD);
N_DE = length(t_DE);
N_ED = length(t_ED);
N_DA = length(t_DA);

% S-Shape profile wrt time
ta_AD = T_AD/3; tb_AD = 2*T_AD/3; tc_AD = T_AD;
ta_DE = T_DE/3; tb_DE = 2*T_DE/3; tc_DE = T_DE;
ta_ED = T_ED/3; tb_ED = 2*T_ED/3; tc_ED = T_ED;
ta_DA = T_DA/3; tb_DA = 2*T_DA/3; tc_DA = T_DA;

% Global time & S-vectors initialization
S_all = []; % [4 x Nt]
Sdot_all = [];
Sddot_all = [];
t_all = [];

%% ===== 1) A -> D (LINE)=====

S_start = A;
S_end = D;
dS = S_end - S_start;

for k = 1:N_AD
    t = t_AD(k);

    rx = Sshape(t, S_start(1), dS(1), ta_AD, tb_AD, tc_AD);
    ry = Sshape(t, S_start(2), dS(2), ta_AD, tb_AD, tc_AD);
    rz = Sshape(t, S_start(3), dS(3), ta_AD, tb_AD, tc_AD);
    rtet = Sshape(t, S_start(4), dS(4), ta_AD, tb_AD, tc_AD);

    S_now = [rx.pos; ry.pos; rz.pos; rtet.pos];
    Sdot_now = [rx.vel; ry.vel; rz.vel; rtet.vel];
    Sddot_now = [rx.acc; ry.acc; rz.acc; rtet.acc];

    S_all = [S_all, S_now];
    Sdot_all = [Sdot_all, Sdot_now];
    Sddot_all = [Sddot_all, Sddot_now];
end
t_all = [t_all, t_AD];

%% ===== 2) D -> E (CIRCLE IN XY PLANE) =====

% Select the centre of the circle in XY plane
Cx = 0.50; % [m]
Cy = 0.00; % [m]

% Compute the Radius
R = sqrt( (D(1)-Cx)^2 + (D(2)-Cy)^2 );

% Polar coordinates
phi0 = atan2(D(2)-Cy, D(1)-Cx); % phi wrt D
phi1 = atan2(E(2)-Cy, E(1)-Cx); % phi wrt E

```

```

dphi_DE = phi1 - phi0; % total phi excursion

% Z and Teta (Linear)
dz_DE = E(3) - D(3);
dthet_DE = E(4) - D(4);

% S-Shape application to D->E
for k = 2:N_DE
    t = t_DE(k);

    rphi = Sshape(t, phi0, dphi_DE, ta_DE, tb_DE, tc_DE);

    phi = rphi.pos;
    phi_p = rphi.vel;
    phi_pp = rphi.acc;

    % Back to XY coordinates
    x = Cx + R*cos(phi);
    y = Cy + R*sin(phi);

    x_p = -R*sin(phi)*phi_p;
    y_p = R*cos(phi)*phi_p;

    x_pp = -R*cos(phi)*phi_p^2 - R*sin(phi)*phi_pp;
    y_pp = -R*sin(phi)*phi_p^2 + R*cos(phi)*phi_pp;

    rz = Sshape(t, D(3), dz_DE, ta_DE, tb_DE, tc_DE);
    rtet = Sshape(t, D(4), dthet_DE, ta_DE, tb_DE, tc_DE);

    S_now = [x; y; rz.pos; rtet.pos];
    Sdot_now = [x_p; y_p; rz.vel; rtet.vel];
    Sddot_now = [x_pp; y_pp; rz.acc; rtet.acc];

    S_all = [S_all, S_now];
    Sdot_all = [Sdot_all, Sdot_now];
    Sddot_all = [Sddot_all, Sddot_now];
end
t_all = [t_all, t_DE(2:end) + t_all(end)];

%% ===== 3) E-> D (CIRCLE IN XY PLANE) =====

```

```

dphi_ED = phi0 - phi1;

dz_ED = D(3) - E(3);
dthet_ED = D(4) - E(4);

for k = 2:N_ED
    t = t_ED(k);

    rphi = Sshape(t, phi1, dphi_ED, ta_ED, tb_ED, tc_ED);

    phi = rphi.pos;
    phi_p = rphi.vel;
    phi_pp = rphi.acc;

    x = Cx + R*cos(phi);
    y = Cy + R*sin(phi);

    x_p = -R*sin(phi)*phi_p;
    y_p = R*cos(phi)*phi_p;

    x_pp = -R*cos(phi)*phi_p^2 - R*sin(phi)*phi_pp;
    y_pp = -R*sin(phi)*phi_p^2 + R*cos(phi)*phi_pp;

    rz = Sshape(t, E(3), dz_ED, ta_ED, tb_ED, tc_ED);
    rtet = Sshape(t, E(4), dthet_ED, ta_ED, tb_ED, tc_ED);

    S_now = [x; y; rz.pos; rtet.pos];
    Sdot_now = [x_p; y_p; rz.vel; rtet.vel];

```

```

Sddot_now = [x_pp; y_pp; rz.acc; rtet.acc];

S_all = [S_all, S_now];
Sdot_all = [Sdot_all, Sdot_now];
Sddot_all = [Sddot_all, Sddot_now];
end
t_all = [t_all, t_ED(2:end) + t_all(end)];

%% ===== 4) D -> A (LINEAR) =====

S_start = D;
S_end = A;
dS = S_end - S_start;

for k = 2:N_DA
    t = t_DA(k);

    rx = Sshape(t, S_start(1), dS(1), ta_DA, tb_DA, tc_DA);
    ry = Sshape(t, S_start(2), dS(2), ta_DA, tb_DA, tc_DA);
    rz = Sshape(t, S_start(3), dS(3), ta_DA, tb_DA, tc_DA);
    rtet = Sshape(t, S_start(4), dS(4), ta_DA, tb_DA, tc_DA);

    S_now = [rx.pos; ry.pos; rz.pos; rtet.pos];
    Sdot_now = [rx.vel; ry.vel; rz.vel; rtet.vel];
    Sddot_now = [rx.acc; ry.acc; rz.acc; rtet.acc];

    S_all = [S_all, S_now];
    Sdot_all = [Sdot_all, Sdot_now];
    Sddot_all = [Sddot_all, Sddot_now];
end
t_all = [t_all, t_DA(2:end) + t_all(end)];

Nt = length(t_all);

%% ===== INVERSE KINEMATICS & DYNAMICS =====

q_all = zeros(nJ, Nt);
qd_all = zeros(nJ, Nt);
qdd_all = zeros(nJ, Nt);

q1t_save = zeros(1,Nt);
q2t_save = zeros(1,Nt);
q3t_save = zeros(1,Nt);
q4t_save = zeros(1,Nt);
qt_save = zeros(4,Nt);
q1tm_save = zeros(1,Nt);
q2tm_save = zeros(1,Nt);
q3tm_save = zeros(1,Nt);
q4tm_save = zeros(1,Nt);

for k = 1:Nt

    S = [ S_all(1,k);
          S_all(2,k);
          S_all(3,k);
          S_all(4,k) ];

    Sp = Sdot_all(:,k);
    Spp = Sddot_all(:,k);

    % INVERSE KINEMATICS
    Q = ROBOTinv_4R(S, L, -1); % Elbow-up

    % --- Joint limits check at time step k ---
    if any(Q < joint_limits(:,1) | Q > joint_limits(:,2))
        error("ERROR: configuration at time step %d violates joint limits.", k);
    end
end

```

```

J      = ROBOTjac_4R(Q, L);
Qp     = J \ Sp;
Jdot   = ROBOTjacP_4R(Q, Qp, L);
Qpp    = J \ (Spp - Jdot*Qp);

q_all(:,k)   = Q;
qd_all(:,k)  = Qp;
qdd_all(:,k) = Qpp;

% Dynamic Jacobian
J_din      = ROBOTjacdin_4R(Q, L);
J_dot_din  = ROBOTjacPdin_4R(Q, Qp, L);

% Extended accelerations
Sdpp = J_dot_din*Qp + J_din*Qpp;

% Inertial Forces
Fsi   = -M*Sdpp;

% Total Forces
Fs    = Fse + Fsi;

% Joint Torques
Fcq   = -J_din.' * Fs;

q1t_save(k) = Fcq(1);
q2t_save(k) = Fcq(2);
q3t_save(k) = Fcq(3);
q4t_save(k) = Fcq(4);
qt_save(:,k) = Fcq;

app_1(k) = q1t_save(k) * qdd_all(1,k);
app_2(k) = q2t_save(k) * qdd_all(2,k);
app_3(k) = q3t_save(k) * qdd_all(3,k);
app_4(k) = q4t_save(k) * qdd_all(4,k);
app(:,k) = Fcq .* qdd_all(:,k);

% Motor Torques
q1tm_save(k) = q1t_save(k)*rid + mot_inertia*qdd_all(1,k)*(1/rid);
q2tm_save(k) = q2t_save(k)*rid + mot_inertia*qdd_all(2,k)*(1/rid);
q3tm_save(k) = q3t_save(k)*rid + mot_inertia*qdd_all(3,k)*(1/rid);
q4tm_save(k) = q4t_save(k)*rid + mot_inertia*qdd_all(4,k)*(1/rid);
end

% Load Factor check
beta = zeros(1,nJ);
rid_opt = zeros(1,nJ);
rid_max = zeros(1,nJ);
rid_min = zeros(1,nJ);

for p=1:nJ

    wrq(p) = rms(qd_all(p,:));
    dwrq(p) = rms(qdd_all(p,:));
    Crsq(p) = rms(qt_save(p,:));
    dwCm(p) = mean(app(p,:));

    rid_opt = sqrt(mot_inertia*dwrq(p)/Crsq(p));

    beta(p) = ((rid/sqrt(mot_inertia)) * Crsq(p) - (sqrt(mot_inertia)/rid) * wrq(p))^2 +
2*(Crsq(p)*dwrq(p) + dwCm(p));

    rid_max(p) = (sqrt(mot_inertia)*(sqrt(alpha-beta(p)+4*dwrq(p)*Crsq(p))...
+ sqrt(alpha - beta(p))))/(2*Crsq(p));

    rid_min(p) = (sqrt(mot_inertia)*(sqrt(alpha-beta(p)+4*dwrq(p)*Crsq(p))...
- sqrt(alpha - beta(p))))/(2*Crsq(p));

```

```

end

figure; hold on; grid on;

joints = 1:4;

% beta points
scatter(joints, beta, 80, 'b', 'filled');

% Threshold alpha
yline(alpha, 'r--', 'LineWidth', 1.8);

xlabel('Joint index');
ylabel('\beta');
title('Motor Sizing Check: \beta values vs. \alpha threshold');
xticks(joints);

legend('\beta_j', '\alpha threshold');

if all(beta < alpha)
    fprintf("All joints satisfy the motor sizing condition  $\alpha > \beta$ .\n");
else
    fprintf("WARNING: Some joints violate  $\alpha > \beta$ .\n");
    idx = find(beta >= alpha);
    fprintf("Joints which do not respect the condition: ");
    fprintf("%d ", idx);
    fprintf("\n");
end

figure; hold on; grid on;

joints = 1:4;

% Plot rid_min
scatter(joints, rid_min, 90, 'g', 'filled');

% Plot rid_max
scatter(joints, rid_max, 90, 'm', 'filled');

% rid
yline(rid, 'b-', 'LineWidth', 1.6);

xlabel('Joint index');
ylabel('Reduction ratio r_{id}');
title('Motor Reduction Ratio Check');
xticks(joints);

legend('r_{id}^{min}', 'r_{id}^{max}', 'chosen r_{id}', ...
    'Location', 'best');

%% --- Kinematics quality index k^{-1} along trajectory ---

nt = numel(t_all);
Kinv = zeros(1, nt);

for k = 1:nt
    Qk = q_all(:,k);

    J = ROBOTjac_4R(Qk, L);

    Jv = J(1:3,:);

    s = svd(Jv);
    smax = max(s);
    smin = min(s);

    if smax < 1e-8

```

```

        Kinv(k) = 0;
    else
        Kinv(k) = smin/smax;
    end
end

% Results
fprintf('\n=== Quality Index results ===\n');
fprintf('k^{-1}_min = %.4f\n', min(Kinv));    % min K^{-1}
fprintf('k^{-1}_max = %.4f\n', max(Kinv));    % max K^{-1}

figure;
plot(t_all, Kinv, 'LineWidth', 1.5);
grid on;
xlabel('Time [s]');
ylabel('k^{-1}');
title('Quality Index vs Time');

% K^{-1} threshold
thr_primary = 0.2;
ylines(thr_primary, '--r', 'Primary threshold');
legend('k^{-1}(t)', 'thr_{primary}');

figure; % Pos
sgtitle('Joint Positions');

for j = 1:4
    subplot(2,2,j); hold on; grid on;
    plot(t_all, rad2deg(q_all(j,:)), 'LineWidth', 1.2);
    xlabel('t [s]');
    ylabel('q [deg]');
    title(sprintf('Joint %d - Position', j));
end

figure; % Vel
sgtitle('Joint Velocities');

for j = 1:4
    subplot(2,2,j); hold on; grid on;
    plot(t_all, qd_all(j,:), 'LineWidth', 1.2);
    xlabel('t [s]');
    ylabel('q\dot [rad/s]');
    title(sprintf('Joint %d - Velocity', j));
end

figure; % Acc
sgtitle('Joint Accelerations');

for j = 1:4
    subplot(2,2,j); hold on; grid on;
    plot(t_all, qdd_all(j,:), 'LineWidth', 1.2);
    xlabel('t [s]');
    ylabel('q\ddot [rad/s^2]');
    title(sprintf('Joint %d - Acceleration', j));
end

figure;
subplot(2,2,1); plot(t_all,q1t_save); xlabel('Time [s]'); ylabel('T\Joint [Nm]'); grid on; title('q1 torque');
subplot(2,2,2); plot(t_all,q2t_save); xlabel('Time [s]'); ylabel('T\Joint [Nm]'); grid on; title('q2 torque');
subplot(2,2,3); plot(t_all,q3t_save); xlabel('Time [s]'); ylabel('T\Joint [Nm]'); grid on; title('q3 torque');
subplot(2,2,4); plot(t_all,q4t_save); xlabel('Time [s]'); ylabel('T\Joint [Nm]'); grid on; title('q4 torque');

figure;

```

```

subplot(2,2,1); plot(t_all,q1tm_save); xlabel('Time [s]'); ylabel('T\_Motor [Nm]'); grid on; title('q1
motor torque');
subplot(2,2,2); plot(t_all,q2tm_save); xlabel('Time [s]'); ylabel('T\_Motor [Nm]'); grid on; title('q2
motor torque');
subplot(2,2,3); plot(t_all,q3tm_save); xlabel('Time [s]'); ylabel('T\_Motor [Nm]'); grid on; title('q3
motor torque');
subplot(2,2,4); plot(t_all,q4tm_save); xlabel('Time [s]'); ylabel('T\_Motor [Nm]'); grid on; title('q4
motor torque');

%% --- TRAJECTORY IN S-SPACE ---

figure;
hold on; grid on;

plot3(S_all(1,:), S_all(2,:), S_all(3,:), 'b', 'LineWidth', 1.5);

% waypoint A-D-E-D-A
W = [A, D, E, D, A];
plot3(W(1,:), W(2,:), W(3,:), 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 6);

xlabel('X [m]');
ylabel('Y [m]');
zlabel('Z [m]');
title('Traiettorie spazio operativo 3D');
axis equal;
view(3);

%% ANIMATION

f1 = figure;
f1.WindowStyle='normal';
hold on; grid on; view(3);

l11=line('XData', [0 0 0], ...
        'YData', [0 0 0], ...
        'ZData', [0 0 0], ...
        'linestyle', '-', 'linewidth', 2, 'color', 'r', ...
        'marker', 'o', 'markersize', 6, 'markerfacecolor', 'k');

trail = plot3(NaN,NaN,NaN, 'LineWidth', 1.5);
trail_X = []; trail_Y = []; trail_Z = [];

for k = 1:size(q_all,2)
    PlotRobot2(q_all(:,k), L, l0, 'r', f1, l11);

    Se = ROBOTdir_4R(q_all(:,k), L); % [X;Y;Z;theta]
    Xe = Se(1)*1e3;
    Ye = Se(2)*1e3;
    Ze = Se(3)*1e3;

    trail_X(end+1) = Xe;
    trail_Y(end+1) = Ye;
    trail_Z(end+1) = Ze;

    set(trail, 'XData', trail_X, 'YData', trail_Y, 'ZData', trail_Z);
    drawnow;
end

%% --Simscape utilities--
tt=t_all;

% Signal Editor

% Joint 1
ww1=qd_all(1,:);
signal1 = Simulink.SimulationData.Signal;
signal1.Name = 'vel1_task2';
signal1.Values = timeseries(ww1*(1/rid), tt);
signal1.Values = setinterpmethod(signal1.Values, 'linear');

```

```

ds1_task2 = Simulink.SimulationData.Dataset;
ds1_task2 = addElement(ds1_task2, signal1, 'velocity');
save('vel1_task2.mat', 'ds1_task2');

% Joint 2
ww2=qd_all(2,:);
signal2 = Simulink.SimulationData.Signal;
signal2.Name = 'vel2_task2';
signal2.Values = timeseries(ww2*(1/rid), tt);
signal2.Values = setinterpmethod(signal2.Values,'linear');
ds2_task2 = Simulink.SimulationData.Dataset;
ds2_task2 = addElement(ds2_task2, signal2, 'velocity');
save('vel2_task2.mat', 'ds2_task2');

% Joint 3
ww3=qd_all(3,:);
signal3 = Simulink.SimulationData.Signal;
signal3.Name = 'vel3_task2';
signal3.Values = timeseries(ww3*(1/rid), tt);
signal3.Values = setinterpmethod(signal3.Values,'linear');
ds3_task2 = Simulink.SimulationData.Dataset;
ds3_task2 = addElement(ds3_task2, signal3, 'velocity');
save('vel3_task2.mat', 'ds3_task2');

% Joint 4
ww4=qd_all(4,:);
signal4 = Simulink.SimulationData.Signal;
signal4.Name = 'vel4_task2';
signal4.Values = timeseries(ww4*(1/rid), tt);
signal4.Values = setinterpmethod(signal4.Values,'linear');
ds4_task2 = Simulink.SimulationData.Dataset;
ds4_task2 = addElement(ds4_task2, signal4, 'velocity');
save('vel4_task2.mat', 'ds4_task2');

% START SIMULATION
sim('Simscape_task2_motors.slx');

% To-Workspace
figure;
sgtitle('Simscape Motor Torques');
subplot(2,2,1); plot(MotorTorque1t2.Time, MotorTorque1t2.Data,'color','r'); grid on; xlabel('Time [s]');
ylabel('T\_motor [Nm]'); title('q1 motor torque'); ylim([-0.09 0.09]);
subplot(2,2,2); plot(MotorTorque2t2.Time, MotorTorque2t2.Data,'color','r'); grid on; xlabel('Time [s]');
ylabel('T\_motor [Nm]'); title('q2 motor torque');
subplot(2,2,3); plot(MotorTorque3t2.Time, MotorTorque3t2.Data,'color','r'); grid on; xlabel('Time [s]');
ylabel('T\_motor [Nm]'); title('q3 motor torque');
subplot(2,2,4); plot(MotorTorque4t2.Time, MotorTorque4t2.Data,'color','r'); grid on; xlabel('Time [s]');
ylabel('T\_motor [Nm]'); title('q4 motor torque');

figure; % Matlab/SIMSCAPE comparison
sgtitle('MATLAB-Simscape Motor Torques comparison');
subplot(2,2,1); plot(MotorTorque1t2.Time, MotorTorque1t2.Data,'color','r'); grid on; xlabel('Time [s]');
ylabel('T\_motor [Nm]'); title('q1 motor torque'); ylim([-0.09 0.09]); hold on;
subplot(2,2,2); plot(MotorTorque2t2.Time, MotorTorque2t2.Data,'color','r'); grid on; xlabel('Time [s]');
ylabel('T\_motor [Nm]'); title('q2 motor torque'); hold on;
subplot(2,2,3); plot(MotorTorque3t2.Time, MotorTorque3t2.Data,'color','r'); grid on; xlabel('Time [s]');
ylabel('T\_motor [Nm]'); title('q3 motor torque'); hold on;
subplot(2,2,4); plot(MotorTorque4t2.Time, MotorTorque4t2.Data,'color','r'); grid on; xlabel('Time [s]');
ylabel('T\_motor [Nm]'); title('q4 motor torque'); hold on;
subplot(2,2,1); plot(t_all,q1tm_save,'color','b'); xlabel('Time [s]'); grid on; ylabel('T\_motor [Nm]');
title('q1 motor torque');
subplot(2,2,2); plot(t_all,q2tm_save,'color','b'); xlabel('Time [s]'); grid on; ylabel('T\_motor [Nm]');
title('q2 motor torque');
subplot(2,2,3); plot(t_all,q3tm_save,'color','b'); xlabel('Time [s]'); grid on; ylabel('T\_motor [Nm]');
title('q3 motor torque');
subplot(2,2,4); plot(t_all,q4tm_save,'color','b'); xlabel('Time [s]'); grid on; ylabel('T\_motor [Nm]');
title('q4 motor torque');

```